arXiv:2605.15871v2 [cs.AI] 7 Oct 2026

Agentic Discovery of Neural Architectures: AIRA-Compose and AIRA-Design

Alberto Pepe[1,*], **Chien-Yu Lin**[1,*], **Despoina Magka**[1], **Bilge Acun**[1], **Thomas Simon Foster**[1,2], **Yannan Nellie Wu**[1], **Anton Protopopov**[1], **Carole-Jean Wu**[1,‡], **Yoram Bachrach**[1,‡]

[1]FAIR at Meta, [2]University of Oxford
[*]Joint first author, [‡]Joint last author

As a step toward recursive self-improvement, we investigate the ability of LLM agents to autonomously design foundation models beyond the standard Transformer paradigm. We introduce a dual-framework approach: **AIRA-Compose** for high-level architecture search, and **AIRA-Design** for low-level mechanistic implementation.

AIRA-Compose deploys an ensemble of 11 agents to navigate a combinatorial design space of fundamental computational primitives (Attention, MLP, Mamba) under a fixed 24-hour compute budget. Operating in two stages, agents iteratively design and evaluate candidates at the million-parameter scale, after which top-performing designs are extrapolated to 350M, 1B, and 3B parameter scales. This search yields 14 novel architectures spanning two families: *AIRAformers* (Transformer-based) and *AIRAhybrids* (Transformer-Mamba-based). When pre-trained at the 1B scale under a fixed token budget, agent-discovered top-performing architectures consistently outperform both Llama 3.2 and Composer-found alternatives. On downstream tasks, AIRAformer-D and AIRAhybrid-D improve accuracy by 2.4% and 3.8% over Llama 3.2, respectively. AIRA-Compose also finds novel model architectures that achieve steeper, more efficient compute-optimal scaling frontiers. AIRAformer-C scales 54% and 71% faster than Llama 3.2 and the best Composer-found Transformer, while AIRAhybrid-C scales 23% and 37% faster than the modified Nemotron-2 and the best Composer-found hybrid, respectively.

AIRA-Design tasks up to 20 agents with directly writing novel attention mechanisms to handle long-range dependencies and implementing high-performing training scripts. Evaluated on the Long Range Arena (LRA) benchmark, the best agent-designed architectures achieve accuracy within 2.3% of human state-of-the-art on document matching and 2.6% on text classification. On the Autoresearch benchmark, Greedy Opus 4.5 optimizes training under a fixed time budget to achieve 0.968 validation bits-per-byte, surpassing the published minimum reference.

Together, AIRA-Compose and AIRA-Design demonstrate that AI research agents can autonomously discover hybrid architectures and algorithmic optimizations that rival or surpass hand-designed baselines. This establishes a flexible, powerful paradigm for discovering the next generation of foundation models and a step towards recursive self improvement.

Date: October 8, 2026
Correspondence: Despoina Magka at despoinam@meta.com, Yoram Bachrach at yorambac@meta.com Meta

1 Introduction

Agents powered by Large Language Models (LLMs) are radically transforming scientific research and have evolved into autonomous systems capable of executing end-to-end research loops (Wang et al., 2024; Andrews et al., 2025; Schmidgall et al., 2025; Yamada et al., 2025). Today, agents can independently formulate hypotheses, write and execute code, evaluate results, and iteratively refine their approaches to solve complex tasks. These include discovering new mathematical constructions (Romera-Paredes et al., 2024; Novikov et al., 2025), achieving human-level performance in competitive programming (Leblond et al., 2023; Li et al., 2024b; Jimenez et al., 2024), replicating existing AI research literature (Xiang et al., 2025; Starace et al., 2025), designing and generating correct, faster, and more efficient machine learning (ML) kernels for custom AI hardware (Liao et al., 2026; Hammond et al., 2025), and driving open-ended ML discoveries (Zhao et al.,

2025; Nathani et al., 2025; Lupidi et al., 2026).

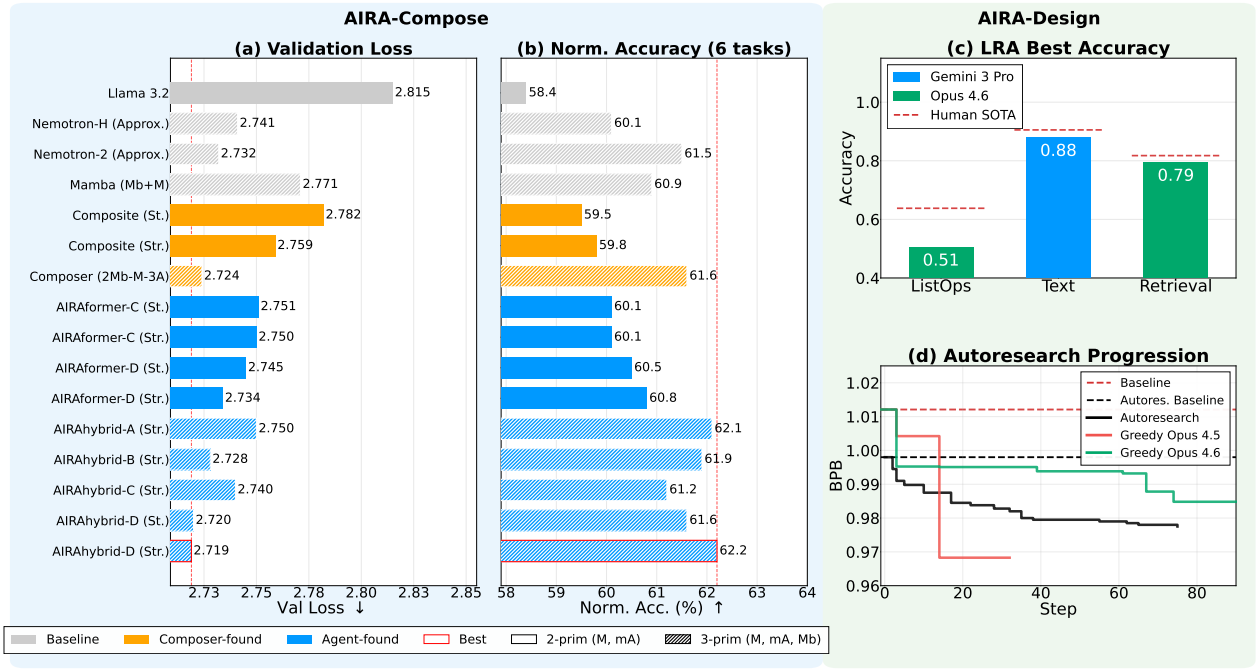


Figure 1 AIRA-Compose and AIRA-Design: agentic frameworks for neural architecture search and model design. (a–c) Downstream evaluations of selected agent-found architectures scaled-up to 1B scale with fixed token budget, alongside baselines and traditional NAS-found models: (a) validation loss, and (b) zero-shot average normalized accuracy across 6 tasks. **(c)** Best test accuracy after 24 GPU hours on the three Long Range Arena tasks. Greedy Opus 4.6 achieves the highest scores on ListOps (0.51) and Retrieval (0.79); Greedy Gemini 3 Pro leads on Text (0.88). **(d)** Autoresearch training-script optimization: cumulative best bits-per-byte (BPB) over agent steps. Greedy Opus 4.6 achieves the lowest BPB across 100 runs.

A compelling frontier in agentic research is **Recursive Self-Improvement (RSI)** (Good, 1966; Schmidhuber, 1987, 2003). In this manuscript, we will use RSI to refer to agents that autonomously discover and optimize the very neural architectures that power them, ultimately advancing their capabilities. Most LLMs today rely on the Transformer architecture (Vaswani et al., 2017; Radford et al., 2018; Devlin et al., 2019), which is characterized by a 1:1 interleaving of multi-head attention and multi-layer perceptron (MLP) layers. Although Transformers remain the gold standard, the research community is increasingly shifting toward post-Transformer paradigms (Peng et al., 2023a; Sun et al., 2023; Gu and Dao, 2024; Rahmani et al., 2025) to address their inherent limitations, such as the quadratic complexity of attention and the memory cost of key-value (KV) caching during inference (Tay et al., 2022; Pope et al., 2023). Moreover, arranging distinct computational primitives into sophisticated hybrid patterns has been shown to yield more efficient and performant foundation models (Adler et al., 2024; Blakeman et al., 2025; Lieber et al., 2024; Yang et al., 2025; Singhal et al., 2025; Alexiuk et al., 2026; Bae et al., 2025). We refer to such models as *hybrid* LLMs. Model design has historically been driven by domain expertise and human intuition. However, as we move toward hybrid models, the design space becomes vast and combinatorial (Liu et al., 2021b; Ren et al., 2021). Relying solely on manual exploration means that highly performant, non-obvious configurations or entirely new computational primitives may be overlooked. We hence argue that **Neural Architecture Search (NAS)** represents an ideal candidate for AI research agents: agents pair educated, context-aware hypotheses with robust automated search and iterative refinement, allowing to explore the design space systematically and propose fundamentally new architectures.

We propose two approaches for the agentic discovery of neural architectures for future hybrid LLMs:

- **AIRA-Compose, or high-level architecture search**: Agents are tasked to find new model architectures based on *predefined* computational primitives, by searching and evaluating model architecture candidates at small scale. We do so by recasting the small scale model architecture exploration of the *Composer*

framework (Acun et al., 2025) into an agentic task. Only top-performing model architectures are scaled up.

- **AIRA-Design, or low-level mechanistic design**: Agents implement *novel* computational primitives and train them efficiently. These models are crafted to tackle the Long Range Arena (LRA) (Tay et al., 2020) and Autoresearch benchmarks (Karpathy, 2026).

The two approaches represent complementary perspectives on a single goal. The former relies on predefined computational primitives, restricting the agents' freedom exclusively to the optimization of their arrangement. The latter is an open-ended code-generation task in which agents must implement fundamentally new models from scratch. Both methodologies are implemented as AIRS-Bench *tasks* (Lupidi et al., 2026), which evaluate an agent's ability to conduct independent scientific research. AIRS-Bench introduces a flexible and scalable structure that enables virtually any ML problem to be cast into a format that agents can understand and operate on. We list contributions below:

- We introduce AIRA-Compose and AIRA-Design — two flexible, agent-based discovery frameworks built on the AIRS-Bench task standard to enable Recursive Self-Improvement. These frameworks introduce 12 agentic tasks with a scalable structure for autonomous research loops. They can be readily extended to operate with different agent harnesses and/or use different reasoning models.
- We demonstrate that our agents can autonomously discover novel, highly performant hybrid architectures by searching and optimizing the arrangement of predefined computational primitives. We categorize them into two families: *AIRAformers*, which include multi-head attention and MLP blocks, and *AIRAhybrids*, which also include Mamba2 State Space Model (SSM) blocks. Both families are characterized by original interleavings of such primitives. When extrapolated to 350M, 1B, and 3B parameter scales, these agent-discovered models exhibit superior isoFLOP scaling properties and consistently outperform established baselines, such as, Llama (Llama Team, 2024) and Nemotron (Blakeman et al., 2025), as well as those found via optimization-based NAS.
- We show that our agents are capable of autonomously engineering high-performing attention mechanisms. On the Long Range Arena (LRA) benchmark, agent-designed models achieved a peak accuracy within 2.3*pp* of human SOTA on document matching (82%) and 2.6*pp* on text classification (91%). Four of our agents achieve an average normalized score above 0.3 across the 3 tasks, with 1 corresponding to human SOTA.
- We show that our agents are capable of iteratively improving the efficiency of small language models training loops. On the open-ended Autoresearch task, our best agent surpassed the published Autoresearch reference minimum by achieving a validation BPB of 0.968. Moreover, we could reproduce a delta with respect to baseline larger than that reported in Karpathy (2026) in 20% of the experiments.

We summarize them in Figure 1. Panels (a–b) present downstream evaluations of selected agent-found architectures scaled-up to 1B scale with a fixed token budget, alongside baselines and traditional NAS-found models: (a) validation loss and (b) zero-shot average normalized accuracy across 6 tasks. Agent-discovered AIRAformer and AIRAhybrid models consistently outperform both established baselines (Llama 3.2, approximated Nemotron-2) and Composer-found alternatives across all three metrics.

Panels (c) and (d) present results from AIRA-Design, where agents must write functional code from scratch rather than arrange predefined blocks. On the Long Range Arena benchmark (c), agents implement novel sub-quadratic attention mechanisms and train them within a fixed GPU budget; the best agent-designed models reach accuracy within 2–3 percentage points of human SOTA across all three tasks. On the Autoresearch benchmark (d), agents iteratively optimize a GPT training script to minimize validation bits-per-byte within a 5-minute wall-clock budget; augmenting the strongest agents with curated literature and code repositories shifts their optimization strategies and yields the lowest BPB across all 100 runs.

The rest of the paper is structured as follows: fundamentals on the AIRA-dojo harness and the AIRS-Bench task structure are described in Section 2; details on AIRA-Compose and AIRA-Design tasks, their objective and datasets employed are presented in Sections 3 and 4, respectively; the experimental setup and relevant metrics are introduced in Section 5; results are presented in Section 6 and grouped into 4 subsections: NAS on two (Section 6.1.1) and three (Section 6.1.2) computational primitives within AIRA-Compose, Long

Range Arena (Section 6.2.1) and Autoresearch (Section 6.2.2) within AIRA-Design; conclusions are drawn in Section 7. A review of related work is provided in Appendix A.

2 Methodology

We build on the definitions and evaluation framework established by AIRS-Bench (Lupidi et al., 2026). Because this infrastructure has been extensively detailed in prior work, we provide only a brief overview. We adopt the definition of an *agent* as the combination of an LLM and a *scaffold*. The scaffold represents the algorithmic search policy and the specific set of operators that determine how the agent explores the solution space. This scaffold is instantiated within a *harness*, the system that manages the agent's environment, execution, and tool access.

We utilize AIRA-dojo (Toledo et al., 2025), a harness designed to evolve code solutions through tree-based exploration. AIRA-dojo guides the agent using structured search policies (e.g., greedy search or Monte Carlo Tree Search) and interacts with candidate Python solutions using four operators:

- *Draft:* Generates the initial set of candidate solutions.
- *Debug:* Identifies and corrects execution or logical errors.
- *Improve:* Refines working solutions to maximize (or minimize) the target evaluation metric.
- *Analyze:* Reads and analyzes the agent solution at each step.

We employ one-shot and greedy scaffolds. The one-shot scaffold corresponds to calling the *draft* operator once, and exactly one solution will be produced per run. The greedy scaffold, on the other hand, explores several solutions through a tree-based search policy, and it starts the search by drafting 5 initial solutions through the *draft* operator. The *improve* operator is applied onto the agent solution that achieves the highest validation fitness. A layer is populated until a new best is found. The *debug* operator is applied whenever the *analyze* operator deems a solution buggy (e.g., the agent produces an architecture with an incorrect number of primitives or an OOM error is raised). Each greedy run will explore several solutions, or "steps", ranging from tens to hundreds per run, based on the compute required by the evaluation script of each task. Each step of the search has a validation fitness, which comes from the agent-generated code to support its submission (i.e., an arrangement of primitives for AIRA-Compose tasks, or a `model.py/train.py` for AIRA-Design tasks), and a test fitness, which is obtained by independently evaluating the agent submission through an independent evaluation script (see below).

Both our architecture search and mechanistic design approaches are formulated as AIRS-Bench tasks (see Table 1). An AIRS-Bench task is fully specified by a {problem, dataset, metric} triplet: The **problem** defines the challenge to be solved (e.g., Neural Architecture Search); the **dataset** specifies which data to solve the challenge over (e.g., DCLM); the **metric** is used to quantify fitness performance (e.g., loss).

AIRS-Bench tasks have a standardized, modular file structure that translates open-ended ML research problems into a format parsable by agentic harnesses. A standard task directory consists of the following components:

- `project_description.md`: The prompt provided to the agent. It details the research objective, the dataset schema, and the specific evaluation setup (e.g., instructing the agent what artifact to submit).
- `prepare.py` & `evaluate_prepare.py`: Scripts responsible for one-time data downloading and environment setup. They handle data sanitization, ensuring test labels are hidden while the agent is building its solution.
- `evaluate.py`: The isolated scoring script containing the ground-truth metric implementation used to automatically evaluate the agent's submission against the test set. For tasks in this manuscript, `evaluate.py` trains the agent-generated architecture or launches the training script produced.
- `metadata.yaml`: A configuration file defining the task constraints, evaluation metrics, required dataset splits, and any specific library dependencies needed for the evaluation environment.

Table 1 The 12 RSI tasks introduced in this manuscript. AIRA-Compose includes 4 tasks encompassing 3 datasets with two and three primitive pools, requiring agents to submit a `submission.csv` file containing a string of 16 primitives to describe an architecture. AIRA-Design tasks are split into LRA (Long Range Arena) for architectural implementation and Autoresearch for high-performing training scripts, requiring agents to submit a `model.py` or a `train.py` artifact, respectively.

Framework	Tasks (Problem, Dataset, Metric)	Submission Artifact

AIRA-Compose: High-level architecture search	**2 Primitives (M, mA) Search:**
	`NeuralArchitectureSearchMAD2PrimitivesAccuracy`
	`NeuralArchitectureSearchBabiStories2PrimitivesLoss`
	`NeuralArchitectureSearchDCLM2PrimitivesLoss`
	3 Primitives (M, mA, Mb) Search:
	`NeuralArchitectureSearchMAD3PrimitivesMbAccuracy`
		`submission.csv`
		`mh-attention mlp mlp`
		`mlp mh-attention mlp`
		`...`
		`mh-attention mlp`
		(16 primitives total)

AIRA-Design: Low-level mechanistic design	**LRA (Long Range Arena):**
	`LongContextReasoningLRATextAccuracy`
	`LongContextReasoningLRAListOpsAccuracy`
	`LongContextReasoningLRARetrievalAccuracy`
	`LongContextReasoningLRATextConfigurableAccuracy`
	`LongContextReasoningLRAListOpsConfigurableAccuracy`
	`LongContextReasoningLRARetrievalConfigurableAccuracy`
	Autoresearch:
	`AutoregressiveLanguageModellingAutoresearchBPB`
	`AutoregressiveLanguageModellingWithLiteratureAutoresearchBPB`
		`model.py / train.py`
		`import flax.linen as nn`
		`class CustomEncoder(`
		`nn.Module):`
		`@nn.compact`
		`def __call__(self, x):`
		`...`

Enhancing AIRS-Bench. The roles of `submission.csv` (i.e., the artifact that the agent is required to produce) and `evaluate.py` (i.e., how the artifact is evaluated) differ between our RSI tasks and standard AIRS-Bench tasks. In standard AIRS-Bench, `submission.csv` contains model predictions on test data (e.g., a regression quantity or predicted class). In our RSI tasks, `submission.csv` specifies a complete architecture or a training loop. Similarly, `evaluate.py` does not simply include metric calculations, like F1 scores or Spearman correlations as in AIRS-Bench, but it encapsulates full training pipelines ported directly from the Composer, LRA, and Autoresearch codebases. This structural choice guarantees consistent evaluation across all agent-generated neural architectures and baselines, enabling agents to focus exclusively on architectural innovations rather than the engineering overhead of implementing training loops.

3 The AIRA-Compose Pipeline

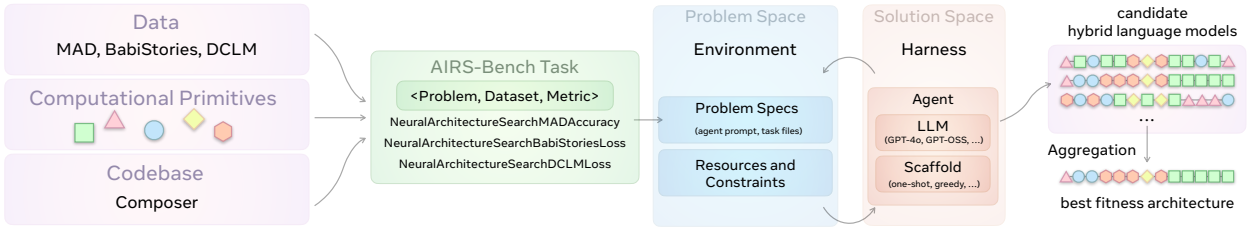


Figure 2 The AIRA-Compose pipeline. We recast *Composer* into equivalent AIRS-Bench tasks. These tasks instruct agents to combine predefined computational primitives to assemble a 16-layer small scale architecture. Tasks are run employing the AIRA-dojo harness with 6 LLMs as core reasoning models. Search results are then ranked, aggregated and scaled to 350M, 1B and 3B parameters size.

AIRA-Compose (Figure 2) relies on the *Composer* framework (Acun et al., 2025). Composer discovers hybrid foundation models with a four-step process: (1) The *Search Engine* uses Bayesian Optimization combined with incremental layer search and width-scaling to discover primitive arrangements; (2) The *Evaluator* is a

fast-proxy training and evaluation loop. Candidate architectures from the search are trained on small proxy datasets; (3) The *Aggregator* post-processes the top candidate architectures discovered during the search. It employs layer-wise clustering techniques to select the most frequent computational primitive to obtain a robust small scale architecture while smoothing out the noise and overfitting coming from proxy training; (4) The *Extrapolator* scales the aggregated small scale architecture to a desired target parameter count (e.g., 350M, 1B, or 3B). This is achieved through *stretching* (proportionally expanding contiguous blocks) or *stacking* (repeating the entire discovered architecture sequentially).

AIRA-Compose recasts steps 1 and 2 as AIRS-Bench tasks. Rather than relying on rigid Bayesian Optimization and deterministic incremental search, agents can freely formulate structural hypotheses and propose novel primitive arrangements (see Figure 3), evaluate them, and iteratively refine their designs based on their prior knowledge. We focus on 16 layer search, since they are small scale proxies that correlate well with equivalent large scale model. Once the agentic exploration is concluded, we leverage steps 3 and 4 to scale the agents' discoveries for final, large-scale evaluation. For the aggregation step, we collect the submitted architectures along with their test scores across all steps from all agents (see Appendix B).

An example of agent-driven NAS is given in Figure 3, showing the first few nodes from a Greedy GPT-5 run on the 3-primitive task. Compared to the Composer framework, which relies on fixed search methodologies, architectures designed at each step come from the agent's own understanding of the problem as presented in `project_description.md`. At each node, the agent articulates its design choices, produces a candidate architecture as a `submission.csv` file, and writes an evaluation script to validate its reasoning. The submitted architecture is then trained and evaluated independently in `evaluate.py`. The node with the highest validation score is selected for further exploration via *improve* operations, which propose new architectures informed by the parent's reasoning and score. This process allows the agent to leverage domain knowledge to navigate the combinatorial space in a meaningful way, rather than relying on predefined structural templates or hand-crafted mutation operators. Scaled across several hundreds of nodes and multiple independent agents, this process yields a semantically diverse exploration, which we believe to be the ultimate advantage of AIRA-Compose.

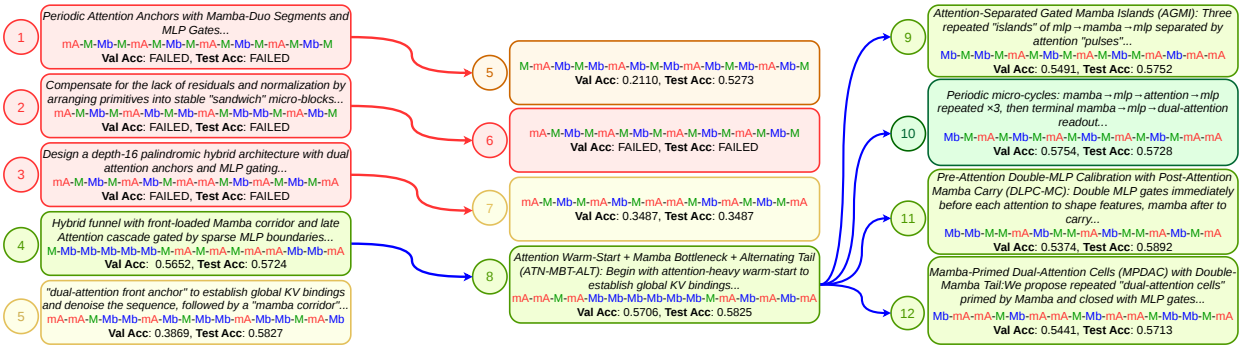


Figure 3 Greedy GPT-5 search (partial) on the 3-primitive NAS task within AIRA-Compose. The agent drafts five initial solutions and iteratively explores the architecture space via a greedy tree search. At each node, the agent reasons on different design choices, writes a candidate architecture to `submission.csv` and evaluates it. The highest-scoring node is then selected for further exploration. Red arrows indicate *debug* operations, while blue arrows indicate *improve* operations. At each improvement, the agent proposes a new architecture informed by the parent's reasoning and score. We report a snippet of the agent's design rationale, the submitted architecture string, the agent's internal validation accuracy, and the true test accuracy obtained from the isolated scoring script.

Primitives. A *primitive* is a computational building block from which model architectures are assembled. We consider *two-primitive* and *three-primitive* search spaces. We employ MLPs (M), multi-head Attention (mA) and Mamba SSM (Mb). The two- and three- primitive search spaces span $2^{16} = 65{,}536$ and $3^{16} \approx 43\text{M}$ possible 16-layer arrangements, respectively. Their configurations at small scale and 350M, 1B, and 3B scale are provided in Appendix D.

Datasets We evaluate architectures on three proxy datasets as in Acun et al. (2025), using metrics chosen to reliably predict at-scale performance: (i) **MAD** (Poli et al., 2024), a suite of six synthetic token-manipulation

tasks (e.g., selective copying, compression, and in-context recall). Models are trained on 800 samples and tested on 1,280, using average accuracy as the metric; (ii) **BabiStories** (Zhang et al., 2025), a synthetic corpus of children's stories. Models train on 927,158 samples and are evaluated on 9,275 via cross-entropy loss; (iii) a fixed subset of **DCLM** (Li et al., 2024a). Models train on 10,000 samples and test on 9,275 via cross-entropy loss. A larger portion of the DCLM corpus is reserved for the large-scale pretraining phase. We allow agents to validate their hypotheses by creating a 70:30 train/validation split on the original train set (Hambardzumyan et al., 2026). Submitted architectures are trained from scratch on the full training set and evaluated on the test set withheld from the agent.

4 AIRA-Design: low-level mechanistic design

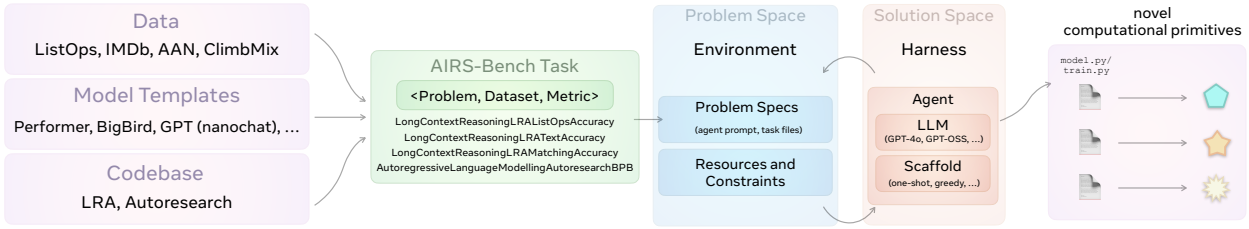


Figure 4 AIRA-Design: agent-driven, low-level mechanistic design. We recast the LRA and Autoresearch codebases into equivalent AIRS-Bench tasks. These tasks instruct agents to design novel computational primitives / full training loops and write them into a `model.py/train.py` file. Tasks are run employing the one-shot and greedy scaffolds of the AIRA-dojo harness with 12 different LLMs as core reasoning models, for a total of up to 20 agents.

For the set of the AIRA-Design tasks, we leverage (1) the LRA benchmark (Tay et al., 2020) — a benchmark suite designed to evaluate sequence models on their ability to capture long-range dependencies, and (2) the Autoresearch benchmark (Karpathy, 2026), an open-ended challenge where agents must autonomously explore, implement, and validate novel research ideas to improve a model's training efficiency. Unlike the high-level architecture search, where agents select and arrange predefined computational primitives, the AIRA-Design tasks require agents to engage in *low-level mechanistic design*: implementing novel neural network architectures from scratch (LRA), as well as optimizing their training loop (Autoresearch).

Long Range Arena (LRA) tasks challenge agents to design memory-efficient, sub-quadratic attention mechanisms that avoid materializing full $O(n^2)$ attention matrices for sequences exceeding 2000–4000 tokens, while preserving the capacity to model long-range interactions. To accomplish this, agents must implement their chosen architecture by producing a complete, executable `model.py` file that defines a `CustomEncoder` (or `CustomDualEncoder` for the matching task) compatible with the provided JAX/Flax training infrastructure.

We evaluate performance on two task versions: *Configurable* and *Non-Configurable*. The *Configurable* version of the tasks allows agents to explicitly specify key global variables directly within `model.py`. This allows them to adjust hyperparameters to better suit their architectures (see Appendix E, Table 12). Conversely, the *Non-Configurable* setup enforces the default LRA parameters, restricting agents freedom to exclusively evaluate agents' design capabilities, independently of their hyperparameter tuning skills. We include the *Configurable* version of the task to allow agents to move toward configurations closer to current SOTA models, which support up to 2 million tokens (Ma et al., 2024).

Autoresearch is an open-ended optimization benchmark in which agents must autonomously improve a GPT language model training script. The goal is to achieve the lowest possible validation bits per byte within a fixed 5-minute wall-clock training budget on a single GPU. Agents are given a baseline `train.py` containing a nanochat model implementation with multi-head causal attention, rotary embeddings, flash attention, and a hybrid Muon–AdamW optimizer. Agents are allowed to modify any aspect of this script, including model architecture (depth, width, attention patterns, positional encodings, activation functions), optimizer and learning rate schedule hyperparameters, and training configuration (batch size, gradient accumulation, warmup/cooldown ratios). On the other hand, the data pipeline, tokenizer, and evaluation harness are read-only. Compared to the LRA tasks, we assess agents capability to design an advanced architecture *and*

an optimized training loop. The task also rewards simplification, and removing unnecessary complexity for additional training steps is explicitly valued when performance is preserved or improved.

We evaluate performance on two task versions: default and *With Literature*. The *With Literature* version of the task is designed to provide agents with relevant context before attempting a solution, and it includes in the task folder a curated selection of 41 research papers provided to the agent as structured paper analyses (`task_info.jsonl`) and 14 reference code repositories for some of the papers. Resources are organized into papers on Architecture Improvements (20), Training Strategies (17), and Optimizers (5). The task `project_description.md` directs the agent to consult these resources before attempting a solution. A detailed summary of the resources employed in the *With Literature* version of the task is provided in Appendix F, Tables 16-18.

4.1 Datasets

The LRA tasks focus on the three text-based datasets of the benchmark. All LRA tasks employ **accuracy** as the evaluation metric: **IMDB Sentiment Classification (Text)**: The Text task uses the IMDB movie reviews dataset for binary sentiment classification. Models must capture semantic relationships across the full document context to accurately classify sentiment; **ListOps**: ListOps presents hierarchical mathematical expressions in prefix notation with nested operations (MAX, MIN, MEDIAN, SUM_MOD). Example sequences such as `[MAX 4 3 [MIN 2 3] 1 0]` require models to parse bracket structure, respect operator precedence, and compute the correct numerical result (0–9); **ACL Anthology Network (AAN) (Retrieval)**: The Retrieval task uses the ACL Anthology Network (AAN) dataset, tokenized at character level, presenting pairs of academic paper abstracts for binary similarity classification. It requires a dual-encoder setup to test whether architectures can efficiently process and compare long documents. For each task, agents have access to training and validation splits during the search phase. The validation split is used to iteratively evaluate hypotheses when running the greedy scaffold. Once agents finalize their `model.py` submission, the architecture is trained from scratch on the full training set, and final performance is evaluated exclusively on a held-out test split that remains strictly inaccessible during the search phase.

The Autoresearch task (Karpathy, 2026) consists of pre-tokenized web text from the **ClimbMix** corpus (Diao et al., 2025). A pre-trained BPE tokenizer with a vocabulary size of ∼8192 is provided. The dataset is split into training shards and a pinned validation shard. The evaluation metric is validation **bits per byte (BPB)**, where lower is better, which is vocabulary-size-independent and enables fair comparison across architectural changes.

5 Experiments

Our experimental setup and metrics follow the paradigm defined by AIRS-Bench (Lupidi et al., 2026). We refer the reader to the original manuscript for a detailed description. We briefly summarize them here. We test tasks on one-shot and greedy scaffolds within AIRA-dojo. We run experiments on 20 seeds for one-shot agents and on 10 seeds for greedy agents, unless otherwise specified. Each run lasts for at most 24 hours or at most 500 steps. Each agent has access to one H200 GPU to draft and validate its solutions. For the search on BabiStories and DCLM, which are larger, we allocate 60 hours per run. Within this time budget, each seed covers a search space of 100–200 small scale architectures in AIRA-Compose, and several hundreds architectures and training loops in AIRA-Design. For AIRA-Compose large scale experiments, 350M models are trained on 8 H200 GPUs, while 1B and 3B models on 16 H200 GPUs. Optimizers and hyperparameters at small and large scale are kept consistent with prior work (Acun et al., 2025).

AIRA-Compose Metrics. We rank small scale models based on their raw scores (accuracy or cross-entropy loss). We evaluate the scaled-up models across three metrics: **(i)** cross-entropy validation loss measured on 1000 samples of the DCLM validation set; **(ii)** 0-shot downstream accuracy and normalized accuracy on 6 tasks: HellaSwag, WinoGrande, ARC-Easy, ARC-Challenge, PIQA, and SciQ; and **(iii)** the **DCLM Core score** (Li et al., 2024a), an unweighted average across 14 tasks evaluated under 0-shot (COPA, CoQA, LAMBADA, OpenBookQA, WinoGrande, XWinograd-EN), 3-shot (AGIEval LSAT-AR), and 10-shot (ARC-Easy, ARC-Challenge, BoolQ, CommonsenseQA, HellaSwag, PIQA, SQuADv2). The primary metric is accuracy for most tasks, normalized accuracy for OpenBookQA, AGIEval LSAT-AR, ARC-Easy, ARC-Challenge, HellaSwag, and PIQA, and F1 score for CoQA and SQuADv2.

AIRA-Design Metrics. We present AIRA-Design results using three metrics: **(i)** raw score, to allow for an immediate comparison with respect to SOTA and established baselines; **(ii)** valid submission rate (VSR): To assess an agent's capability to consistently formulate a working solution and submit it confidently. The VSR on task t for an agent a is defined as:

$$\mathrm{VSR}_{a,t} = \frac{V_{a,t}}{T_{a,t}} \tag{1}$$

where $V_{a,t}$ is the number of successfully submitted runs for agent a on task t, $T_{a,t}$ is the total number of runs for agent a on task t; **(iii)** normalized score (NS), to provide an intuitive measure of the progress on the benchmark, we define and use NS of an agent a on a task t as:

$$\mathrm{NS}_t^a = \frac{\phi_t(s_t^a) - \phi_t(s_t^{\min})}{\phi_t(s_t^{\mathrm{sota}}) - \phi_t(s_t^{\min})} \tag{2}$$

where $s_t^{\min}$ corresponds to the worst score observed across all seeds and agents on task t, s_t^{sota} is the SOTA score sourced from literature, and s_t^a is the score achieved by agent a on the same task. This normalizes scores to a value range between 0 and 1 for minimum and SOTA performances, respectively. NS > 1 if it exceeds SOTA. The transformation ϕ_t applies the **march of 9s** (Karpathy and Patel, 2025):

$$\phi_t(s) = -\log_{10}(|s - s_t^{\mathrm{opt}}|) \tag{3}$$

where s_t^{opt} is the overall possible optimal score for the task (in our case 1.0). Failed and invalid submissions are treated as submissions with a 0 normalized score.

We regard the LRA tasks as proper, open-ended AI research challenges in pure AIRS-Bench style. Because these tasks require writing novel, functional code from scratch, all three metrics are required for comprehensive evaluation. For AIRA-Compose tasks, agents only output a formatted string mapping to predefined blocks, making VSR virtually 100%; we therefore report only raw scores and the scaled-up evaluation metrics. Similarly, for Autoresearch tasks, we focus on raw scores expressed as validation BPB.

6 Results

6.1 AIRA-Compose

6.1.1 2 primitives (M, mA)

We evaluated 10 agents across three datasets by pairing five LLMs (Code World Model Carbonneaux et al. (2025), o3-mini, gpt-oss-20b, gpt-oss-120b, and GPT-4o) with one-shot and greedy scaffolds. This yielded 150 runs per dataset (50 greedy and 100 one-shot). We further augmented the search with 20 additional greedy GPT-5 runs exclusively on the MAD dataset. Since agents only need to output a formatted string that maps to predefined PyTorch blocks, we deliberately employed non-SOTA models, which are fully capable of handling this structural simplicity. The search yielded 2,307 unique architectures explored, probing 3.17% of the search space. See Appendix C, Figures 18–20 for the search dynamic per agent. We rank all designed models across agents and seeds and rank them on their test accuracy (see Appendix B). We then aggregate them into robust base patterns and scale them. This resulted in 6 agent-discovered architectures (base 16-layer pattern in parentheses):

- **AIRAformer-A** (`(A + M) + (2A + 2M) + 4 × (A + M) + 2M`): The aggregated 16-layer architecture is **stretched** to large scale.
- **AIRAformer-B** (`2A + 5 × (A + M) + 4M`): evaluated in its **stacked** variant.
- **AIRAformer-C** (`(2A + M) + 3 × (A + M) + (2A + M) + 4A`): evaluated in its **stacked** and **stretched** variants.
- **AIRAformer-D** (`5 × (2A + M) + A`): evaluated in its **stacked** and **stretched** variants.

Aggregation techniques to obtain them and their composition at 350M, 1B, and 3B scale are shown in Appendix D. Notably, despite different aggregation techniques, these models converge on shared attention-to-MLP ratios: 7:9 for AIRAformers A and B, and 11:5 for C and D.

IsoToken Analysis. We pretrain 9 models: 6 AIRAformers, Llama 3.2 baseline and 2 Composer-found models (Composite Stacked and Stretched, which are the best Composer-found models in 2-primitive space) at the 1B scale for a fixed, predefined token budget corresponding to 71,565 steps, derived from Acun et al. (2025). With a batch size of 4, sequence length of 8,192, and DP of 16, this corresponds to ∼37.5 billion tokens, or ∼ 38 TPP. Each architecture is trained with 3 random seeds, and we report the mean performance across them. We report the 3 metrics for the 9 pretrained models in Table 2. A comprehensive breakdown is given in Appendix C, Table 7, Figure 22.

Table 2 Pretraining results of 2-primitive architectures at 1B scale with fixed token budget (37.5B tokens). We report validation loss, 0-shot accuracy on 6 downstream tasks (in %) and few-shots DCLM score on 14 downstream tasks (in %), averaged over 3 seeds (±std). Values in parentheses show normalized accuracy where applicable. Best results in **bold**, second best underlined.

Architecture	Val Loss ↓	ARC-C	ARC-E	HellaS.	PIQA	SciQ	WinoG.	Avg ↑	DCLM Core Score ↑
Llama 3.2	$2.815_{\pm.003}$	26.1 (30.0)	62.3 (56.0)	41.4 (53.1)	72.2 (72.5)	87.2 (80.2)	56.0	57.5 (58.4)	$46.9_{\pm0.3}$
Composite (St.)	$2.782_{\pm.021}$	27.7 (30.2)	62.9 (57.1)	42.4 (55.1)	72.2 (72.3)	88.0 (82.7)	57.3	58.4 (59.5)	$46.6_{\pm1.4}$
Composite (Str.)	$2.759_{\pm.002}$	28.1 (31.0)	64.0 (58.5)	42.6 (54.8)	72.2 (72.6)	88.1 (81.8)	55.3	58.4 (59.8)	$47.3_{\pm0.1}$
AIRAf.-A (Str.)	$2.752_{\pm.002}$	**29.8** (**32.2**)	**65.4** (**59.0**)	43.2 (56.0)	72.6 (**72.9**)	88.0 (82.7)	58.4	59.6 (60.6)	$48.5_{\pm0.2}$
AIRAf.-B (St.)	$2.752_{\pm.001}$	29.4 (31.3)	65.3 (**59.0**)	43.2 (56.3)	**72.9** (**72.9**)	88.6 (83.7)	57.9	59.5 (60.6)	$48.1_{\pm0.1}$
AIRAf.-C (St.)	$2.751_{\pm.002}$	28.4 (31.4)	63.5 (57.5)	42.9 (55.3)	72.5 (72.7)	89.3 (83.6)	58.3	59.1 (60.1)	$48.8_{\pm0.2}$
AIRAf.-C (Str.)	$2.750_{\pm.001}$	29.1 (31.1)	64.0 (57.9)	42.9 (55.4)	71.8 (72.3)	**89.5** (83.5)	56.7	59.0 (60.1)	$48.1_{\pm0.2}$
AIRAf.-D (St.)	$2.745_{\pm.002}$	28.4 (31.6)	63.1 (58.9)	43.3 (56.2)	72.4 (72.5)	88.7 (83.1)	58.1	59.0 (60.5)	$48.4_{\pm0.2}$
AIRAf.-D (Str.)	**2.734**$_{\pm.001}$	29.4 (31.5)	63.7 (**59.0**)	**43.7** (**56.4**)	**72.9** (72.4)	89.4 (**84.5**)	**58.9**	**59.7** (**60.8**)	**48.9**$_{\pm0.4}$

AIRAformers outperform the Composite baselines across both validation loss and downstream tasks performance. AIRAformer-D Stretched achieves the lowest validation loss of 2.734, highest average 0-shot accuracy of 59.7% (60.8%) and highest DCLM Core score (48.9%), outperforming both Llama 3.2 and Composer-found models. The low variance across seeds indicates that the observed improvements are statistically robust.

IsoFLOP Analysis. We pretrain the 8 models (excluding Llama 3.2) across three parameter scales (350M, 1B, and 3B) under five FLOPs budgets: 2×10^{19}, 4×10^{19}, 8×10^{19}, 2×10^{20}, and 4×10^{20} FLOPs, for a total of 120 experiments (see Appendix D, Table 10 for FLOPs count). Attention-heavy AIRAformer-C and AIRAformer-D architectures achieve superior validation loss in the fixed-token-budget setting, as expected (Table 2), while balanced architectures are more compute-efficient in the isoFLOP regime. The additional attention layers in AIRAformer-C/D provide representational benefits when trained for longer, but they are less efficient when compute is the binding constraint.

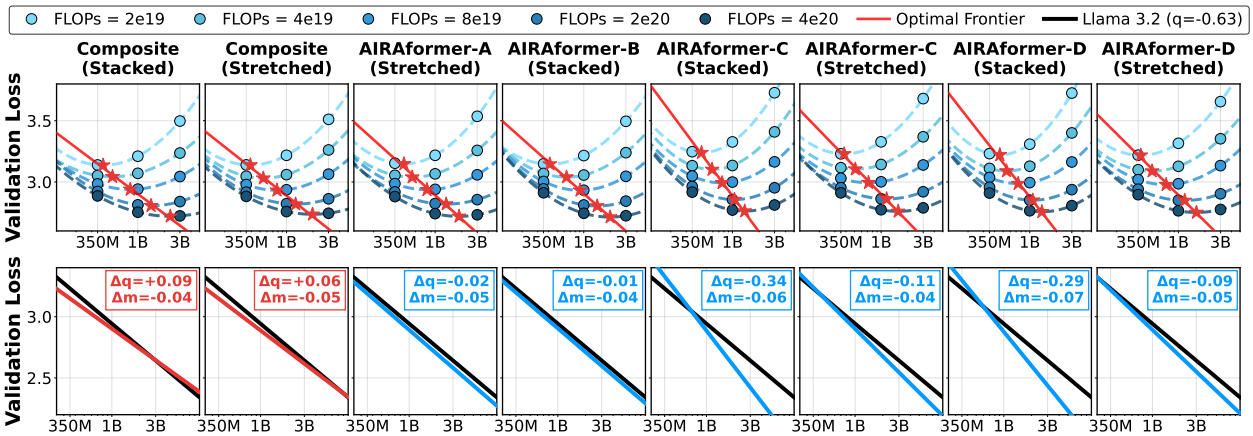


Figure 5 IsoFLOP scaling curves and optimal frontier comparison (M, mA). Top: Validation loss versus model size for each architecture across 5 FLOPs budgets. **Bottom:** Comparison of each architecture's optimal frontier against the Llama 3.2 scaling law derived from (Acun et al., 2025).

We fit isoFLOP parabolas to the validation loss as a function of model size for each FLOP budget (Figure 5, top) and infer the optimal frontier from their minima. Figure 5, bottom, compares each architecture's optimal

frontier against Llama 3.2's reported scaling law, and reports Δq (slope difference, lower is better) and Δm (intercept offset, lower is better) for each architecture with respect to it. AIRAformer-C Stacked and AIRAformer-D Stacked achieve the most favorable scaling coefficients, indicating steeper scaling than Llama 3.2. The Composite baselines exhibit slightly flatter slopes than Llama 3.2, suggesting diminishing advantage as model size increases.

6.1.2 3 primitives (M, mA, Mb)

We use MAD for the 3 primitives search due to better agreement between the small scale ranking and large scale performance. A total of 170 agentic runs yielded 2,248 unique architectures, probing 0.0052% of the search space. Agent-powered AIRA-Compose discovers robust patterns by probing just a fraction of the design space, mitigating the combinatorial explosion of 3-primitive spaces in traditional NAS. We obtain 8 hybrid architectures:

- **AIRAhybrid-A** (`2Mb + M + 11Mb + 2M`): evaluated in its **stretched** variant.
- **AIRAhybrid-B** (`3 × (2Mb + M + A) + 2Mb + 2M`): evaluated in **stacked** and **stretched** variants.
- **AIRAhybrid-C** (`2Mb + A + 2 × (Mb + A) + M + 2 × (A + Mb + A + M)`): evaluated in its **stretched** variant.
- **AIRAhybrid-D** (`2Mb + M + 2 × (Mb + M) + A + M + Mb + M + A + M + Mb + M + A`): evaluated in **stacked** and **stretched** variants.
- **AIRAhybrid-E** (`5 × (Mb + M + A) + M`): evaluated in **stacked** and **stretched** variants.

Table 3 Pretraining results of 3-primitive hybrid architectures at 1B scale with fixed token budget (37.5B tokens). We report validation loss, 0-shot accuracy on 6 downstream tasks (in %), and few-shots DCLM score on 14 downstream tasks (in %). Values in parentheses show normalized accuracy where applicable. Best results in **bold**, second best underlined. Results are from a single seed.

Architecture	Val Loss ↓	ARC-C	ARC-E	HellaS.	PIQA	SciQ	WinoG.	Avg ↑	DCLM Core Score ↑
Nemotron-H Approx.	2.741	29.1 (31.3)	65.3 (59.9)	43.4 (56.3)	73.3 (73.7)	87.8 (82.7)	56.4	59.2 (60.1)	48.5
Nemotron-2 Approx.	2.732	30.7 (33.2)	65.9 (**61.9**)	44.2 (57.2)	**74.0** (73.9)	88.7 (84.3)	58.5	60.3 (61.5)	48.4
Mamba (Mb + M)	2.771	30.1 (32.5)	64.9 (61.2)	43.2 (55.8)	72.9 (73.1)	88.8 (85.1)	57.7	59.6 (60.9)	46.1
Composer (2Mb-M-3A)	2.724	30.0 (33.1)	65.0 (61.6)	43.9 (56.9)	71.9 (73.9)	89.6 (85.3)	58.8	59.9 (61.6)	**49.3**
AIRAhybrid-A (Str.)	2.750	30.5 (**33.5**)	**66.6** (61.5)	44.0 (56.6)	73.9 (73.4)	89.7 (85.2)	57.2	60.3 (62.1)	47.5
AIRAhybrid-B (St.)	2.732	29.9 (33.4)	64.8 (61.2)	44.0 (57.1)	73.1 (73.4)	89.9 (84.1)	58.9	60.1 (61.8)	49.0
AIRAhybrid-B (Str.)	2.728	30.0 (31.1)	65.1 (61.2)	44.2 (57.6)	73.4 (74.0)	89.5 (85.7)	59.2	60.2 (61.9)	49.1
AIRAhybrid-C (Str.)	2.740	30.2 (32.2)	65.0 (58.5)	43.3 (56.1)	72.8 (73.6)	**90.2** (85.4)	58.0	59.9 (61.2)	48.9
AIRAhybrid-D (St.)	2.720	29.5 (30.9)	65.1 (60.0)	**44.3** (57.6)	72.8 (73.7)	89.9 (**85.8**)	**59.8**	60.2 (61.6)	48.4
AIRAhybrid-D (Str.)	**2.719**	**32.0** (32.9)	66.3 (61.4)	44.1 (**57.8**)	73.6 (**74.7**)	88.6 (84.2)	58.2	**60.5** (**62.2**)	48.5
AIRAhybrid-E (St.)	2.736	27.7 (30.3)	64.3 (59.5)	43.5 (56.5)	72.1 (73.3)	88.9 (85.3)	58.6	59.2 (61.0)	48.8
AIRAhybrid-E (Str.)	2.732	29.8 (32.5)	64.9 (60.4)	44.1 (57.1)	72.3 (72.9)	89.8 (**85.8**)	57.7	59.8 (61.7)	48.6

IsoToken Analysis. We pretrain 12 models: 8 AIRAhybrids, 3 hybrid baselines (Mamba, Nemotron-H and Nemotron-2) and one Composer-found architecture at the 1B scale at $\sim$ 38 TPP. Note that Nemotron-H and Nemotron-2 are only *approximated* versions to smaller scale, with MoEs replaced by MLP layers. The Composer architecture is obtained by performing the search over 6-layer small scale models, which is then stacked to obtain an equivalent model at large scale. Table 3 summarizes the validation loss and downstream task performance. AIRAhybrids often outperform established baselines, including Mamba and approximated Nemotron-2, across both linguistic and reasoning benchmarks. AIRAhybrid-D (Stretched) achieves the lowest validation loss of 2.719 and the highest 0-shot accuracy of 60.5%. While specific architectures show localized strengths (e.g., AIRAhybrid-C Stretched on SciQ and Nemotron-2 on PIQA), AIRAhybrids generally maintains superior average 0-shot accuracy.

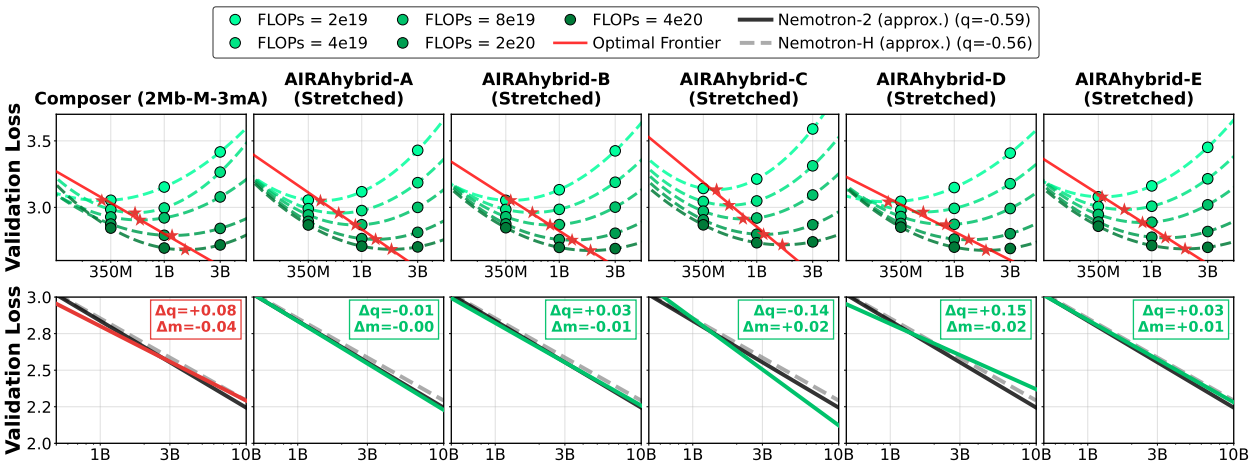


Figure 6 IsoFLOP scaling curves and optimal frontier comparison (M, mA, Mb). Top: Validation loss versus model size for each architecture across 5 FLOPs budgets. **Bottom:** Comparison of each architecture's optimal frontier against the approximated Nemotron-2 and Nemotron-H.

IsoFLOP Analysis. We pretrain the 11 models (excluding Mamba due to poor isotoken performance) at 3 scales, 350M, 1B and 3B parameters under 5 FLOPs budgets, for a total of 165 experiments. Results for selected models are shown in Figure 6 (see Appendix C for full results).

We compare each architecture's optimal frontier against the approximated Nemotron-H and Nemotron-2's scaling law in Figure 6, reporting Δq (slope difference) and Δm (intercept offset) relative to Nemotron-2. Architectures with balanced Mamba-to-attention ratios (Nemotron-2, AIRAhybrid-B Stretched and AIRAhybrid-D Stretched) consistently achieve the lowest validation loss across model sizes and FLOP budgets, while the attention-heavy AIRAhybrid-C Stretched ranks last. However, AIRAhybrid-C Stretched also achieves the steepest scaling slope ($\Delta q = -0.14$), indicating faster improvement with scale than Nemotron-2, despite its higher intercept. AIRAhybrid-A Stretched, the only pure-SSM design, also scales slightly better ($\Delta q = -0.01$). The Composer baseline, on the other hand, exhibits a flatter slope. These results mirror the two-primitive behavior: attention-heavy architectures are less compute-efficient at fixed FLOP budgets but exhibit steeper scaling slopes that may yield superior performance at larger scale.

Finally, Figure 7 presents the 1B-scale latency – validation loss Pareto frontiers under isoToken and isoFLOP protocols. Model latency is computed by scaling the number of layers per block type by its corresponding GPU(H100)-profiled block latency. Under the isoToken analysis, AIRAhybrids-D (Stacked):17 and AIRAhybrids-D (Stretched):18 advance the previously-achieved frontier of Composer Acun et al. (2025), leading to lower validation loss at strictly lower latencies than their Composer counterpart, which compensates strong downstream task performance with high latency.

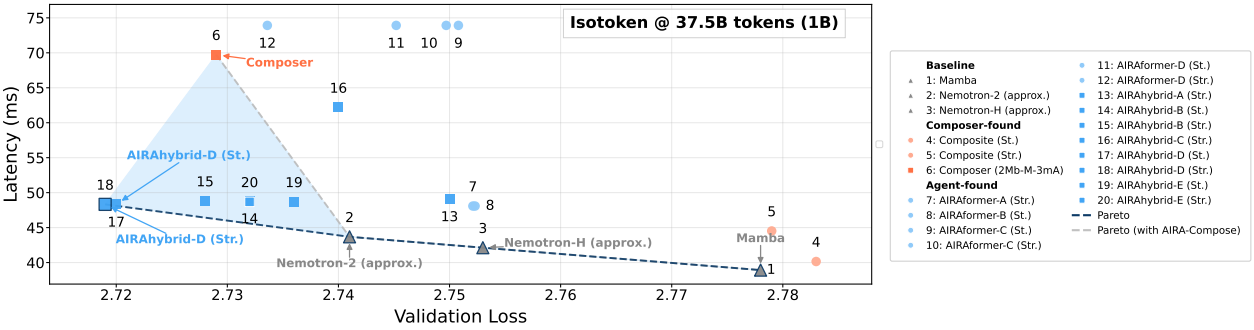


Figure 7 Latency–validation loss Pareto frontiers at the 1B-parameter scale. Isotoken comparison (37.5B). Total latency is the sum of per-block latencies across all layers. Dashed lines indicate Pareto frontiers with and without agent-discovered architectures; the shaded region highlights the gap between the two.

6.2 AIRA-Design

6.2.1 Long Range Arena

We evaluated 20 agents across 12 unique models: all 12 in greedy mode (GPT-5, o3, o3-mini, GPT-4o, CWM, gpt-oss-120b, gpt-oss-20b, Devstral-Small 24B, Opus 4.6, Opus 4.5, Gemini 3.0 Pro, and Gemini 3.1 Pro) and 8 only in one-shot mode (CWM, o3, o3-mini, gpt-oss-20b, gpt-oss-120b, GPT-4o, Devstral-Small 24B, and GPT-5). The 12 greedy agents were run for 10 seeds across three datasets and two setups (Configurable and Non-Configurable), totaling 720 runs. The 8 one-shot agents were run for 20 seeds each, totaling 960 runs. In total, we aggregated results from 1,680 runs, representing a few thousands of architectural primitives autonomously designed, implemented, and validated.

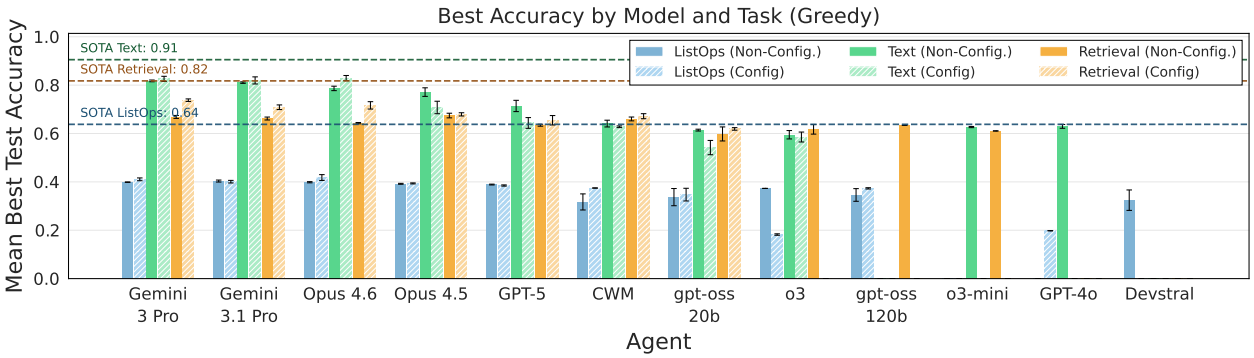


Figure 8 Mean best test accuracy achieved by the 8 greedy agents across the 6 low-level mechanistic design tasks. Solid bars represent performance using the original LRA benchmark configuration, while striped bars indicate results from the Configurable LRA setup that permits agent-specified hyperparameters. Error bars represent standard error of the mean across 10 random seeds per task. Horizontal dashed lines indicate SOTA performance over the 3 datasets: ListOps (0.64), Text (0.91), and Retrieval (0.82). Agents are ordered by decreasing average performance.

Figure 8 shows the average best test accuracy per model and agent. It captures the peak accuracy achieved during exploration, even if the agent ultimately failed to submit that model as a valid final solution. Greedy Gemini 3 Pro achieved the highest mean best accuracy across nearly all task-configuration combinations, followed by Greedy Gemini 3.1 Pro and Opus 4.6.

Greedy Gemini 3 Pro and Greedy Opus 4.6 generated the best solutions for 2 of the 6 task configurations each, while Greedy Gemini 3.1 Pro and Greedy Opus 4.5 achieved the best accuracy in 1 task each. A summary of each best solution for the 6 tasks provided in Appendix E, Table 14. Agents produced functional solutions incorporating established efficient attention mechanisms such as linear attention with ELU+1 kernels, hierarchical pooling strategies, and blockwise local attention with recurrent memory. Several solutions integrate known architectural components including depthwise convolutions and gated linear units. While these designs demonstrate competent engineering (combining existing techniques in task-appropriate ways) they do not represent fundamentally novel contributions to the efficient attention literature. The discovered architectures largely recombine and adapt ideas from prior work (e.g., Performer, Longformer, Conformer) rather than introducing new theoretical insights. This suggests that current agents excel at engineering-level synthesis and adaptation, but fall short of genuine scientific innovation in mechanistic design.

Figure 9 shows the Valid Submission Rate (VSR), i.e., the proportion of runs yielding working, validated solutions. VSR varies significantly, with Greedy Opus 4.6 always successfully submitting a solution, to below 10% for weaker models. VSR was generally higher in the Non-Configurable setup, indicating that the expanded hyperparameter search space in the Configurable setup increases the difficulty of producing valid code. Notably, one-shot agents failed to produce any valid submissions. This confirms that single-turn code generation is insufficient for mechanistic design, and that iterative refinement, debugging, and validation are essential.

Figure 10 and Table 4 summarize the average Normalized Score (NS) per agent and per task, respectively. Among greedy agents in the non-configurable setup, ListOps emerges as the most tractable task with an average NS of 0.257 ± 0.054, while Text classification in the configurable setup remains the most challenging

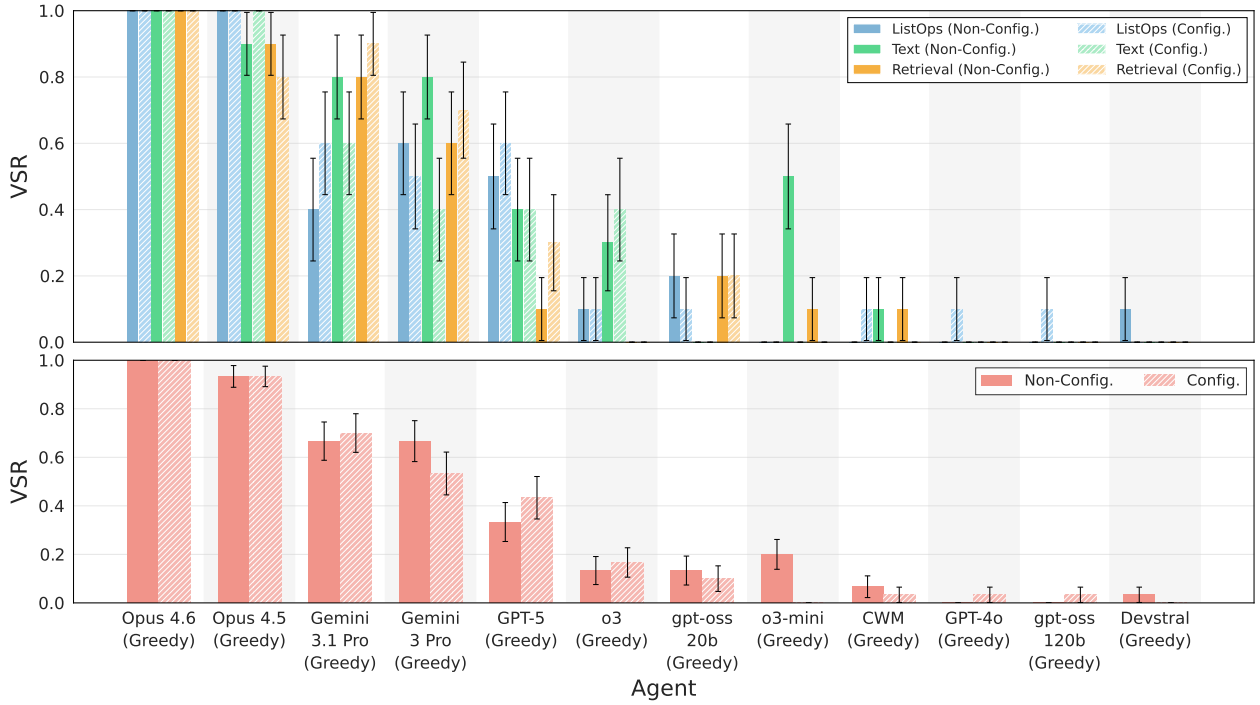


Figure 9 Valid Submission Rate (VSR) across agents and setups. Top: per-task VSR showing the proportion of runs that produced valid submissions (i.e., final accuracy > 0) for each of the 6 task configurations. Bottom: average VSR across 3 tasks, where missing tasks contribute 0 to the average. Solid bars represent the original LRA benchmark configuration; striped bars indicate the Configurable LRA setup. Error bars represent standard error of the mean. Agents are ordered by decreasing average performance. No valid submission was achieved by one-shot agents.

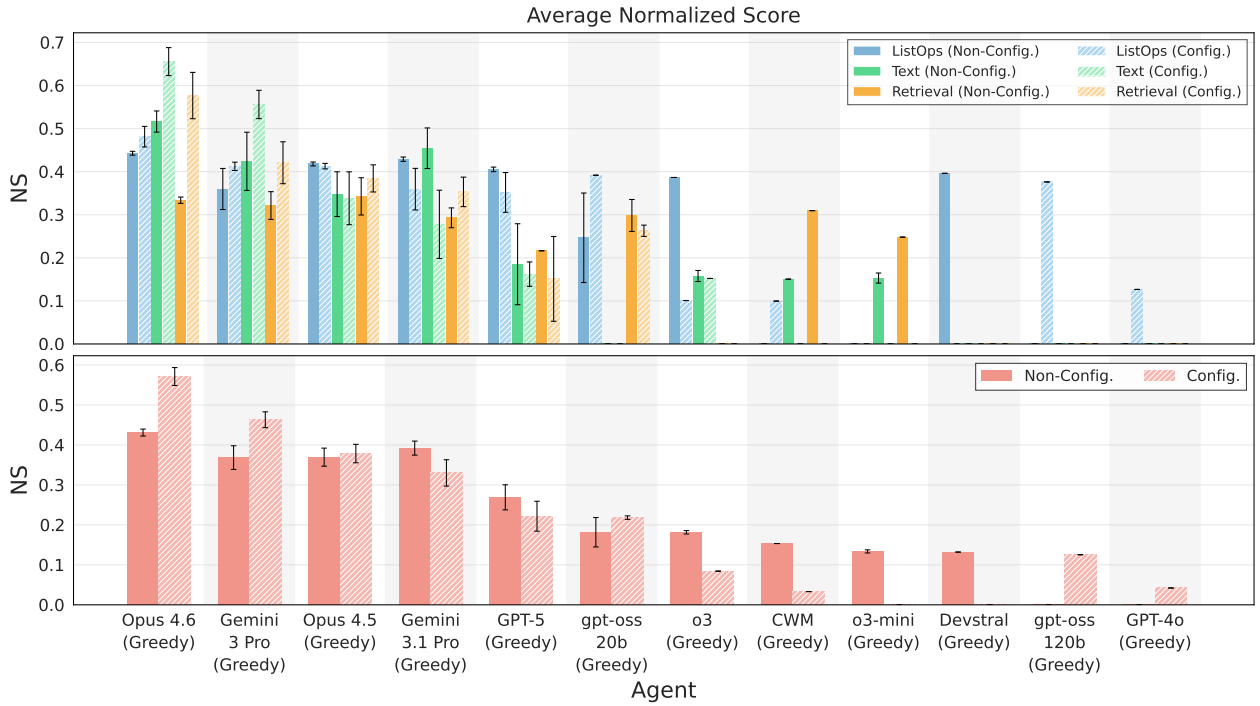


Figure 10 Normalized Score (NS) using the march of 9s across agents and setups. Top: per-task NS computed only over valid submissions. Bottom: average NS across 3 tasks, where missing tasks contribute 0 to the average. Solid bars represent the original LRA benchmark configuration; striped bars indicate the Configurable LRA setup. Error bars represent standard error of the mean. Agents are ordered by decreasing average performance. No valid submission was achieved by one-shot agents.

(NS = 0.178 ± 0.064). The configurable setup generally yields lower NS values across all tasks despite offering greater architectural flexibility. This performance gap suggests that current agents struggle to effectively leverage hyperparameter optimization and often fail to navigate the larger search space. Even top-performing agents achieve NS values well below 1.0, showing a substantial gap between agentic design and human SOTA performance.

Table 4 Average Normalized Score per task. Agents with no valid submissions contribute 0 to the average. Left: greedy agents only ($n = 12$). Right: all agents including one-shot ($n = 20$).

	Greedy Agents ($n = 12$)		**One-Shot + Greedy Agents** ($n = 20$)	
Task	**Non-Config.**	**Config.**	**Non-Config.**	**Config.**
ListOps	0.257 ± 0.054	0.259 ± 0.049	0.154 ± 0.043	0.156 ± 0.041
Text	0.199 ± 0.053	0.178 ± 0.064	0.119 ± 0.039	0.107 ± 0.043
Retrieval	0.197 ± 0.041	0.179 ± 0.058	0.118 ± 0.033	0.107 ± 0.040

Since the six mechanistic design tasks follow the AIRS-Bench task structure, we can benchmark agent performance against the broader suite. The 20 tasks in AIRS-Bench are classified into four difficulty bands: *easy*, *medium*, *hard*, and *expert*, based on the normalized scores achieved across agents. When averaged across all 20 agents, all LRA tasks fall into the *hard* category ($0.10 \leq \text{NS} < 0.20$). This is in line with other code generation tasks in the AIRS-Bench suite. The challenging nature of LRA tasks stems from four key factors: (i) a training bias toward PyTorch, which limits LLM proficiency in JAX/Flax; (ii) the inherent complexity of writing low-level, functionally complete attention mechanisms from scratch; (iii) fixed compute budgets that penalize inefficient implementations; and (iv) frequent out-of-memory (OOM) errors and numerical instabilities that invalidate theoretically sound models during the compilation or training phase.

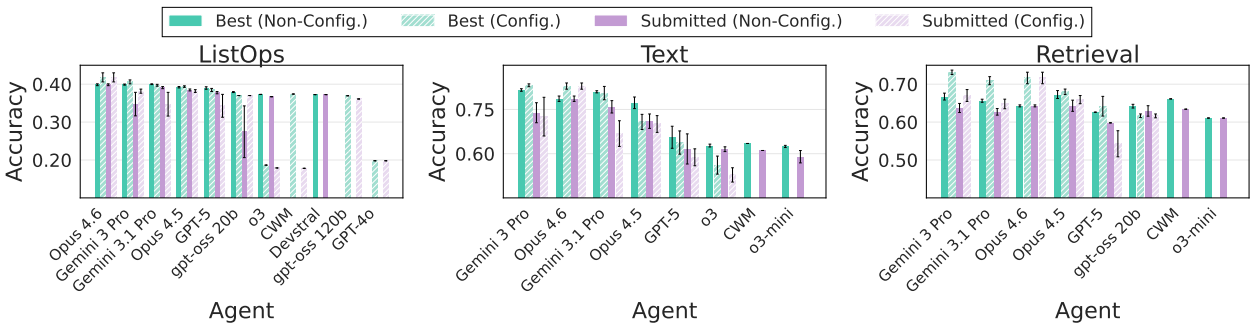


Figure 11 The generalization gap. Comparison of best observed accuracy versus submitted (final) accuracy for greedy agents across the 3 LRA tasks. Each subplot shows results for a single task, with bars grouped by metric type (best vs submitted) and configuration (solid for original LRA, striped for Configurable). Only runs with valid submissions are included. Error bars represent standard error of the mean. Agents are ordered by decreasing best accuracy per task. The gap between best and submitted accuracy reflects the generalization gap, measuring the agent's ability to detect the best-performing model for final submission.

Figure 11 compares the best accuracy achieved during exploration versus the accuracy of the final submitted model. The gap between these metrics, known as generalization gap, measures an agent's ability to identify and select its best-performing solution. Across all tasks, we observe generally small gaps, with submitted accuracy closely tracking best accuracy for most agents. This suggests that agents are competent at recognizing promising solutions during exploration, though occasional larger gaps indicate room for improved validation strategies.

Configurability produces mixed results across the 12 greedy agents: 7 regressed on average while 4 improved, with Opus 4.6 and gpt-oss-20b gaining the most (+4.5*pp* each), followed by Gemini 3 Pro (+3.0*pp*) and Gemini 3.1 Pro (+1.8*pp*); o3-mini produced no valid configurable runs. The strongest regressions occurred in weaker agents (CWM −11.5*pp*, Devstral −10.8*pp*, o3 −8.3*pp*), suggesting they struggle more with the expanded hyperparameter search space. The impact is task-dependent: on Retrieval, configurability helped in

6 agents, led by Opus 4.6 and gpt-oss-20b (+7.4*pp* each) and Gemini 3 Pro (+6.9*pp*)—while on Text, results were mixed, with top agents such as Opus 4.6 improving (+4.3*pp*) but weaker agents suffering large drops (CWM −32.6*pp*, o3 −12.6*pp*). ListOps remained largely stable for top agents. Relative rankings were mostly preserved: Gemini 3 Pro maintained its top position, though Opus 4.6 rose from fourth to second.

6.2.2 Autoresearch

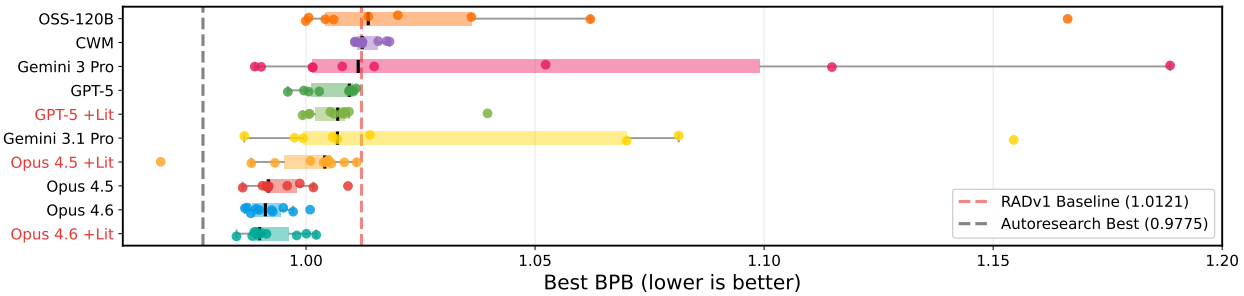


Figure 12 Autoresearch: distribution of best validation BPB per seed. Each dot represents one seed's best BPB achieved during its run. Boxes span the interquartile range (Q25–Q75) with bars extending to the most extreme point within 1× IQR. Dashed lines mark the RADv1 (i.e., AIRA-DOJO) baseline (BPB = 1.0121, red) and the Autoresearch best (BPB = 0.9775, black). Results are ordered by decreasing median BPB (in black).

We evaluated 7 agents, greedy × {CWM, gpt-oss-120b, GPT-5, Opus 4.5, Opus 4.6, Gemini 3.0 Pro, Gemini 3.1 Pro}. The 7 greedy agents were run for 10 seeds on the single Autoresearch task, for a total of 70 runs. Across all runs, 3,936 out of 11,530 total steps (34.1%) yielded a valid numerical fitness value. Additionally, we ran our top-3 agents, greedy GPT-5, Opus 4.5 and Opus 4.6, on the Autoresearch task version enhanced with code and literature, for a total of 30 additional experiments and ∼6000 agent-designed training loops. The distribution of the best BPB per seed is shown in Figure 12. Both Greedy Gemini 3 Pro and Greedy Gemini 3.1 Pro show the largest variance in their best BPB across seeds, indicating inconsistent optimization: some seeds find strong solutions while others stagnate or crash. By contrast, the Opus 4.5 and Opus 4.6 families cluster tightly below 1.00, reflecting more reliable convergence. In terms of median performance, Opus 4.6 +Lit (0.990), Opus 4.6 (0.991), and Opus 4.5 (0.992) all perform substantially below the RADv1 baseline. CWM (1.012) and OSS-120B (1.014) have medians near or above the baseline, indicating these agents rarely improve upon the starting script. Notably, adding literature access does not uniformly help: GPT-5 +Lit and Opus 4.5 +Lit underperform compared to their base variant, while Opus 4.6 +Lit slightly improves over Opus 4.6.

The baseline `train.py` script provided to all agents achieves a validation BPB of 1.0121, shown as a red dashed line. While the model architecture and training pipeline are identical to the original Autoresearch setup, the evaluation environments differ in several ways (see Appendix F, Table 15): our agents run on H200 GPUs rather than the H100s used in the original benchmark, and they operate in a sandboxed environment with no internet access and slightly different requirements. These differences could account for the gap in starting BPB (1.012 for our agents vs. 0.998 for the Autoresearch work for the same `train.py` script).

The two setups also differ in how the agent interacts with the code. In the original Autoresearch framework, the agent operates as an interactive coding assistant with full shell access: it reads the current `train.py`, makes a targeted edit, runs the script, inspects the output, and either commits or reverts the change within a persistent session. This produces smaller but consistent improvements at each step. In AIRA-DOJO with the greedy scaffold, the agent regenerates the entire `train.py` from scratch at each step, branching from the current best solution with minimal context on past explorations. Agents therefore introduce multiple changes simultaneously, making it harder to isolate which modifications help. Additionally, task instructions are not always respected—agents may omit required imports or preambles, causing evaluation crashes. This translates into trajectories with fewer improvements per run but characterized by larger deltas.

Figure 13 compares the BPB improvement trajectory of the best seed for Opus 4.5 and Opus 4.6, with and without literature access, against the original Autoresearch baseline. We selected the top-performing agents

and the seed achieving the lowest final BPB.

Table 5 details the improvement steps for each agent run shown in Figure 13. Each agent exhibits a distinct optimization strategy. Greedy Opus 4.5 begins by widening the model and removing value embeddings (step 1), then achieves its largest gain ($\Delta = 0.015$) by widening further while restoring them (step 2), followed by targeted refinements: sparsifying value embeddings (step 3) and adding learnable attention output scaling (step 4). Greedy Opus 4.6 starts with multiple simultaneous changes—deeper model, smaller head dimension, different window pattern (step 1)—then partially reverts several of these while increasing weight decay (step 2, $\Delta = 0.017$), and achieves its final improvement by halving the batch size to fit more optimizer steps (step 4). The variants with Literature show a notably different behavior. Opus 4.5 +Literature achieves the single largest improvement in the table ($\Delta = 0.036$) by introducing focal loss to replace cross-entropy around step 15. Opus 4.6 +Literature scores the most number of improvements with steady, moderate gains, progressively tuning depth, batch size, learning rates, and MLP width. These trajectories show how literature access can nudge agents toward qualitatively different optimization strategies.

We refer the reader to Appendix G for a detailed breakdown of the features modified by the agents across all runs and their impact on the final validation BPB. A detailed progression of the 100 runs is given in Figure 28. The full implementation of the solution achieving the lowest validation BPB is given in Appendix H.

Table 5 Improvement steps for the best seed of Greedy Opus 4.5, Opus 4.6, and the two +Literature variants on the Autoresearch task. Each row describes the changes introduced at that step relative to the previous one.

Step	Opus 4.5 (seed 8)	Opus 4.6 (seed 2)	Opus 4.5 +Lit (seed 10)	Opus 4.6 +Lit (seed 4)
Baseline	BPB = 1.012. ReLU^2 MLP, `depth` = 8, `aspect_ratio` = 64, `head_dim` = 128, MHA (6 heads), `window_pattern` = SSSL, `matrix_lr` = 0.04, `embedding_lr` = 0.6, `weight_decay` = 0.2, `total_batch` = 2^{19}, Muon `ns_steps` = 5, value embeddings on alternating layers.			
1	**1.008** ($\Delta = 0.005$) Wider (`aspect` 64→80); removed value embeddings; `matrix_lr` ↑0.045; `weight_decay` ↓0.10; batch $2^{19} \rightarrow 2^{18}$; `ns_steps` 5→3.	**1.007** ($\Delta = 0.005$) Deeper (`depth` 8→10); `aspect` 64→58; `head_dim` 128→64 (9 heads); window SSSL→SSL; `matrix_lr` ↑0.05; batch $2^{19} \rightarrow 2^{18}$.	**1.004** ($\Delta = 0.008$) Deeper (`depth` 8→10); `embedding_lr` ↑0.8; `matrix_lr` ↑0.05; `weight_decay` ↓0.15; `warmdown` 0.5→0.7.	**0.995** ($\Delta = 0.017$) Deeper (`depth` 8→10); batch $2^{19} \rightarrow 2^{18}$; `embedding_lr` ↑0.8; `matrix_lr` ↑0.05; `warmdown` 0.5→0.7.
2	**0.992** ($\Delta = 0.015$) Wider (`aspect` 80→96); restored value embeddings; `matrix_lr` ↓0.038; `embedding_lr` ↓0.55; `weight_decay` ↑0.12; `ns_steps` 3→4.	**0.990** ($\Delta = 0.017$) Reverted `head_dim` 64→128; `aspect` 58→64; window SSL→SSSL; `embedding_lr` ↑0.8; `weight_decay` ↑0.3.	**0.968** ($\Delta = 0.036$) Embed init std 1.0→0.5; `ns_steps` 5→4; `softcap` 15→18; `aspect` 64→56 (narrower); **focal loss** (γ=1.0) replacing CE.	**0.995** ($\Delta < 0.001$) `depth` 10→12; batch $2^{18} \rightarrow 2^{17}$; short window $T/2 \rightarrow T/4$; `ns_steps` 5→4; `embedding_lr` ↑1.2; near-full cosine decay (`warmdown` 0.97).
3	**0.987** ($\Delta = 0.005$) Value embeds sparsified: every-2nd → every-3rd layer + last. `softcap` 18→16. LR retuned (`matrix_lr` 0.038→0.04, `embedding_lr` 0.55→0.58).	**0.990** ($\Delta < 0.001$) `depth` 10→8; `aspect` 64→96 (wider); `embedding_lr` ↑1.0; `weight_decay` ↑0.35; short window halved ($T/2 \rightarrow T/4$); `softcap` 15→20.	—	**0.994** ($\Delta = 0.001$) Train at `seq_len`=1024 (half) for speed; `device_batch` 64→128; full cosine schedule; `final_lr_frac` ↓0.01.
4	**0.986** ($\Delta = 0.001$) Added learnable per-head attention output scale (`attn_out_scale`, init 1.0, lr = 0.1). Applied RMSNorm to attention output before scaling.	**0.987** ($\Delta = 0.003$) Batch $2^{18} \rightarrow 2^{17}$ (more optim steps); `device_batch` 128→64; `matrix_lr` ↓0.035; `embedding_lr` ↓0.8; `warmdown` 0.5→0.7.	—	**0.993** ($\Delta = 0.001$) Reverted to full `seq_len`=2048; `device_batch` 128→32; LRs reduced (`embedding` 1.2→0.7, `matrix` 0.06→0.035); `weight_decay` ↓0.10.
5	—	—	—	**0.988** ($\Delta = 0.005$) `depth` 12→10; `device_batch` 32→64; `x0_lambdas` init ↑0.15; short window $T/8 \rightarrow T/4$; LRs retuned up.
6	—	—	—	**0.985** ($\Delta = 0.003$) `mlp_ratio` 4.0→4.5 (wider MLP); `softcap` 20→18; WSD schedule (`stable`=29%, `warmdown`=70%); `embedding_lr` ↑0.9; `matrix_lr` ↑0.055; `weight_decay` ↓0.12.

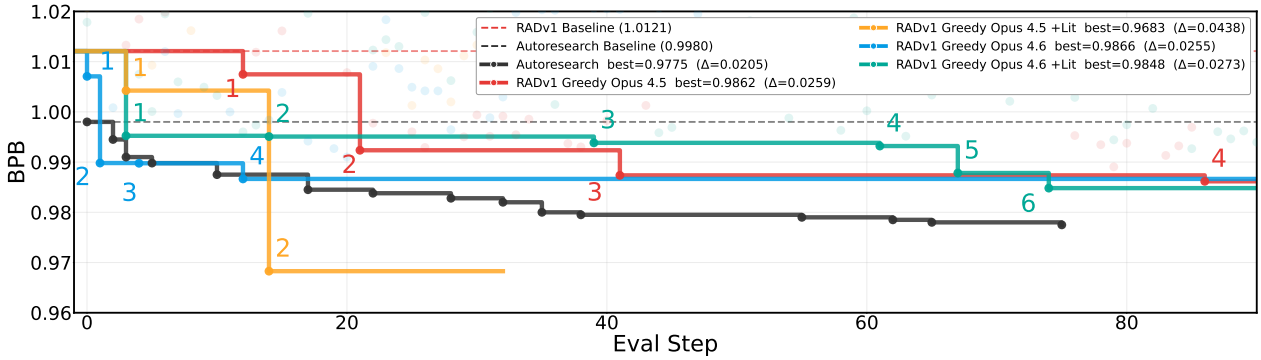


Figure 13 Autoresearch: best-seed BPB progression for Greedy Opus 4.5 and Greedy Opus 4.6, with and without literature. Each step line tracks the cumulative best BPB over evaluation steps, with numbered annotations marking improvements. Faded dots indicate non-improving evaluations. The red dashed line marks the RADv1 baseline (BPB = 1.0121), while the black dashed line marks the original Autoresearch baseline (BPB = 0.998).

7 Conclusion

We assessed agents' capability to design new neural architectures and training methods for foundation models. We did so through two complementary approaches, AIRA-Compose and AIRA-Design, spanning 12 new diverse agentic tasks derived from 3 distinct frameworks: Composer, LRA, and Autoresearch.

With AIRA-Compose, we deployed 11 agents across 340 24-hour and 300 60-hour runs, demonstrating their ability to autonomously formulate hypotheses on the optimal arrangement of computational primitives and construct original architectures. Unlike traditional methods constrained by rigid optimization objectives, these agents creatively and systematically navigate vast combinatorial search spaces. As a result, we introduce 14 novel architectures, AIRAformers and AIRAhybrids, which exhibit favorable downstream task performance, robust scaling properties, and promising loss–efficiency trade-offs.

Through AIRA-Design, we evaluated the ability of agents to design novel attention mechanisms and efficient training loops for a small language model. On the Long Range Arena (LRA) benchmark we deployed 20 agents, and show that Greedy Gemini 3 Pro and Greedy Opus 4.6 achieve a peak accuracy within 3% of the current human SOTA. Strongest agents could efficiently tweak hyperparameters to their advantage and improve their designs. On the open-ended Autoresearch optimization task, we employed 7 agents and showed that augmenting top-performing ones with relevant literature and code repositories shifted their optimization strategies and improved convergence, enabling Greedy Opus 4.5 to surpass the published Autoresearch reference baseline with a final validation BPB of 0.968.

Our results suggest that agent-driven architecture search and design is not only feasible, but it can already yield highly competitive designs to power the next generation of foundation models and agents, ultimately achieving recursive self-improvement.

Limitations and Future Work

AIRA-Compose. AIRA-Compose relies on small-scale proxy evaluation, which does not always faithfully predict large-scale performance. This also limits the number of primitive combinations that can be reliably explored, as translating configurations from a small to a large scale requires careful tuning. Testing AIRA-Compose on advanced harnesses such as AIRA_2 (Hambardzumyan et al., 2026), which can assign multiple GPUs per agent, will allow us to perform NAS on larger models and reduce the proxy–target gap. The aggregation and extrapolation steps also remain non-agentic: future work will introduce agentic tasks that do not just search and evaluate, but also aggregate and scale autonomously. Moreover, our search relies on a single dataset per task; allowing agents to validate candidates across multiple datamixes would yield more robust base patterns. Finally, optimizing only over primitive arrangements effectively underutilizes the agents' capabilities. Enriching the search space with tunable hyperparameters or normalization choices would significantly expand the design space and fully exploit the LLMs' potential. Because AIRA-Compose is built as a modular, LLM-agnostic template, we are confident it can be extended to incorporate these enhancements.

AIRA-Design. On the LRA benchmark, agents still fell short of producing paradigm-shifting scientific innovation in mechanistic design. The discovered architectures largely recombine and adapt ideas from prior work (e.g., Performer, Longformer, Conformer) rather than introducing new theoretical insights, suggesting that current agents excel at engineering-level synthesis and adaptation but not yet at genuine algorithmic innovation. One-shot agents failed to produce any valid submissions, confirming that iterative refinement and debugging are essential for low-level code generation tasks. The configurable setup generally degraded performance for weaker agents, which struggle when the design space is expanded. Additionally, LRA tasks expose a strong training bias toward PyTorch, which limits LLM proficiency in JAX/Flax—the framework required by the LRA benchmark. On the Autoresearch task, the greedy scaffold's full-file regeneration paradigm prevents surgical, single-variable edits, making causal attribution of improvements challenging and leading to compound modifications that are hard to interpret. Literature access does not uniformly help: some agents underperform with additional context, suggesting that effectively integrating prior knowledge into optimization strategies remains an open challenge. Future work could include: testing AIRA-Design tasks on more advanced harnesses with persistent state; employing agents with tool-use capabilities for interactive debugging; extending the LRA evaluation to more recent long-range benchmarks; and developing scaffolds that enable incremental, interpretable code modifications rather than full rewrites.

References

Samira Abnar, Harshay Shah, Dan Busbridge, Alaaeldin El-Nouby, Joshua M Susskind, and Vimal Thilak. Parameters vs FLOPs: Scaling Laws for Optimal Sparsity for Mixture-of-Experts Language Models. In *International Conference on Machine Learning*, pages 204–230. PMLR, 2025.

Bilge Acun, Prasoon Sinha, Newsha Ardalani, Sangmin Bae, Alicia Golden, Chien-Yu Lin, Meghana Madhyastha, Fei Sun, Neeraja J Yadwadkar, and Carole-Jean Wu. Composer: A search framework for hybrid neural architecture design. *arXiv preprint arXiv:2510.00379*, 2025.

Bo Adler, Niket Agarwal, Ashwath Aithal, Dong H Anh, Pallab Bhattacharya, Annika Brundyn, Jared Casper, Bryan Catanzaro, Sharon Clay, Jonathan Cohen, et al. Nemotron-4 340b technical report. *arXiv preprint arXiv:2406.11704*, 2024.

Joshua Ainslie, James Lee-Thorp, Michiel De Jong, Yury Zemlyanskiy, Federico Lebrón, and Sumit Sanghai. Gqa: Training generalized multi-query transformer models from multi-head checkpoints. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pages 4895–4901, 2023.

Chris Alexiuk, Chintan Patel, and NVIDIA. Nemotron 3 Super: Maximum Compute Efficiency for Complex Multi-Agent Applications, 2026. URL https://developer.nvidia.com/blog.

Allen Institute for AI. Olmo-Hybrid: Architectural Improvements for Language Modeling. 2026. URL https://allenai.org/papers/olmo-hybrid.

Pierre Andrews, Amine Benhalloum, Gerard Moreno-Torres Bertran, Matteo Bettini, Amar Budhiraja, Ricardo Silveira Cabral, Virginie Do, Romain Froger, Emilien Garreau, Jean-Baptiste Gaya, Hugo Laurençon, Maxime Lecanu, Kunal Malkan, Dheeraj Mekala, Pierre Ménard, Grégoire Mialon, Ulyana Piterbarg, Mikhail Plekhanov, Mathieu Rita, Andrey Rusakov, Thomas Scialom, Vladislav Vorotilov, Mengjue Wang, and Ian Yu. ARE: Scaling Up Agent Environments and Evaluations, 2025. URL https://arxiv.org/abs/2509.17158.

Anthropic. Claude Code: An Agentic CLI for Repository-Level Engineering. Technical Report, 2025. URL https://www.anthropic.com/news/claude-code. Accessed: 2026-03-27.

Sangmin Bae, Bilge Acun, Haroun Habeeb, Seungyeon Kim, Chien-Yu Lin, Liang Luo, Junjie Wang, and Carole-Jean Wu. Hybrid architectures for language models: Systematic analysis and design insights. *arXiv preprint arXiv:2510.04800*, 2025.

Aarti Basant, Abhijit Khairnar, Abhijit Paithankar, Abhinav Khattar, Adithya Renduchintala, Aditya Malte, Akhiad Bercovich, Akshay Hazare, Alejandra Rico, Aleksander Ficek, et al. Nvidia nemotron nano 2: An accurate and efficient hybrid mamba-transformer reasoning model. *arXiv preprint arXiv:2508.14444*, 2025.

Ali Behrouz, Peilin Zhong, and Vahab Mirrokni. Titans: Learning to memorize at test time. *arXiv preprint arXiv:2501.00663*, 2024.

Akhiad Bercovich, Itay Levy, Izik Golan, Mohammad Dabbah, Ran El-Yaniv, Omri Puny, Ido Galil, Zach Moshe, Tomer Ronen, Najeeb Nabwani, et al. Llama-nemotron: Efficient reasoning models. *arXiv preprint arXiv:2505.00949*, 2025.

Aaron Blakeman, Aarti Basant, Abhinav Khattar, Adithya Renduchintala, Akhiad Bercovich, Aleksander Ficek, Alexis Bjorlin, Ali Taghibakhshi, Amala Sanjay Deshmukh, Ameya Sunil Mahabaleshwarkar, et al. Nemotron-h: A family of accurate and efficient hybrid mamba-transformer models. *arXiv preprint arXiv:2504.03624*, 2025.

Yelysei Bondarenko, Markus Nagel, and Tijmen Blankevoort. Quantizable transformers: Removing outliers by helping attention heads do nothing. *Advances in Neural Information Processing Systems*, 36:75067–75096, 2023.

Hector Borobia, Elies Seguí-Mas, and Guillermina Tormo-Carbó. Functional Component Ablation Reveals Specialization Patterns in Hybrid Language Model Architectures. *arXiv preprint arXiv:2603.22473*, 2026.

Quentin Carbonneaux, Gal Cohen, Jonas Gehring, Jacob Kahn, Jannik Kossen, Felix Kreuk, Emily McMilin, Michel Meyer, Yuxiang Wei, David Zhang, et al. CWM: An open-weights LLM for research on code generation with world models. *arXiv preprint arXiv:2510.02387*, 2025.

Yilong Chen, Junyuan Shang, Zhengyu Zhang, Jiawei Sheng, Tingwen Liu, Shuohuan Wang, Yu Sun, Hua Wu, and Haifeng Wang. Mixture of hidden-dimensions transformer. *arXiv preprint arXiv:2412.05644*, 2024.

Tri Dao and Albert Gu. Transformers are ssms: Generalized models and efficient algorithms through structured state space duality. *arXiv preprint arXiv:2405.21060*, 2024.

Aaron Defazio, Xingyu Yang, Harsh Mehta, Konstantin Mishchenko, Ahmed Khaled, and Ashok Cutkosky. The road less scheduled. *Advances in Neural Information Processing Systems*, 37:9974–10007, 2024.

Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. Bert: Pre-training of deep bidirectional transformers for language understanding. In *Proceedings of the 2019 conference of the North American chapter of the association for computational linguistics: human language technologies, volume 1 (long and short papers)*, pages 4171–4186, 2019.

Shizhe Diao, Yu Yang, Yonggan Fu, Xin Dong, Dan Su, Markus Kliegl, Zijia Chen, Peter Belcak, Yoshi Suhara, Hongxu Yin, et al. Climb: Clustering-based iterative data mixture bootstrapping for language model pre-training. *arXiv e-prints*, pages arXiv–2504, 2025.

Juechu Dong, Boyuan Feng, Driss Guessous, Yanbo Liang, and Horace He. Flex attention: A programming model for generating optimized attention kernels. *arXiv preprint arXiv:2412.05496*, 2(3):4, 2024a.

Xin Dong et al. Hymba: A Hybrid Mamba-Transformer Architecture for Small Language Models. *arXiv preprint arXiv:2411.13676*, 2024b.

Thomas Elsken, Jan Hendrik Metzen, and Frank Hutter. Neural architecture search: A survey. *Journal of Machine Learning Research*, 20(55):1–21, 2019.

Jonas Geiping and Tom Goldstein. Cramming: Training a Language Model on a single GPU in one day. In *International Conference on Machine Learning*, pages 11117–11143. PMLR, 2023.

Fabian Gloeckle, Badr Youbi Idrissi, Baptiste Rozière, David Lopez-Paz, and Gabriel Synnaeve. Better & faster large language models via multi-token prediction. *arXiv preprint arXiv:2404.19737*, 2024.

Irving John Good. Speculations concerning the first ultraintelligent machine. In *Advances in computers*, volume 6, pages 31–88. Elsevier, 1966.

Google DeepMind. Gemini 3.1 Pro: A Smarter Model for Your Most Complex Tasks. Google DeepMind Blog, 2026. URL https://blog.google/innovation-and-ai/models-and-research/gemini-models/gemini-3-1-pro/. Accessed: 2026-03-27.

Google Gemini Team. Gemini 2.5: Pushing the frontier with advanced reasoning, multimodality, long context, and next generation agentic capabilities. *arXiv preprint arXiv:2507.06261*, 2025.

Google Gemini Team, Petko Georgiev, Ving Ian Lei, Ryan Burnell, Libin Bai, Anmol Gulati, Garrett Tanzer, Damien Vincent, Zhufeng Pan, Shibo Wang, et al. Gemini 1.5: Unlocking multimodal understanding across millions of tokens of context. *arXiv preprint arXiv:2403.05530*, 2024.

Albert Gu and Tri Dao. Mamba: Linear-time sequence modeling with selective state spaces. In *First conference on language modeling*, 2024.

Yuxian Gu, Qinghao Hu, Shang Yang, Haocheng Xi, Junyu Chen, Song Han, and Han Cai. Jet-nemotron: Efficient language model with post neural architecture search. *arXiv preprint arXiv:2508.15884*, 2025.

Alexander Hägele, Elie Bakouch, Atli Kosson, Loubna B Allal, Leandro Von Werra, and Martin Jaggi. Scaling laws and compute-optimal training beyond fixed training durations. *Advances in Neural Information Processing Systems*, 37:76232–76264, 2024.

Karen Hambardzumyan, Nicolas Baldwin, Edan Toledo, Rishi Hazra, Michael Kuchnik, Bassel Al Omari, Thomas Simon Foster, Anton Protopopov, Jean-Christophe Gagnon-Audet, Ishita Mediratta, et al. AIRA_2: Overcoming Bottlenecks in AI Research Agents. *arXiv preprint arXiv:2603.26499*, 2026.

Alec M. Hammond, Aram Markosyan, Aman Dontula, Simon Mahns, Zacharias Fisches, Dmitrii Pedchenko, Keyur Muzumdar, Natacha Supper, Mark Saroufim, Joe Isaacson, Laura Wang, Warren Hunt, Kaustubh Gondkar, Roman Levenstein, Gabriel Synnaeve, Richard Li, Jacob Kahn, and Ajit Mathews. Agentic Operator Generation for ML ASICs. 2025. URL https://arxiv.org/abs/2512.10977.

Chaoyue He, Xin Zhou, Di Wang, Hong Xu, Wei Liu, and Chunyan Miao. The AutoResearch Moment: From Experimenter to Research Director. 2026.

Pin-Lun Hsu, Yun Dai, Vignesh Kothapalli, Qingquan Song, Shao Tang, Siyu Zhu, Steven Shimizu, Shivam Sahni, Haowen Ning, and Yanning Chen. Liger kernel: Efficient triton kernels for llm training. *arXiv preprint arXiv:2410.10989*, 2024.

Shengding Hu, Yuge Tu, Xu Han, Chaoqun He, Ganqu Cui, Xiang Long, Zhi Zheng, Yewei Fang, Yuxiang Huang, Weilin Zhao, et al. MiniCPM: Unveiling the potential of small language models with scalable training strategies. *arXiv preprint arXiv:2404.06395*, 2024.

Zihao Huang, Qiyang Min, Hongzhi Huang, Defa Zhu, Yutao Zeng, Ran Guo, and Xun Zhou. Ultra-sparse memory network. *arXiv preprint arXiv:2411.12364*, 2024.

Rustem Islamov, Roman Machacek, Aurelien Lucchi, Antonio Silveti-Falls, Eduard Gorbunov, and Volkan Cevher. On the Role of Batch Size in Stochastic Conditional Gradient Methods. *arXiv preprint arXiv:2603.21191*, 2026.

Nilesh Jain, Rohit Yadav, Sagar Kotian, and Claude AI. AutoResearch-RL: Perpetual Self-Evaluating Reinforcement Learning Agents for Autonomous Neural Architecture Discovery. *arXiv preprint arXiv:2603.07300*, 2026.

Mojan Javaheripi, Gustavo de Rosa, Subhabrata Mukherjee, Shital Shah, Tomasz Religa, Caio Cesar Teodoro Mendes, Sebastien Bubeck, Farinaz Koushanfar, and Debadeepta Dey. Litetransformersearch: Training-free neural architecture search for efficient language models. *Advances in Neural Information Processing Systems*, 35:24254–24267, 2022.

Ganesh Jawahar, Subhabrata Mukherjee, Xiaodong Liu, Young Jin Kim, Muhammad Abdul-Mageed, Laks Lakshmanan, Ahmed Hassan, Sebastien Bubeck, Jianfeng Gao, et al. AutoMoE: Heterogeneous mixture-of-experts with adaptive computation for efficient neural machine translation. In *Findings of the Association for Computational Linguistics: ACL 2023*, pages 9116–9132, 2023.

Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. SWE-bench: Can Language Models Resolve Real-World GitHub Issues?, 2024. URL https://arxiv.org/abs/2310.06770.

Keller Jordan, Jeremy Bernstein, Brendan Rappazzo, @fernbear.bsky.social, Boza Vlado, You Jiacheng, Franz Cesista, Braden Koszarsky, and @Grad62304977. modded-nanogpt: Speedrunning the NanoGPT baseline, 2024. URL https://github.com/KellerJordan/modded-nanogpt.

Andrej Karpathy. nanochat: The best ChatGPT that $100 can buy, 2025. URL https://github.com/karpathy/nanochat.

Andrej Karpathy. Autoresearch. https://github.com/karpathy/autoresearch, 2026. GitHub repository.

Andrej Karpathy and Dwarkesh Patel. Andrej karpathy — agi is still a decade away. The Dwarkesh Podcast, Oct 2025.

Sanghoon Kim, Dahyun Kim, Chanjun Park, Wonsung Lee, Wonho Song, Yunsu Kim, Hyeonwoo Kim, Yungi Kim, Hyeonju Lee, Jihoo Kim, et al. Solar 10.7 b: Scaling large language models with simple yet effective depth up-scaling. In *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 6: Industry Track)*, pages 23–35, 2024.

Rémi Leblond et al. AlphaCode 2 Technical Report, 2023. URL https://storage.googleapis.com/deepmind-media/AlphaCode2/AlphaCode2_Tech_Report.pdf.

Jaerin Lee, Bong Gyun Kang, Kihoon Kim, and Kyoung Mu Lee. Grokfast: Accelerated grokking by amplifying slow gradients. *arXiv preprint arXiv:2405.20233*, 2024.

Houyi Li, Wenzhen Zheng, Qiufeng Wang, Hanshan Zhang, Zili Wang, Shijie Xuyang, Yuantao Fan, Zhenyu Ding, Haoying Wang, Ning Ding, et al. Predictable Scale: Part I, Step Law–Optimal Hyperparameter Scaling Law in Large Language Model Pretraining. *arXiv preprint arXiv:2503.04715*, 2025.

Jeffrey Li, Alex Fang, Georgios Smyrnis, Maor Ivgi, Matt Jordan, Samir Yitzhak Gadre, Hritik Bansal, Etash Guha, Sedrick Scott Keh, Kushal Arora, et al. Datacomp-lm: In search of the next generation of training sets for language models. *Advances in Neural Information Processing Systems*, 37:14200–14282, 2024a.

Ziming Li, Qianbo Zang, David Ma, Jiawei Guo, Tuney Zheng, Minghao Liu, Xinyao Niu, Yue Wang, Jian Yang, Jiaheng Liu, Wanjun Zhong, Wangchunshu Zhou, Wenhao Huang, and Ge Zhang. AutoKaggle: A Multi-Agent Framework for Autonomous Data Science Competitions, 2024b. URL https://arxiv.org/abs/2410.20424.

Gang Liao, Hongsen Qin, Ying Wang, Alicia Golden, Michael Kuchnik, Yavuz Yetim, Jia Jiunn Ang, Chunli Fu, Yihan He, Samuel Hsia, Zewei Jiang, Dianshi Li, Uladzimir Pashkevich, Varna Puvvada, Feng Shi, Matt Steiner, Ruichao Xiao, Nathan Yan, Xiayu Yu, Zhou Fang, Roman Levenstein, Kunming Ho, Haishan Zhu, Alec Hammond, Richard Li, Ajit Mathews, Kaustubh Gondkar, Abdul Zainul-Abedin, Ketan Singh, Hongtao Yu, Wenyuan Chi, Barney Huang, Sean Zhang, Noah Weller, Zach Marine, Wyatt Cook, Carole-Jean Wu, and Gaoxiang Liu. KernelEvolve: Scaling Agentic Kernel Coding for Heterogeneous AI Accelerators at Meta. 2026. URL https://arxiv.org/abs/2512.23236.

Opher Lieber, Barak Lenz, Hofit Bata, Gal Cohen, Jhonathan Osin, Itay Dalmedigos, Erez Safahi, Shaked Meirom, Yonatan Belinkov, Shai Shalev-Shwartz, et al. Jamba: A hybrid transformer-mamba language model. *arXiv preprint arXiv:2403.19887*, 2024.

Zhixuan Lin, Evgenii Nikishin, Xu Owen He, and Aaron Courville. Forgetting transformer: Softmax attention with a forget gate. *arXiv preprint arXiv:2503.02130*, 2025.

Hanxiao Liu, Zihang Dai, David So, and Quoc V Le. Pay attention to MLPs. *Advances in neural information processing systems*, 34:9204–9215, 2021a.

Hong Liu, Zhiyuan Li, David Hall, Percy Liang, and Tengyu Ma. Sophia: A scalable stochastic second-order optimizer for language model pre-training. *arXiv preprint arXiv:2305.14342*, 2023.

Jingyuan Liu, Jianlin Su, Xingcheng Yao, Zhejun Jiang, Guokun Lai, Yulun Du, Yidao Qin, Weixin Xu, Enzhe Lu, Junjie Yan, et al. Muon is scalable for llm training. *arXiv preprint arXiv:2502.16982*, 2025.

Yuqiao Liu, Yanan Sun, Bing Xue, Mengjie Zhang, Gary G Yen, and Kay Chen Tan. A survey on evolutionary neural architecture search. *IEEE transactions on neural networks and learning systems*, 34(2):550–570, 2021b.

Zechun Liu, Changsheng Zhao, Forrest Iandola, Chen Lai, Yuandong Tian, Igor Fedorov, Yunyang Xiong, Ernie Chang, Yangyang Shi, Raghuraman Krishnamoorthi, et al. MobileLLM: Optimizing sub-billion parameter language models for on-device use cases. In *Forty-first International Conference on Machine Learning*, 2024.

Llama Team. The Llama 3 Herd of Models. *arXiv preprint arXiv:2407.21783*, 2024.

Enzhe Lu, Zhejun Jiang, Jingyuan Liu, Yulun Du, Tao Jiang, Chao Hong, Shaowei Liu, Weiran He, Enming Yuan, Yuzhi Wang, et al. MoBA: Mixture of block attention for long-context LLMs. *arXiv preprint arXiv:2502.13189*, 2025.

Alisia Lupidi, Bhavul Gauri, Thomas Simon Foster, Bassel Al Omari, Despoina Magka, Alberto Pepe, Alexis Audran-Reiss, Muna Aghamelu, Nicolas Baldwin, Lucia Cipolina-Kun, et al. AIRS-Bench: a Suite of Tasks for Frontier AI Research Science Agents. *arXiv preprint arXiv:2602.06855*, 2026.

Xuezhe Ma, Xiaomeng Yang, Wenhan Xiong, Beidi Chen, Lili Yu, Hao Zhang, Jonathan May, Luke Zettlemoyer, Omer Levy, and Chunting Zhou. Megalodon: Efficient llm pretraining and inference with unlimited context length. *Advances in Neural Information Processing Systems*, 37:71831–71854, 2024.

Saaketh Narayan, Abhay Gupta, Mansheej Paul, and Davis Blalock. μnit Scaling: Simple and Scalable FP8 LLM Training. *arXiv preprint arXiv:2502.05967*, 2025.

Deepak Nathani, Lovish Madaan, Nicholas Roberts, Nikolay Bashlykov, Ajay Menon, Vincent Moens, Amar Budhiraja, Despoina Magka, Vladislav Vorotilov, Gaurav Chaurasia, et al. MLGym: A New Framework and Benchmark for Advancing AI Research Agents. *arXiv preprint arXiv:2502.14499*, 2025.

Alexander Novikov, Ngân Vũ, Marvin Eisenberger, Emilien Dupont, Po-Sen Huang, Adam Zsolt Wagner, Sergey Shirobokov, Borislav Kozlovskii, Francisco JR Ruiz, Abbas Mehrabian, et al. AlphaEvolve: A coding agent for scientific and algorithmic discovery. *arXiv preprint arXiv:2506.13131*, 2025.

OpenAI Team. gpt-oss-120b & gpt-oss-20b model card. *arXiv preprint arXiv:2508.10925*, 2025.

Matteo Pagliardini, Pierre Ablin, and David Grangier. AdEMAMix: Better and Faster Training with Older Gradients. In *OPT 2024: Optimization for Machine Learning*, 2024.

Bo Peng, Eric Alcaide, Quentin Anthony, Alon Albalak, Samuel Arcadinho, Stella Biderman, Huanqi Cao, Xin Cheng, Michael Chung, Leon Derczynski, et al. Rwkv: Reinventing rnns for the transformer era. In *Findings of the association for computational linguistics: EMNLP 2023*, pages 14048–14077, 2023a.

Bowen Peng, Jeffrey Quesnelle, Honglu Fan, and Enrico Shippole. YaRN: Efficient context window extension of large language models. *arXiv preprint arXiv:2309.00071*, 2023b.

Michael Poli, Armin W Thomas, Eric Nguyen, Pragaash Ponnusamy, Björn Deiseroth, Kristian Kersting, Taiji Suzuki, Brian Hie, Stefano Ermon, Christopher Re, et al. Mechanistic Design and Scaling of Hybrid Architectures. In *International Conference on Machine Learning*, pages 40908–40950. PMLR, 2024.

Reiner Pope, Sholto Douglas, Aakanksha Chowdhery, Jacob Devlin, James Bradbury, Jonathan Heek, Kefan Xiao, Shivani Agrawal, and Jeff Dean. Efficiently scaling transformer inference. *Proceedings of machine learning and systems*, 5:606–624, 2023.

Alec Radford, Karthik Narasimhan, Tim Salimans, Ilya Sutskever, et al. Improving language understanding by generative pre-training. 2018.

Babak Rahmani, Mark Schöne, Heiner Kremer, Fabian Falck, Hitesh Ballani, and Jannes Gladrow. Implicit language models are RNNs: balancing parallelization and expressivity. *arXiv preprint arXiv:2502.07827*, 2025.

Esteban Real, Alok Aggarwal, Yanping Huang, and Quoc V Le. Regularized evolution for image classifier architecture search. In *Proceedings of the aaai conference on artificial intelligence*, volume 33, pages 4780–4789, 2019.

Liliang Ren, Yang Liu, Yadong Lu, Yelong Shen, Chen Liang, and Weizhu Chen. Samba: Simple hybrid state space models for efficient unlimited context language modeling. *arXiv preprint arXiv:2406.07522*, 2024.

Pengzhen Ren, Yun Xiao, Xiaojun Chang, Po-Yao Huang, Zhihui Li, Xiaojiang Chen, and Xin Wang. A comprehensive survey of neural architecture search: Challenges and solutions. *ACM Computing Surveys (CSUR)*, 54(4):1–34, 2021.

Bernardino Romera-Paredes, Mohammadamin Barekatain, Alexander Novikov, Matej Balog, M Pawan Kumar, Emilien Dupont, Francisco JR Ruiz, Jordan S Ellenberg, Pengming Wang, Omar Fawzi, et al. Mathematical discoveries from program search with large language models. *Nature*, 625(7995):468–475, 2024.

Samuel Schmidgall, Yusheng Su, Ze Wang, Ximeng Sun, Jialian Wu, Xiaodong Yu, Jiang Liu, Michael Moor, Zicheng Liu, and Emad Barsoum. Agent laboratory: Using llm agents as research assistants. *Findings of the Association for Computational Linguistics: EMNLP 2025*, pages 5977–6043, 2025.

Jürgen Schmidhuber. *Evolutionary principles in self-referential learning, or on learning how to learn: the meta-meta-... hook*. PhD thesis, Technische Universität München, 1987.

Jürgen Schmidhuber. Gödel machines: Self-referential universal problem solvers making provably optimal self-improvements. *arXiv preprint cs.LO/0309048*, 2003.

Minju Seo, Jinheon Baek, Seongyun Lee, and Sung Ju Hwang. Paper2code: Automating code generation from scientific papers in machine learning. *arXiv preprint arXiv:2504.17192*, 2025.

Noam Shazeer. Glu variants improve transformer. *arXiv preprint arXiv:2002.05202*, 2020.

Soumye Singhal, Jiaqi Zeng, Alexander Bukharin, Yian Zhang, Gerald Shen, Ameya Sunil Mahabaleshwarkar, Bilal Kartal, Yoshi Suhara, Akhiad Bercovich, Itay Levy, et al. Llama-nemotron: Efficient reasoning models. In *The Exploration in AI Today Workshop at ICML 2025*, 2025.

Giulio Starace, Oliver Jaffe, Dane Sherburn, James Aung, Jun Shern Chan, Leon Maksin, Rachel Dias, Evan Mays, Benjamin Kinsella, Wyatt Thompson, et al. PaperBench: Evaluating AI's Ability to Replicate AI Research. *arXiv preprint arXiv:2504.01848*, 2025.

Yutao Sun, Li Dong, Shaohan Huang, Shuming Ma, Yuqing Xia, Jilong Xue, Jianyong Wang, and Furu Wei. Retentive network: A successor to transformer for large language models. *arXiv preprint arXiv:2307.08621*, 2023.

Shawn Tan, Songlin Yang, Aaron Courville, Rameswar Panda, and Yikang Shen. Scaling stick-breaking attention: An efficient implementation and in-depth study. *arXiv preprint arXiv:2410.17980*, 2024.

Yi Tay, Mostafa Dehghani, Samira Abnar, Yikang Shen, Dara Bahri, Philip Pham, Jinfeng Rao, Liu Yang, Sebastian Ruder, and Donald Metzler. Long range arena: A benchmark for efficient transformers. *arXiv preprint arXiv:2011.04006*, 2020.

Yi Tay, Mostafa Dehghani, Dara Bahri, and Donald Metzler. Efficient transformers: A survey. *ACM Computing Surveys*, 55(6):1–28, 2022.

Howe Tissue, Venus Wang, and Lu Wang. Scaling law with learning rate annealing. *arXiv preprint arXiv:2408.11029*, 2024.

Edan Toledo, Karen Hambardzumyan, Martin Josifoski, Rishi Hazra, Nicolas Baldwin, Alexis Audran-Reiss, Michael Kuchnik, Despoina Magka, Minqi Jiang, Alisia Maria Lupidi, Andrei Lupu, Roberta Raileanu, Kelvin Niu, Tatiana Shavrina, Jean-Christophe Gagnon-Audet, Michael Shvartsman, Shagun Sodhani, Alexander H. Miller, Abhishek Charnalia, Derek Dunfield, Carole-Jean Wu, Pontus Stenetorp, Nicola Cancedda, Jakob Nicolaus Foerster, and Yoram Bachrach. AI Research Agents for Machine Learning: Search, Exploration, and Generalization in MLE-bench, 2025. URL https://arxiv.org/abs/2507.02554.

Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. *Advances in neural information processing systems*, 30, 2017.

Lei Wang, Chen Ma, Xueyang Feng, Zeyu Zhang, Hao Yang, Jingsen Zhang, Zhiyuan Chen, Jiakai Tang, Xu Chen, Yankai Lin, et al. A survey on large language model based autonomous agents. *Frontiers of Computer Science*, 18 (6):186345, 2024.

Colin White, Mahmoud Safari, Rhea Sukthanker, Binxin Ru, Thomas Elsken, Arber Zela, Debadeepta Dey, and Frank Hutter. Neural architecture search: Insights from 1000 papers. *arXiv preprint arXiv:2301.08727*, 2023.

Haocheng Xi, Han Cai, Ligeng Zhu, Yao Lu, Kurt Keutzer, Jianfei Chen, and Song Han. COAT: Compressing optimizer states and activation for memory-efficient fp8 training. *arXiv preprint arXiv:2410.19313*, 2024.

Yanzheng Xiang, Hanqi Yan, Shuyin Ouyang, Lin Gui, and Yulan He. SciReplicate-Bench: Benchmarking llms in agent-driven algorithmic reproduction from research papers. *arXiv preprint arXiv:2504.00255*, 2025.

Shufang Xie, Huishuai Zhang, Junliang Guo, Xu Tan, Jiang Bian, Hany Hassan Awadalla, Arul Menezes, Tao Qin, and Rui Yan. Residual: Transformer with dual residual connections. *arXiv preprint arXiv:2304.14802*, 2023.

Mingyu Xu, Wei Cheng, Bingning Wang, and Weipeng Chen. KV shifting attention enhances language modeling. *arXiv preprint arXiv:2411.19574*, 2024.

Yutaro Yamada, Robert Tjarko Lange, Cong Lu, Shengran Hu, Chris Lu, Jakob Foerster, Jeff Clune, and David Ha. The AI Scientist-v2: Workshop-Level Automated Scientific Discovery via Agentic Tree Search, 2025. URL https://arxiv.org/abs/2504.08066.

An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, Chengen Huang, Chenxu Lv, et al. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*, 2025.

Ge Yang, Edward Hu, Igor Babuschkin, Szymon Sidor, Xiaodong Liu, David Farhi, Nick Ryder, Jakub Pachocki, Weizhu Chen, and Jianfeng Gao. Tuning large neural networks via zero-shot hyperparameter transfer. *Advances in Neural Information Processing Systems*, 34:17084–17097, 2021.

Greg Yang, James B Simon, and Jeremy Bernstein. A spectral condition for feature learning. *arXiv preprint arXiv:2310.17813*, 2023.

Yufeng Yang et al. Gated DeltaNet: A Linear Attention Alternative to SSMs. *arXiv preprint arXiv:2412.06464*, 2024.

Tianzhu Ye, Li Dong, Yuqing Xia, Yutao Sun, Yi Zhu, Gao Huang, and Furu Wei. Differential Transformer. In *The Thirteenth International Conference on Learning Representations*, 2025.

Xunjian Yin, Xinyi Wang, Liangming Pan, Li Lin, Xiaojun Wan, and William Yang Wang. Gödel agent: A self-referential agent framework for recursively self-improvement. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 27890–27913, 2025.

Jingyang Yuan, Huazuo Gao, Damai Dai, Junyu Luo, Liang Zhao, Zhengyan Zhang, Zhenda Xie, Yuxing Wei, Lean Wang, Zhiping Xiao, et al. Native sparse attention: Hardware-aligned and natively trainable sparse attention. In

Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), pages 23078–23097, 2025.

Jianyu Zhang, Niklas Nolte, Ranajoy Sadhukhan, Beidi Chen, and Leon Bottou. Memory mosaics. In *The Thirteenth International Conference on Learning Representations*, 2025.

Zhengyan Zhang, Yixin Song, Guanghui Yu, Xu Han, Yankai Lin, Chaojun Xiao, Chenyang Song, Zhiyuan Liu, Zeyu Mi, and Maosong Sun. ReLU^2 Wins: Discovering Efficient Activation Functions for Sparse LLMs. *arXiv preprint arXiv:2402.03804*, 2024.

Bingchen Zhao, Despoina Magka, Minqi Jiang, Xian Li, Roberta Raileanu, Tatiana Shavrina, Jean-Christophe Gagnon-Audet, Kelvin Niu, Shagun Sodhani, Michael Shvartsman, Andrei Lupu, Alisia Lupidi, Edan Toledo, Karen Hambardzumyan, Martin Josifoski, Thomas Foster, Lucia Cipolina-Kun, Abhishek Charnalia, Derek Dunfield, Alexander H. Miller, Oisin Mac Aodha, Jakob Foerster, and Yoram Bachrach. The Automated LLM Speedrunning Benchmark: Reproducing NanoGPT Improvements, 2025. URL https://arxiv.org/abs/2506.22419.

Barret Zoph and Quoc Le. Neural Architecture Search with Reinforcement Learning. In *International Conference on Learning Representations*, 2017.

Barret Zoph, Vijay Vasudevan, Jonathon Shlens, and Quoc V Le. Learning transferable architectures for scalable image recognition. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 8697–8710, 2018.

Appendix

A Related Work

Hybrid Language Models. Transformers are the backbone of modern LLMs (Llama Team, 2024; Google Gemini Team et al., 2024; Google Gemini Team, 2025; OpenAI Team, 2025), but the quadratic cost of self-attention limits long-context processing. To address this, the field is moving towards hybrid architectures (Dao and Gu, 2024; Ren et al., 2024; Gu and Dao, 2024). These models interleave diverse computational primitives (e.g., Attention, SSMs) to balance efficiency with expressiveness (Bae et al., 2025). SSMs like Mamba-2 (Dao and Gu, 2024), Gated DeltaNet (GDN) (Yang et al., 2024), Samba (Ren et al., 2024), and Hymba (Dong et al., 2024b) represent a compelling alternative because of their linear complexity, striking a balance of performance and cost-efficiency while still successfully capturing complex data dynamics (Rahmani et al., 2025). The interleaving of SSM and Attention blocks does not act as a Transformer fallback, but it serves as the language modeling backbone of hybrid architectures (Borobia et al., 2026), yielding models that surpass the theoretical limits of either primitive alone (Allen Institute for AI, 2026). This paradigm shift powers several SOTA models, including Qwen3-Next (Yang et al., 2025), Nemotron-H (Blakeman et al., 2025), Nemotron Nano 2 (Basant et al., 2025) and Nemotron 3 Super (Alexiuk et al., 2026).

Neural Architecture Search. Hybrid models design implies a vast combinatorial search space, and manual tuning is not sufficient, hence researchers rely on Neural Architecture Search (NAS) (Elsken et al., 2019; Liu et al., 2021b; White et al., 2023). Historically, NAS has been studied for smaller models using reinforcement learning (Zoph and Le, 2017) or evolutionary algorithms (Real et al., 2019) to combine small "cells" into full architectures (Zoph et al., 2018). Applying traditional NAS methods to LLMs is prohibitive, and most attempts rely on approximations. AutoMOE (Jawahar et al., 2023) utilizes a supernet approach, which trains an over-parameterized network to carve smaller sub-networks from it. LiteTransformerSearch (Javaheripi et al., 2022) employs zero-cost performance proxies, estimating a model's fully-trained accuracy based on untrained properties like initial gradient flow and parameter count. Recent works have introduced the Post Neural Architecture Search (PostNAS) paradigm, which prunes existing pretrained models or replaces standard attention blocks with linear variants (Bercovich et al., 2025; Gu et al., 2025). Mechanistic design principles are used to analytically derive the optimal allocation of diverse layers (Poli et al., 2024), while empirical search frameworks like Composer (Acun et al., 2025) bypass computational bottlenecks using small scale proxies to identify block arrangements that reliably perform well at scale.

Agents towards Recursive Self-Improvement. Advances in agentic reasoning have enabled LLMs to function as autonomous researchers, capable of scientific and algorithmic discovery (Novikov et al., 2025; Anthropic, 2025; Google DeepMind, 2026). More benchmarks are emerging to evaluate agentic capabilities: AIRS-Bench (Lupidi et al., 2026), assessing the full research cycle pipeline, MLGym (Nathani et al., 2025) and MLE-bench (Toledo et al., 2025), focusing on engineering tasks, and PaperBench (Starace et al., 2025), targeting scientific paper reproduction. An immediate next step in agentic research is represented by agents that iteratively discover and implement superior versions of themselves (Good, 1966; Schmidhuber, 1987; Yin et al., 2025). Recent efforts include the Automated LLM Speedrunning Benchmark (Zhao et al., 2025), which challenges agents to minimize training time to a target loss, and Autoresearch (Karpathy, 2026), which automates model code and training loop optimization. These agentic workflows are facilitated by *nanochat* (Karpathy, 2025), a minimal, hackable environment spanning the entire LLM pipeline.

B Small Scale Ranking

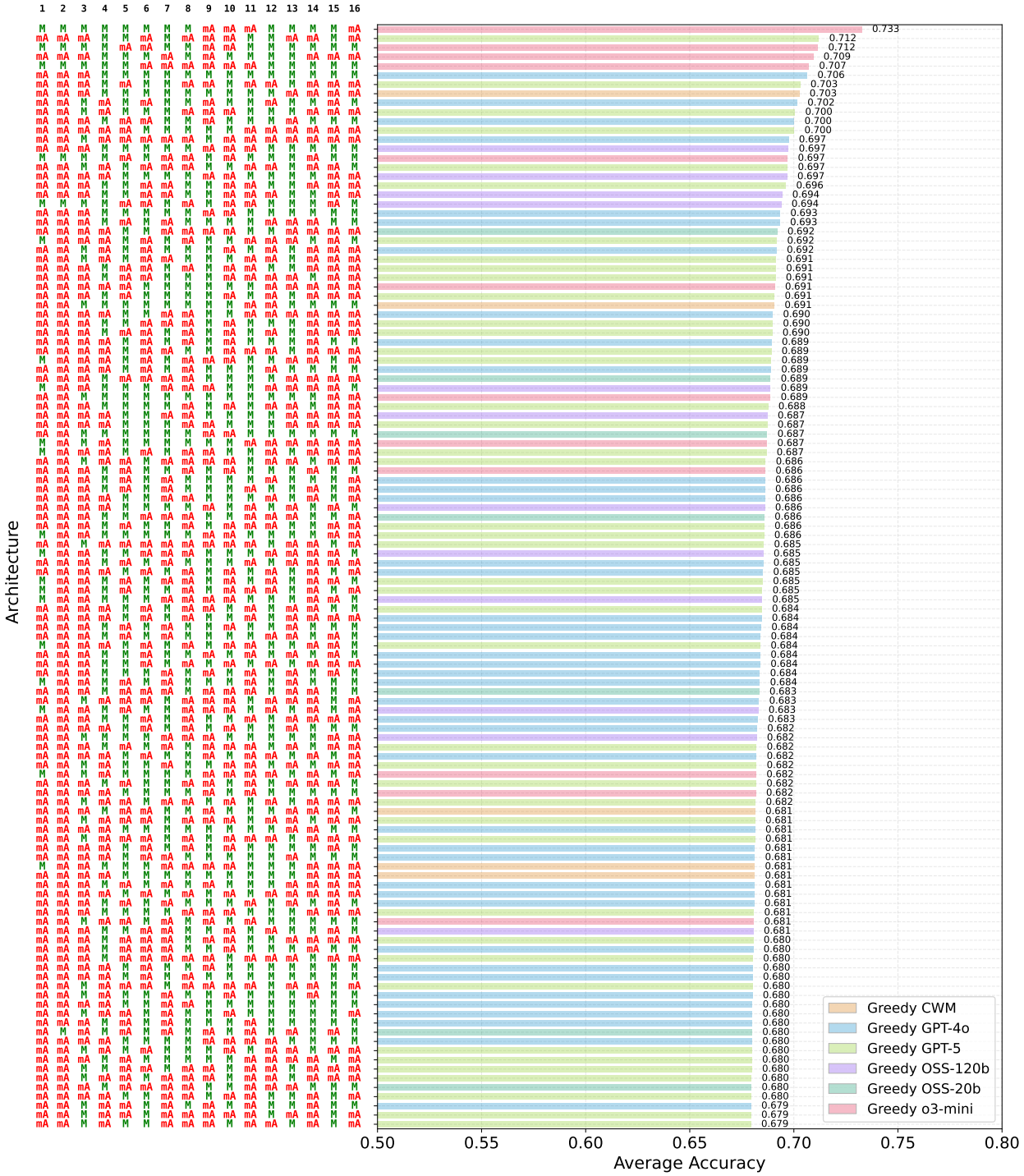


Figure 14 Agent-discovered Architecture Ranking on MAD 2-Primitives (M, mA). Top 120 small scale architectures discovered during greedy search, ranked by test performance. Each row displays the primitive sequence (M in green, mA in red) alongside its accuracy. Bar colors indicate the discovering agent.

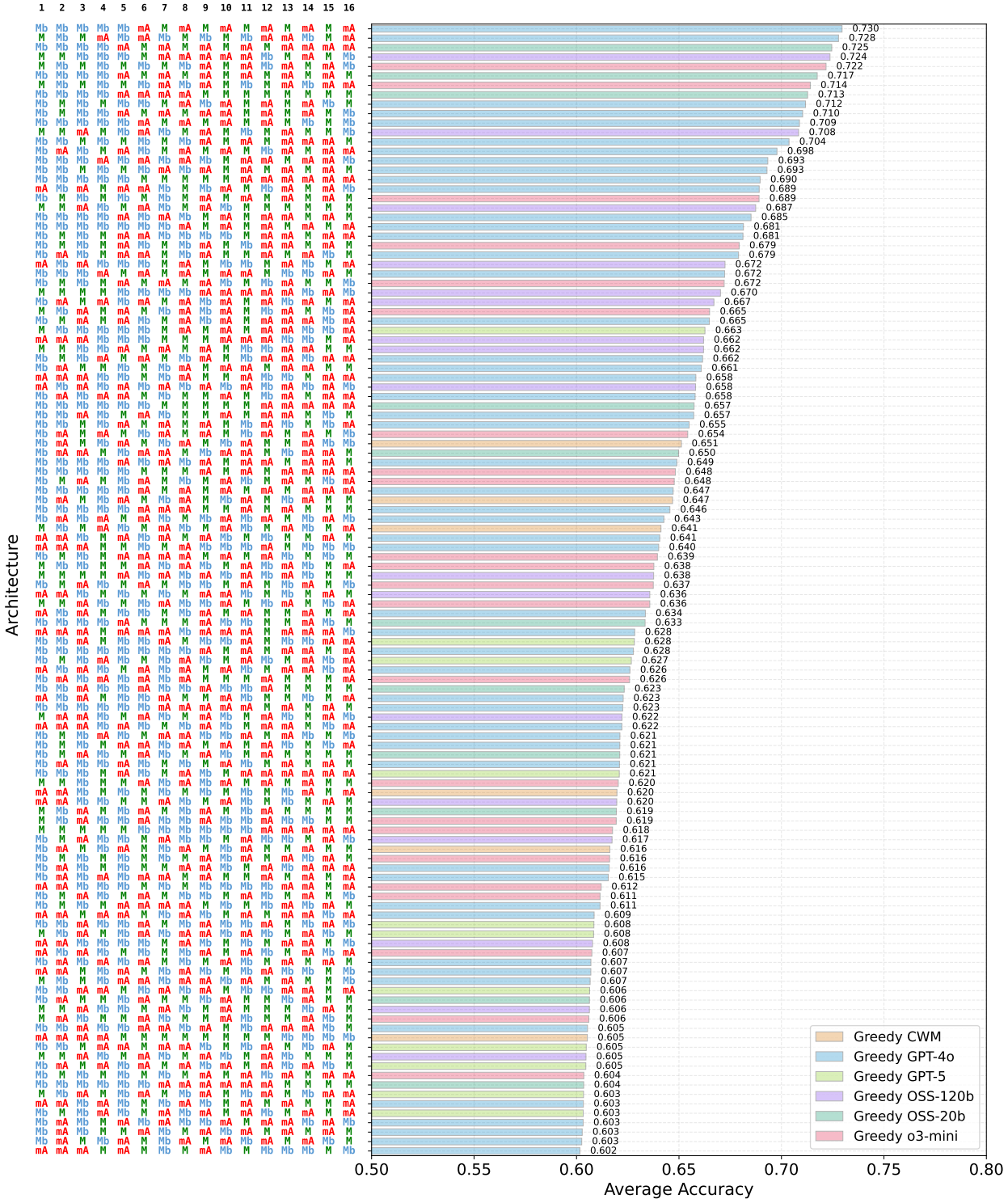


Figure 15 Agent-discovered Architecture Ranking on MAD 3-Primitives (M, mA, Mb). Top 120 small scale architectures discovered during greedy search, ranked by test performance. Each row displays the primitive sequence (M in green, mA in red, Mb in blue) alongside its accuracy. Bar colors indicate the discovering agent.

C Additional Details

Table 6 Environment requirements for AIRA-Compose tasks

Package	Version	Package	Version	Package	Version
torch	2.6.0	einops	–	transformers	–
torchvision	0.21.0	seaborn	–	tokenizers	–
torchaudio	2.6.0	torchmetrics	–	huggingface-hub	–
ninja	–	pytorch-lightning	–	safetensors	–
datasets	4.0.0	opt-einsum	–	filelock	–
scikit-learn	–	ray	–	fsspec	–
multiprocess	–	scipy	–	triton	3.1.0
ax-platform	–	absl-py	–	flash-attn	2.8.3
submitit	–	tiktoken	–	causal-conv1d	1.5.3.post1
blobfile	–	pyyaml	–	mamba-ssm	2.2.6.post3
numpy	–	tqdm	–		
pandas	–	packaging	–		

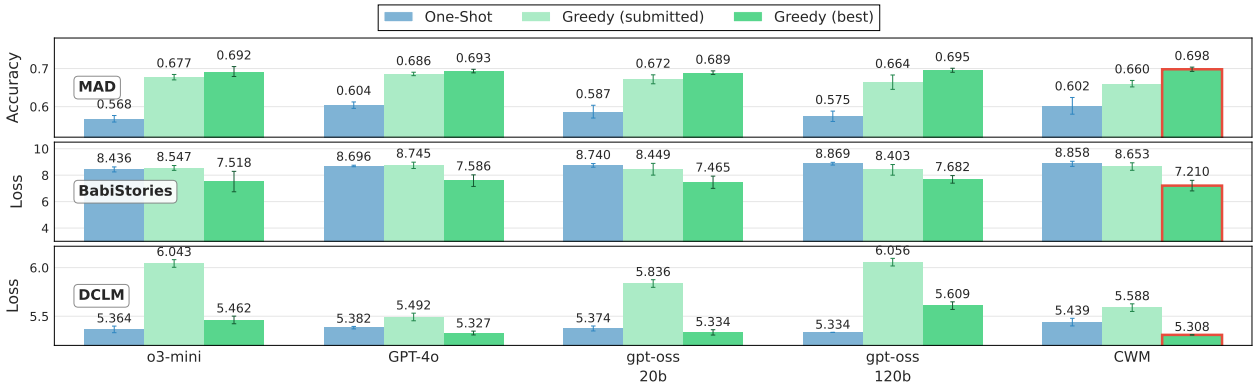


Figure 16 Comparison of LLM agents on the 2-Primitives architecture search task across three datasets. Performance of 11 agents, {o3-mini, GPT-4o, gpt-oss-20b, gpt-oss-120b, and CWM} × {one-shot, greedy} and greedy GPT-5. For greedy agents, we distinguish between *submitted* solutions (selected based on best validation fitness) and *best* solutions (achieving the highest test performance across all iterations). Top panel: MAD dataset (accuracy, higher is better); middle and bottom panels: BabiStories and DCLM datasets (loss, lower is better). The 2-Primitives search space is constrained to two primitives (M, mA). Error bars represent 95% confidence intervals. Best performing agent is highlighted in red.

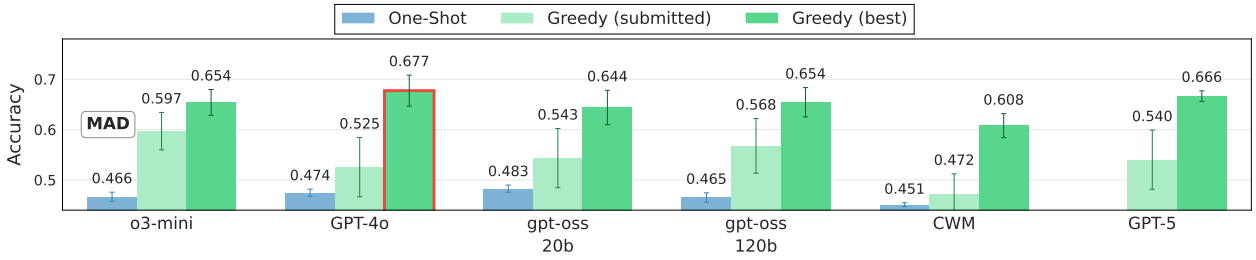


Figure 17 Comparison of LLM agents on the 3-Primitives architecture search task on MAD dataset. Performance of 10 agents, {o3-mini, GPT-4o, gpt-oss-20b, gpt-oss-120b, and CWM} × {one-shot, greedy}. For greedy agents, we distinguish between *submitted* solutions (selected based on best validation fitness) and *best* solutions (achieving the highest test performance across all iterations). The 3-Primitives search space is constrained to three primitives (M, mA, Mb). Error bars represent 95% confidence intervals. Best performing agent is highlighted in red.

Table 7 expands the aggregate DCLM Core Scores reported in Tables 2 and 3 by providing the full per-task breakdown across all 14 evaluation tasks. The Composer-found (2Mb-M-3A) architecture achieves the highest

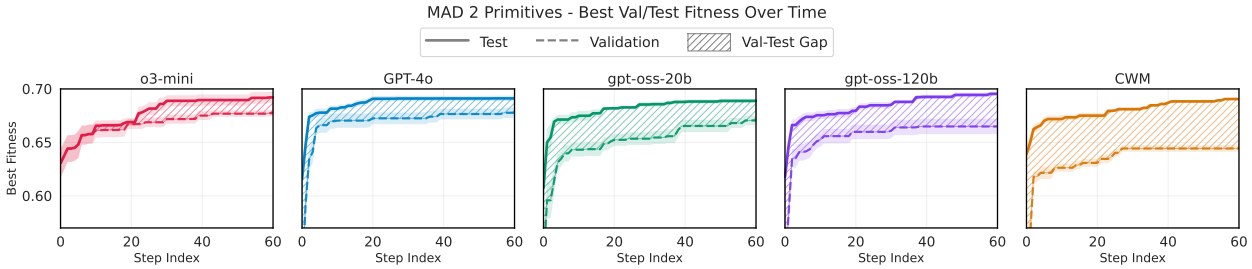


Figure 18 Running Maximum Fitness Trajectories for MAD 3 Primitives (M, mA). Best validation (dashed line) and test (solid line) fitness achieved up to each step index, averaged across runs, for the greedy agents: o3-mini, GPT-4o, gpt-oss-20b, gpt-oss-120b, CWM. Shaded regions indicate 95% confidence intervals, and hatched areas highlight the generalization gap between validation and test fitness.

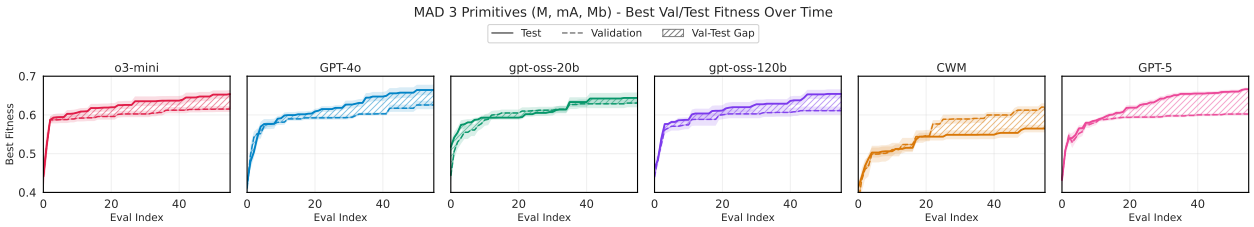


Figure 19 Running Maximum Fitness Trajectories for MAD 3 Primitives (M, mA, Mb). Best validation (dashed line) and test (solid line) fitness achieved up to each step index, averaged across runs, for the greedy agents: o3-mini, GPT-4o, gpt-oss-20b, gpt-oss-120b, CWM and GPT-5. Shaded regions indicate 95% confidence intervals, and hatched areas highlight the generalization gap between validation and test fitness.

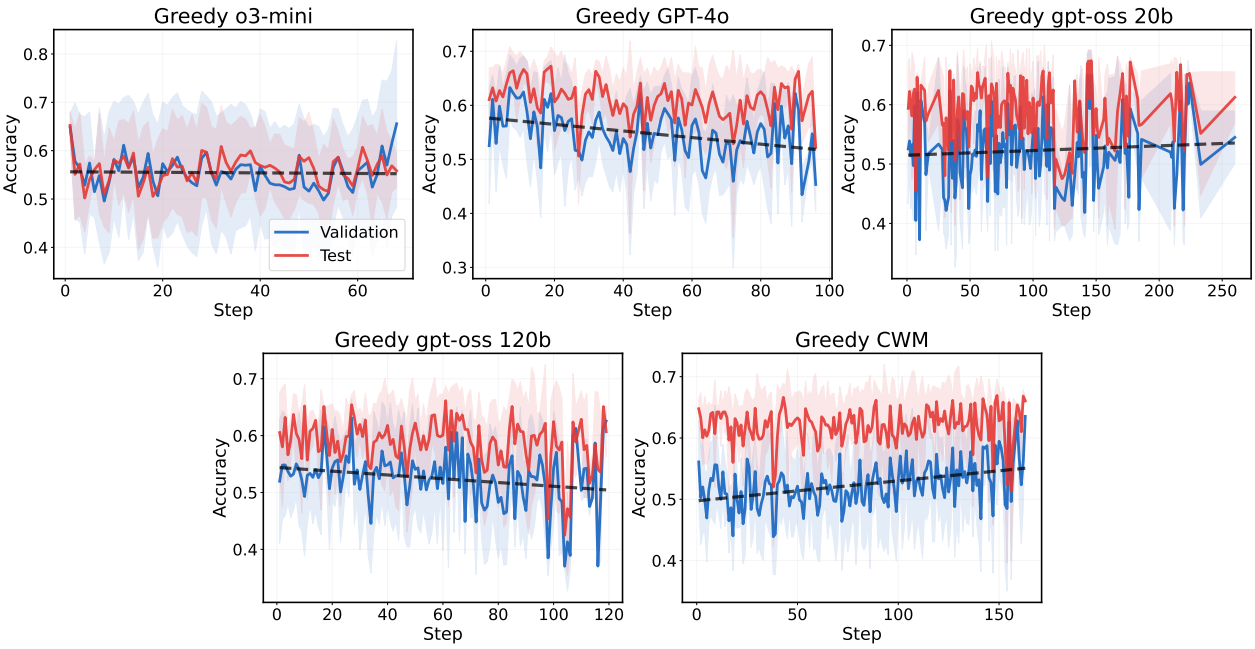


Figure 20 Averaged exploration trajectories for the 2-primitive (M, mA) greedy search on the MAD benchmark. Each panel shows the mean validation (blue) and test (red) accuracy per step across 10 seeds, with ±1 standard deviation shading and validation accuracy linear trend lines. The gap between validation and test accuracy reflects the generalization challenge of agent-designed architectures under the proxy evaluation used for greedy selection.

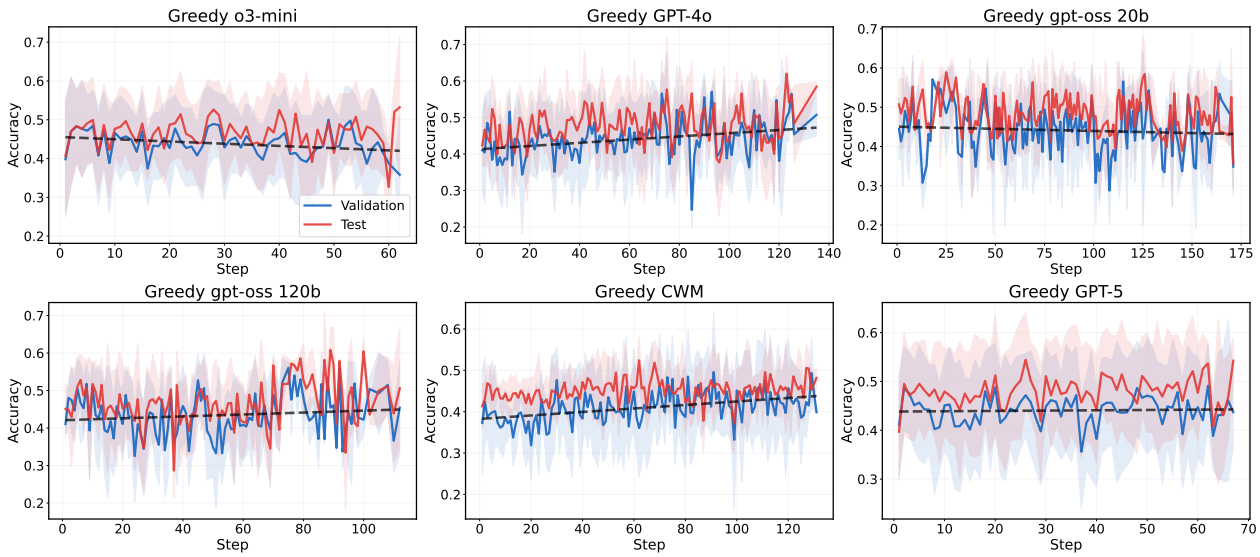


Figure 21 Averaged exploration trajectories for the 3-primitive (M, mA, Mb) greedy search on the MAD benchmark. Each panel shows the mean validation (blue) and test (red) accuracy per step across 10 seeds, with ±1 standard deviation shading and validation accuracy linear trend lines. The gap between validation and test accuracy reflects the generalization challenge of agent-designed architectures under the proxy evaluation used for greedy selection.

Table 7 Per-task DCLM Core Score breakdown at 1B scale (isotoken budget, 37.5B tokens). Metric per task in parentheses. **Bold** = best, underline = second best per column *within each primitive group*. Last column: DCLM Core Score = unweighted average (%).

Model	COPA (Acc.)	CoQA (F1 Score)	LAMBADA (Acc.)	OBQA (Norm. Acc.)	WinoGr. (Acc.)	XWino. (Acc.)	AGIEval (Norm. Acc.)	ARC-C (Norm. Acc.)	ARC-E (Norm. Acc.)	BoolQ (Acc.)	CSQA (Acc.)	HSwag (Norm. Acc.)	PIQA (Norm. Acc.)	SQuADv2 (F1 Score)	**Average**
2-Primitive (M, mA)															
Llama 3.2	78.0	45.4	46.8	33.4	56.4	76.7	20.9	33.1	64.3	37.9	20.1	53.3	73.3	17.1	46.9
Composite (St.)	75.7	43.1	43.0	33.5	57.3	76.7	20.0	34.7	65.1	37.9	**20.6**	54.7	73.1	16.6	46.6
Composite (Str.)	**78.3**	44.6	44.5	34.5	56.2	78.5	20.3	**36.2**	67.8	37.9	20.3	55.7	72.5	14.3	47.3
AIRAformer-A (Str.)	75.7	50.4	47.2	34.6	57.7	**80.4**	**21.0**	35.9	67.9	37.9	19.8	57.3	**73.8**	19.1	48.5
AIRAformer-B (St.)	75.7	50.6	46.2	34.3	57.2	80.1	18.7	35.5	67.7	37.9	19.9	57.4	73.5	19.5	48.1
AIRAformer-C (St.)	77.7	53.2	49.0	**35.3**	58.3	80.2	20.3	35.3	67.2	37.9	18.9	56.6	73.4	**20.2**	48.8
AIRAformer-C (Str.)	75.0	52.1	**49.3**	33.5	56.6	78.8	18.6	34.7	67.2	37.9	**20.6**	57.0	73.1	19.2	48.1
AIRAformer-D (St.)	74.3	**53.9**	**49.3**	33.7	57.7	80.3	19.1	35.5	68.0	37.9	19.0	57.4	73.4	18.5	48.4
AIRAformer-D (Str.)	76.7	52.7	49.2	35.0	**59.0**	79.6	18.3	35.5	**68.3**	37.9	20.5	**58.1**	73.5	**20.2**	**48.9**
3-Primitive (M, mA, Mb)															
Nemotron-H (Approx.)	79.0	49.0	49.1	36.2	56.2	81.3	**23.0**	34.6	66.2	37.9	19.4	57.2	74.3	15.1	48.5
Nemotron-2 (Approx.)	76.0	47.6	48.6	35.8	58.3	81.1	20.9	35.7	66.8	38.0	21.1	57.9	73.0	16.6	48.4
Mamba (Mb+M)	79.0	34.4	45.2	35.6	57.7	80.1	16.1	35.6	64.8	37.9	19.8	55.9	73.6	9.6	46.1
Composer (2Mb-M-3A)	79.0	53.2	51.8	34.6	58.3	81.3	19.6	35.9	67.3	37.9	22.0	57.6	73.5	18.2	**49.3**
AIRAhybrid-A (Str.)	**82.0**	40.6	47.6	35.2	57.2	79.3	19.6	34.7	67.5	37.9	**22.9**	57.2	72.7	9.9	47.5
AIRAhybrid-B (St.)	78.0	50.5	**51.9**	36.4	**59.9**	**82.6**	18.3	36.0	66.2	37.9	19.2	**58.5**	73.6	16.6	49.0
AIRAhybrid-B (Str.)	76.0	49.6	50.5	37.0	58.7	81.8	21.3	**36.8**	66.6	37.9	21.5	58.4	**75.0**	16.1	49.1
AIRAhybrid-C (Str.)	76.0	**53.5**	51.2	35.6	57.9	80.6	22.6	34.6	65.8	37.9	19.1	57.0	73.1	**19.6**	48.9
AIRAhybrid-D (St.)	79.0	49.1	51.4	35.8	58.3	81.8	16.1	35.1	66.8	37.9	19.2	57.2	73.2	16.0	48.4
AIRAhybrid-D (Str.)	74.0	50.8	50.6	37.2	57.9	81.5	17.4	**36.8**	**67.7**	37.9	20.0	57.8	73.8	16.0	48.5
AIRAhybrid-E (St.)	78.0	46.9	51.7	36.2	59.7	82.1	20.9	**36.8**	67.3	37.9	21.1	57.8	72.7	14.3	48.8
AIRAhybrid-E (Str.)	74.0	48.3	51.5	**37.4**	59.0	81.7	**23.0**	34.0	66.8	37.9	19.6	58.0	73.7	16.0	48.6

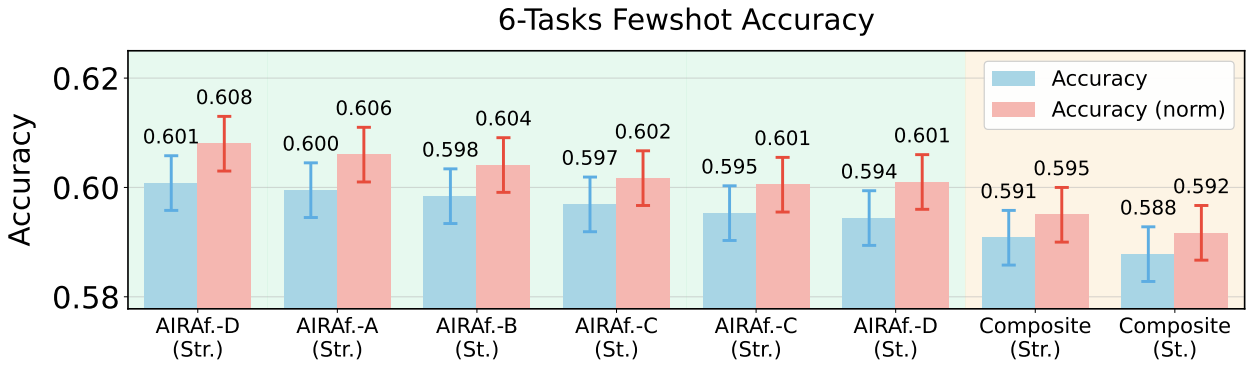


Figure 22 Downstream evaluation of AIRAformer architectures at 1B scale. 0-shot accuracy on 6 commonsense reasoning tasks (ARC-C, ARC-E, HellaSwag, PIQA, SciQ, WinoGrande), showing both raw and normalized accuracy across three seeds.

overall average (49.3%), while the AIRAhybrid models show complementary strengths: AIRAhybrid-B variants achieve the best scores on WinoGrande (59.9), XWinograd (82.6), HellaSwag (58.5), and PIQA (75.0), while AIRAhybrid-E (Stretched) leads on OpenBookQA (37.4) and ties for best on AGIEval (23.0). BoolQ remains essentially at chance level (∼37.9%) across all architectures, indicating that 1B-scale models with our training setup uniformly fail under 10-shot evaluation regardless of architecture. Among the 2-primitive models, the AIRAformer variants consistently outperform the Composite and Llama baselines on CoQA, SQuADv2, and HellaSwag, with AIRAformer-D (Stretched) achieving the highest average (48.9%).

Figures 23–24 present the isoFLOP validation loss across three model sizes (350M, 1B, and 3B parameters) and five FLOP budgets for both the 2-primitive and 3-primitive settings. In the 2-primitive case, balanced architectures consistently outperform attention-heavy ones: AIRAformer-A Stretched, AIRAformer-B Stacked, and the Composite baselines achieve lower validation loss than AIRAformer-C and AIRAformer-D (which have A/M ratios exceeding 2) across all scales and budgets. For example, at the 1B scale with 4×10^{20} FLOPs, AIRAformer-A Stretched achieves a validation loss of 2.7415 compared to 2.7647–2.7732 for the attention-heavy variants.

In the 3-primitive hybrid setting, we observe a similar pattern: architectures that distribute compute more evenly across Mamba, MLP, and attention primitives—such as AIRAhybrid-C Stretched and the Composer (2Mb-M-3A) model—consistently reach the lowest validation losses, while Mamba-only configurations (AIRAhybrid-A) and attention-dominated hybrids underperform.

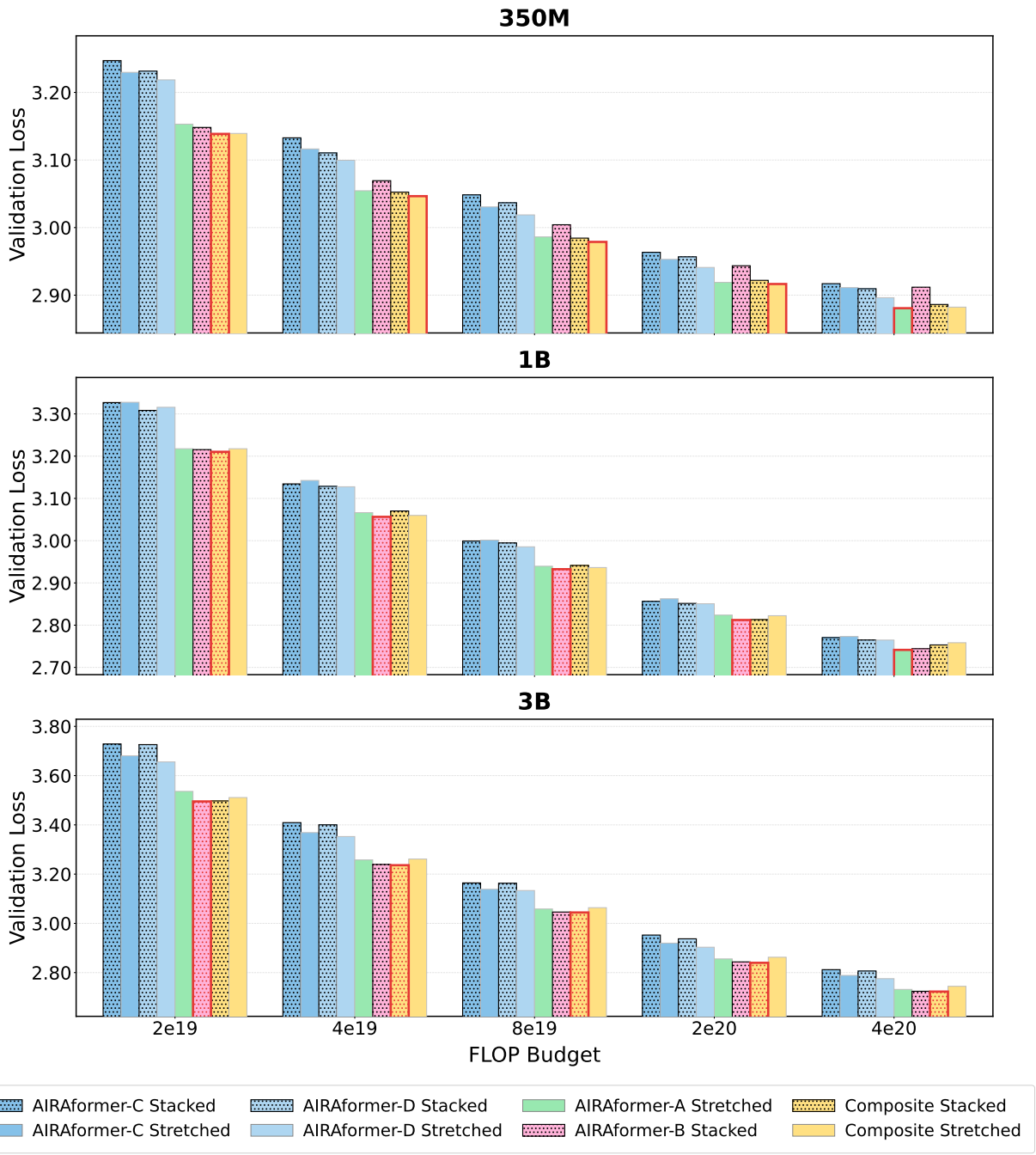


Figure 23 IsoFLOP comparison of validation loss across model sizes. Each subplot shows validation loss for 8 architectures (6 AIRAformer variants and 2 Composite baselines) at a given model size (350M, 1B, 3B), evaluated across 5 FLOP budgets ranging from 2×10^{19} to 4×10^{20}. Bars with red borders indicate the architecture achieving the lowest validation loss at each FLOP budget.

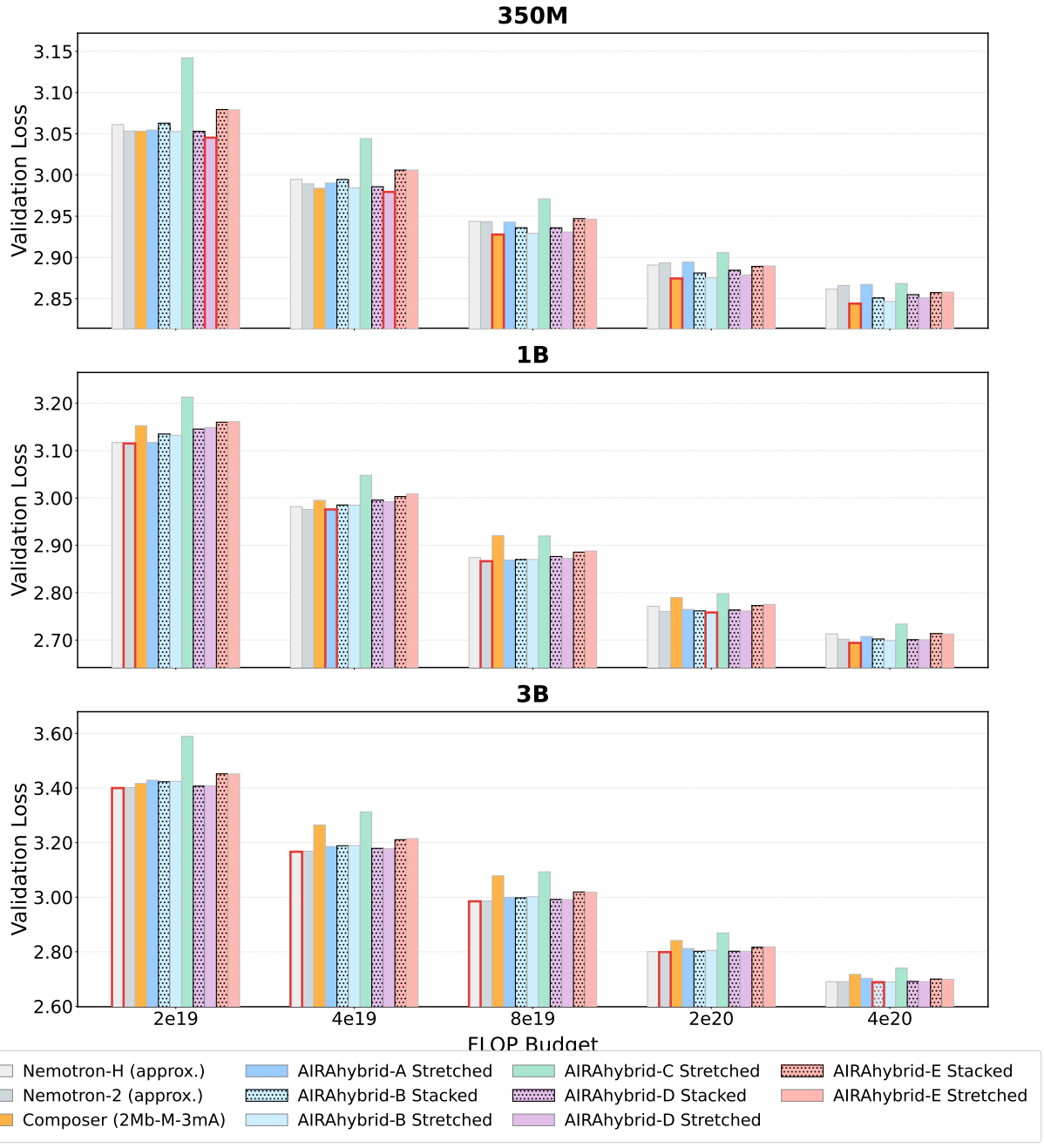


Figure 24 IsoFLOP comparison of validation loss across model sizes, 3-primitive case. Each subplot shows validation loss for 11 architectures (8 AIRAhybrids, 2 baselines and 1 Composer-found baseline) at a given model size (350M, 1B, 3B), evaluated across 5 FLOP budgets ranging from 2×10^{19} to 4×10^{20}. Bars with red borders indicate the architecture achieving the lowest validation loss at each FLOP budget.

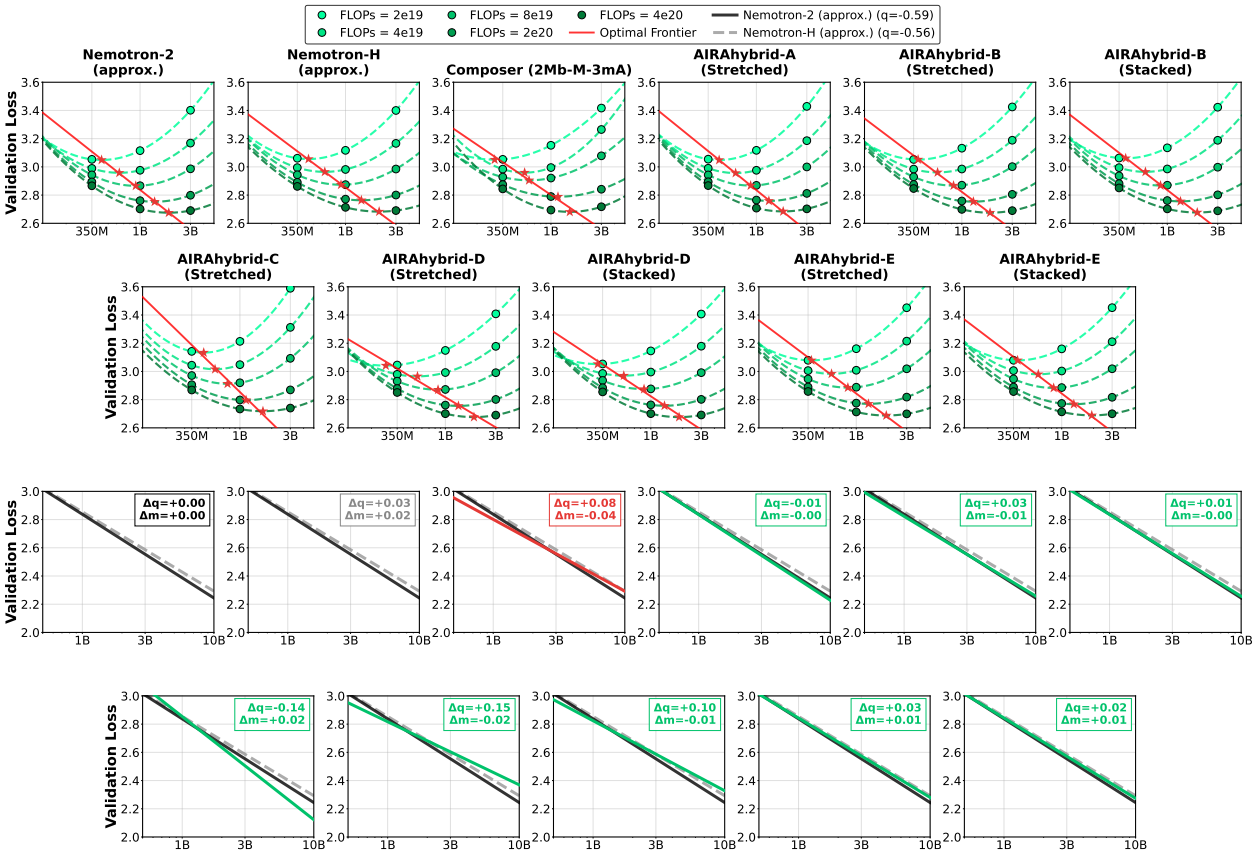


Figure 25 IsoFLOP scaling curves and optimal frontier comparison (M, mA, Mb), full results. Top: Validation loss versus model size for each architecture across 5 FLOPs budgets. **Bottom:** Comparison of each architecture's optimal frontier against the approximated Nemotron-2 and Nemotron-H.

D Aggregation and Scaling Patterns

D.1 Aggregation methods

In the two-primitive space:

- **AIRAformer-A** (base pattern: `(A + M) + (2A + 2M) + 4 × (A + M) + 2M`): Obtained from the top-20 performing architectures on each of the 3 datasets. Within each dataset, the architectures are weighted using a strong exponential decay inversely proportional to their rank. Finally, the 3 datasets are given equal weight in the overall aggregation.
- **AIRAformer-B** (base pattern: `2A + 5 × (A + M) + 4M`): Obtained via N_1 aggregation on the top-400 architectures across all 5 agents on the MAD dataset.
- **AIRAformer-C** (base pattern: `(2A + M) + 3 × (A + M) + (2A + M) + 4A`): Obtained via N_1 aggregation on the top cluster after k-means clustering (using 3 clusters) on architectures derived from 20 seeds search of greedy GPT-5 agent on the MAD dataset.
- **AIRAformer-D** (base pattern: `5 × (2A + M) + A`): Obtained via N_2 aggregation on the top cluster after k-means clustering (using 3 clusters) on architectures derived from 20 seeds search of greedy GPT-5 agent on the MAD dataset.

In the three-primitive space:

- **AIRAhybrid-A** (base pattern: `2Mb + M + 11Mb + 2M`): Obtained via N_0 aggregation on the top cluster after k-means clustering (using 3 clusters) on the architectures found by greedy GPT-5.
- **AIRAhybrid-B** (base pattern: `3 × (2Mb + M + A) + 2Mb + 2M`): Obtained via N_1 aggregation on the top cluster after k-means clustering (using 3 clusters) on the architectures found by greedy GPT-5.
- **AIRAhybrid-C** (base pattern: `2Mb + A + 2 × (Mb + A) + M + 2 × (A + Mb + A + M)`): Obtained via N_2 aggregation on the top cluster after k-means clustering (using 3 clusters) on the architectures found by greedy GPT-5.
- **AIRAhybrid-D** (base pattern: `2Mb + M + 2 × (Mb + M) + A + M + Mb + M + A + M + Mb + M + A`): Obtained via N_1 aggregation on the top cluster after k-means clustering (using 3 clusters) on the architectures found by all 6 agents.
- **AIRAhybrid-E** (base pattern: `5 × (Mb + M + A) + M`): Obtained via N_2 aggregation on the top cluster after k-means clustering (using 3 clusters) on the architectures found by all 6 agents.

D.2 Scaling patterns

We provide a breakdown of the primitives and their implementation at small scale (16-layer proxy) and large scale (350M, 1B, 3B) below. At small scale, all primitives share a model dimension $d = 128$. At large scale, the model dimension d, number of attention heads n_h, head dimension d_h, and hidden dimensions are configured per scale as detailed in the IsoFLOP methodology section below.

- **MLP (M):** A feed-forward network. At small scale, a standard two-layer MLP with ReLU activation and hidden dimension $h_{\text{mlp}} = 258$. At large scale, a SwiGLU variant (Shazeer, 2020) with gated projections: gate and up projections $d \to h_{\text{mlp}}$ and a down projection $h_{\text{mlp}} \to d$, where h_{mlp} is computed from d and a scale-dependent expansion factor (see below).
- **Multi-head Attention (mA):** Standard multi-head causal self-attention (Vaswani et al., 2017). At small scale, we use $n_h = 16$ query heads with head dimension $d_h = 8$ and $n_{\text{kv}} = 16$ key-value heads (i.e., multi-head attention without grouping). At large scale, grouped-query attention (GQA) (Ainslie et al., 2023) is employed with $n_{\text{kv}} = 8$ key-value heads shared across a larger number of query heads, yielding a total KV dimension of $d_{\text{kv}} = n_{\text{kv}} \times d_h$.
- **Mamba (Mb):** A selective state-space model (SSM) based on Mamba-2 (Dao and Gu, 2024), which processes sequences with linear complexity through data-dependent gating and a hardware-efficient

selective scan. At small scale, the SSM uses an expansion factor $e_{\text{ssm}} = 1.25$ (yielding SSM hidden dimension $d_{\text{ssm}} = e_{\text{ssm}} \cdot d = 160$), a state dimension $n_s = 4$, a convolution kernel size $k = 4$, and $n_g = 1$ group. At large scale, the SSM hidden dimension is instead computed as $d_{\text{ssm}} = \texttt{multiple_of}(\lfloor \frac{2}{3} \cdot 3d \rfloor, 256)$, with a larger state dimension n_s and SSM head dimension $d_h^{\text{ssm}} = 64$ (see below for per-scale values).

Table 8 2-primitive architecture configurations across parameter scales. We report the full layer pattern at each scale, total depth (L), parameter counts (with / without embeddings, in billions), and the approximated ratio of Attention to MLP blocks (A:M).

Architecture	Scale	L	Params (B)	A:M	Pattern
Llama 3.2	**350M**	28	0.35 / 0.55	1.0:1	14×(A-M)
	1B	32	0.97 / 1.23	1.0:1	16×(A-M)
	3B	56	2.82 / 3.21	1.0:1	28×(A-M)
Composite (Stacked)	**350M**	24	0.35 / 0.55	0.5:1	4×(2A-4M)
	1B	27	1.00 / 1.26	0.5:1	4×(2A-4M)-A-2M
	3B	51	3.00 / 3.39	0.5:1	8×(2A-4M)-A-2M
Composite (Stretched)	**350M**	26	0.39 / 0.59	0.4:1	3A-8M-3A-5M-2A-5M
	1B	29	1.06 / 1.32	0.5:1	4A-9M-4A-5M-2A-5M
	3B	54	3.17 / 3.56	0.5:1	7A-16M-7A-10M-4A-10M
AIRAformer-A (Stretched)	**350M**	26	0.34 / 0.54	0.8:1	2×(A-M-2A-2M-3×(A-M)-A-3M)
	1B	32	1.05 / 1.31	0.8:1	2×(A-M-2A-2M)-4×(2A-2M)-4M
	3B	56	2.97 / 3.36	0.8:1	2×(A-M-4A-4M-6×(A-M)-2A-6M)
AIRAformer-B (Stacked)	**350M**	23	0.35 / 0.55	0.4:1	(3A-4×(M-A)-5M)-7M
	1B	32	1.05 / 1.31	0.8:1	2×(3A-4×(M-A)-5M)
	3B	53	2.94 / 3.33	0.7:1	3×(3A-4×(M-A)-5M)-5M
AIRAformer-C (Stacked)	**350M**	34	0.34 / 0.54	2.4:1	2×(2×(2A-M)-3×(A-M)-4A)-2A
	1B	48	1.10 / 1.36	2.2:1	3×(2×(2A-M)-3×(A-M)-4A)
	3B	76	2.92 / 3.31	2.4:1	5×(2×(2A-M)-3×(A-M)-4A)-6A
AIRAformer-C (Stretched)	**350M**	34	0.35 / 0.55	3.0:1	4A-2M-2A-2M-2A-2M-2A-8A
	1B	48	1.10 / 1.36	3.0:1	6A-3M-3A-3M-3A-3M-3A-3M-6A-3M-12A
	3B	70	2.92 / 3.31	2.4:1	10A-5M-5A-5M-5A-5M-5A-20A-3A
AIRAformer-D (Stacked)	**350M**	34	0.34 / 0.54	2.4:1	2×(5×(2A-M)-A)-2A
	1B	48	1.10 / 1.36	2.2:1	3×(5×(2A-M)-A)
	3B	76	2.92 / 3.31	2.4:1	5×(5×(2A-M)-A)-6A
AIRAformer-D (Stretched)	**350M**	34	0.35 / 0.55	2.0:1	4A-2M-4A-2M-4A-2M-4A-2M-4A-2M-2A-M
	1B	48	1.10 / 1.36	2.2:1	6A-3M-6A-3M-6A-3M-6A-3M-6A-3M-3A
	3B	70	2.92 / 3.31	2.0:1	10A-5M-10A-5M-10A-5M-10A-5M-10A-5M-5A-3M

D.3 IsoFLOP Budget Methodology

We compute the training floating-point operations (FLOPs) per token per layer for each primitive using the standard convention. Each linear operation of shape $(M, K) \times (K, N)$ costs $2MKN$ FLOPs in the forward pass. This is multiplied by a factor of 3 for training (accounting for one forward and two backward passes), yielding $6 \cdot d_{\text{in}} \cdot d_{\text{out}}$ FLOPs per projection per token.

Model Configurations. All three model scales share a sequence length $s = 8{,}192$, $n_{\text{kv}} = 8$ key-value heads, and a constant batch size of 524,288 tokens per step. The batch size is derived via DP × local_bs × acc_steps × s (e.g., $16 \times 4 \times 1 \times 8{,}192$ at the 1B scale).

For the **2-primitive experiments**:

- **350M:** $d = 1536$, $n_h = 24$, $d_h = 64$, $d_{\text{kv}} = 512$
- **1B:** $d = 2048$, $n_h = 32$, $d_h = 64$, $d_{\text{kv}} = 512$
- **3B:** $d = 3072$, $n_h = 24$, $d_h = 128$, $d_{\text{kv}} = 1024$

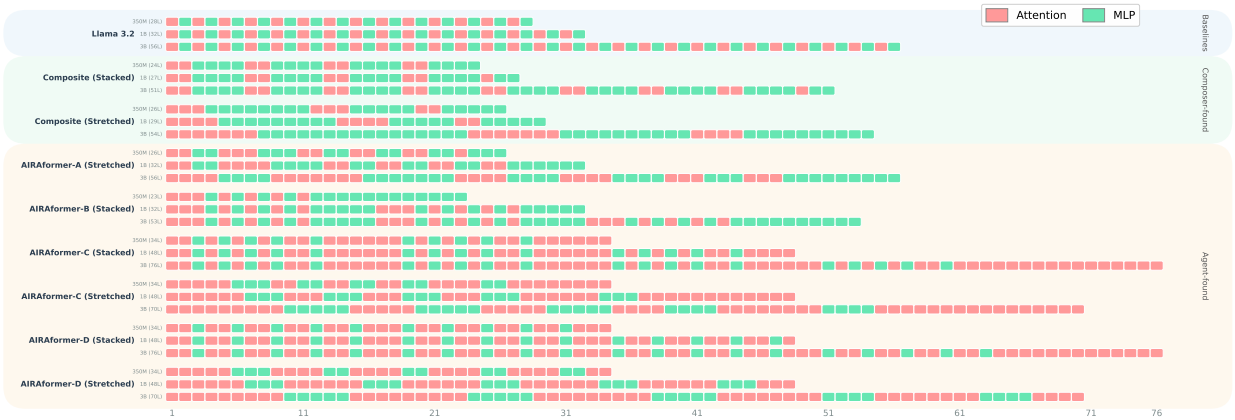


Figure 26 Architecture layer patterns for all discovered and baseline models across three parameter scales (350M, 1B, 3B), 2-primitive case. Each row within an architecture shows the sequence of Attention (red) and MLP (green) layers. Llama 3.2 uses a uniform 1:1 alternating pattern. Composite architectures are MLP-heavy (5A+11M base ratio). AIRAformer-A and AIRAformer-B (7A+9M base ratio) were obtained via custom and N_1 aggregation across multiple models, respectively. AIRAformer-C and AIRAformer-D (11A+5M base ratio) were discovered via N_1 and N_2 aggregation on the top cluster after 3-means clustering on GPT-5 rankings. Stacked variants repeat the base pattern, while Stretched variants proportionally scale each contiguous group of layers.



Figure 27 Architecture layer patterns for all discovered and baseline models across three parameter scales (350M, 1B, 3B), 3-primitive case (M, mA, Mb). Each row within an architecture shows the sequence of Attention (red), MLP (green) and Mamba (blue) layers. We compare our 8 agent-discovered architectures (AIRAhybrid A-E) with 2 baselines, Nemotron-H and Nemotron-2, approximated to the desired scales, and 1 Composer-found architecture.

Table 9 3-primitive (M, mA, Mb) architecture configurations across parameter scales. We report the full layer pattern at each scale, total depth (L), parameter counts (with / without embeddings, in billions), and the number of Attention (A), MLP (M), and Mamba (Mb) blocks.

Architecture	Scale	L	A	M	Mb	Params (B)	Pattern
Nemotron-H (Approx.)	**350M**	18	2	8	8	0.36 / 0.55	Mb-M-A-5×(M-Mb)-A-M-Mb-M-Mb
	1B	28	3	13	12	1.00 / 1.26	Mb-M-2×(A-4×(M-Mb)-M)-A-M-Mb-M
	3B	37	4	17	16	2.96 / 3.36	Mb-M-2×(A-4×(M-Mb)-M)-A-2×(M-Mb)-M-A-M-3×(Mb-M)
Nemotron-2 (Approx.)	**350M**	17	2	7	8	0.33 / 0.52	3×(Mb-M)-Mb-A-4×(M-Mb)-A-M-Mb
	1B	29	3	13	13	1.02 / 1.28	2×(3×(Mb-M)-Mb-A-M)-Mb-M-Mb-A-5×(M-Mb)-M
	3B	37	4	16	17	2.91 / 3.30	2×(3×(Mb-M)-Mb-A-M)-Mb-M-Mb-A-7×(M-Mb)-A-M-Mb
Mamba (Mb+M)	**350M**	17	0	8	9	0.36 / 0.56	8×(Mb-M)-Mb
	1B	26	0	13	13	0.99 / 1.25	13×(Mb-M)
	3B	35	0	17	18	2.98 / 3.37	17×(Mb-M)-Mb
Composer (2Mb-M-3A)	**350M**	26	12	4	10	0.34 / 0.53	4×(2Mb-M-3A)-2Mb
	1B	45	21	8	16	1.04 / 1.30	7×(2Mb-M-3A)-2Mb-M
	3B	57	27	10	20	2.98 / 3.38	9×(2Mb-M-3A)-2Mb-M
AIRAhybrid-A (Stretched)	**350M**	20	0	3	17	0.33 / 0.53	2Mb-M-14Mb-2M-Mb
	1B	32	0	6	26	0.97 / 1.24	4Mb-2M-22Mb-4M
	3B	44	0	8	36	3.01 / 3.41	6Mb-3M-30Mb-5M
AIRAhybrid-B (Stretched)	**350M**	20	4	6	10	0.34 / 0.54	3×(2Mb-M-A)-3×(Mb-M)-A-Mb
	1B	32	6	10	16	0.98 / 1.24	3×(4Mb-2M-2A)-2×(2Mb-2M)
	3B	43	9	13	21	2.93 / 3.32	3×(5Mb-3M-3A)-3Mb-3M-3Mb-M
AIRAhybrid-B (Stacked)	**350M**	20	5	7	8	0.35 / 0.54	3×(Mb-M-A)-2×(3×(Mb-M)-A)-Mb
	1B	32	6	10	16	0.98 / 1.24	2×(3×(2Mb-M-A)-2×(Mb-M))
	3B	43	7	12	24	2.94 / 3.33	2×(3×(3Mb-M-A)-2×(Mb-M))-Mb-M-A-Mb-M
AIRAhybrid-C (Stretched)	**350M**	29	16	3	10	0.33 / 0.53	4Mb-2A-Mb-3×(2A-Mb-2A-M)-Mb-A-Mb-A
	1B	40	21	6	13	0.86 / 1.12	5Mb-3A-2Mb-3×(3A-2Mb-3A-2M)
	3B	61	35	6	20	2.73 / 3.12	8Mb-5A-3Mb-2×(5A-3Mb-5A-3M)-5A-3Mb-5A
AIRAhybrid-D (Stretched)	**350M**	18	3	8	7	0.35 / 0.54	2Mb-M-Mb-3×(M-Mb-M-A)-Mb-M
	1B	32	6	14	12	1.08 / 1.34	4Mb-2M-2Mb-3×(2M-2Mb-2M-2A)
	3B	39	6	17	16	3.01 / 3.41	5Mb-2M-2Mb-3×(2M-2Mb-2M-2A)-3×(Mb-M)
AIRAhybrid-D (Stacked)	**350M**	18	3	8	7	0.35 / 0.54	Mb-M-Mb-3×(M-Mb-M-A)-Mb-M-Mb
	1B	32	6	14	12	1.08 / 1.34	2×(2Mb-3×(M-Mb-M-A))
	3B	39	7	17	15	2.98 / 3.37	2×(2Mb-3×(M-Mb-M-A))-3×(Mb-M)-A
AIRAhybrid-E (Stretched)	**350M**	20	6	7	7	0.34 / 0.54	5×(Mb-M-A)-M-Mb-M-A-Mb
	1B	32	10	12	10	0.97 / 1.23	5×(2Mb-2M-2A)-2M
	3B	44	14	15	15	2.93 / 3.32	4×(3Mb-3M-3A)-3Mb-3M-2A
AIRAhybrid-E (Stacked)	**350M**	20	6	7	7	0.34 / 0.54	5×(Mb-M-A)-M-Mb-M-A-Mb
	1B	32	10	12	10	0.97 / 1.23	2×(5×(Mb-M-A)-M)
	3B	44	14	16	14	2.98 / 3.38	2×(5×(Mb-M-A)-M)-4×(Mb-M-A)

For the **3-primitive hybrid experiments**:

- **350M:** $d = 1536$, $n_h = 24$, $d_h = 64$, $d_{\text{kv}} = 512$
- **1B:** $d = 2048$, $n_h = 32$, $d_h = 64$, $d_{\text{kv}} = 512$
- **3B:** $d = 3072$, $n_h = 32$, $d_h = 96$, $d_{\text{kv}} = 768$

For the **2-primitive (Attention + MLP)** experiments, the SwiGLU MLP hidden dimension h is computed as $h = \texttt{multiple_of}(\lfloor \frac{2}{3} \cdot 4d \cdot f \rfloor, 1024)$. We use an expansion factor of $f = 1.0$ at the 350M and 3B scales, and $f = 1.4$ at the 1B scale. This yields $h = 4{,}096$ (350M), $h = 8{,}192$ (1B), and $h = 8{,}192$ (3B).

For the **3-primitive (Attention + MLP + Mamba)** experiments, the expansion factor is fixed at $f = 1.4$ across all scales, resulting in $h = 6{,}144$ (350M), $h = 8{,}192$ (1B), and $h = 12{,}288$ (3B). The Mamba2 SSM additionally utilizes a state dimension $n_s = 128$ (350M and 1B) or $n_s = 256$ (3B), a convolution kernel size $k = 4$, $n_g = 1$ group, and an SSM head dimension $d_h^{\text{ssm}} = 64$. The SSM hidden dimension is defined as $d_{\text{ssm}} = \texttt{multiple_of}(\lfloor \frac{2}{3} \cdot 3d \rfloor, 256)$, yielding $d_{\text{ssm}} = 3{,}072$ (350M), $d_{\text{ssm}} = 4{,}096$ (1B), and $d_{\text{ssm}} = 6{,}144$ (3B). Consequently, the number of SSM heads is $n_{\text{ssm}} = d_{\text{ssm}}/d_h^{\text{ssm}}$, which equates to 48, 64, and 96 heads, respectively.

Embeddings are tied with vocabulary size 128,256.

Per-Primitive FLOPs per Token (Training). For **Grouped-Query Attention**, the computational cost comprises four linear projections—Q ($d \to d$), K ($d \to d_{\text{kv}}$), V ($d \to d_{\text{kv}}$), and O ($d \to d$)—alongside the quadratic attention computation ($QK^\top$ and softmax $\cdot\, V$):

$$F_{\text{Attn}} = 6(2d^2 + 2d \cdot d_{\text{kv}}) + 12sd \tag{4}$$

For the **SwiGLU MLP**, the three projections (gate and up: $d \to h$; down: $h \to d$) result in:

$$F_{\text{MLP}} = 6 \cdot 3 \cdot d \cdot h = 18dh \tag{5}$$

For the **Mamba2 SSM**, the cost decomposes into four components: (i) an input projection $d \to d_{\text{in_proj}}$ where $d_{\text{in_proj}} = 2d_{\text{ssm}} + 2n_g n_s + n_{\text{ssm}}$; (ii) a depthwise `conv1d` over $d_{\text{conv}} = d_{\text{ssm}} + 2n_g n_s$ channels with kernel size k; (iii) the selective scan, costing $2 \cdot n_{\text{ssm}} \cdot d_h^{\text{ssm}} \cdot n_s$ per token; and (iv) an output projection $d_{\text{ssm}} \to d$. This yields the following total cost:

$$F_{\text{SSM}} = 6(d \cdot d_{\text{in_proj}} + d_{\text{conv}} \cdot k + 2n_{\text{ssm}} d_h^{\text{ssm}} n_s + d_{\text{ssm}} \cdot d) \tag{6}$$

Training Steps from FLOP Budget. Given an architecture with L_{Attn} attention layers, L_{MLP} MLP layers, and L_{SSM} Mamba layers layers, the total FLOPs per training step are:

$$C_{\text{step}} = (L_{\text{Attn}} \cdot F_{\text{Attn}} + L_{\text{MLP}} \cdot F_{\text{MLP}} + L_{\text{SSM}} \cdot F_{\text{SSM}}) \times B \tag{7}$$

where $B = 524{,}288$ is the constant batch size in tokens.

The number of training steps T for a target FLOP budget $\mathcal{F}$ is calculated as $T = \lfloor \mathcal{F}/C_{\text{step}} \rfloor$.

For example, at the 1B scale, the Composite (Stacked) architecture ($L_{\text{Attn}} = 10$, $L_{\text{MLP}} = 19$) yields:

$$C_{\text{step}} = (10 \cdot 263{,}192{,}576 + 19 \cdot 301{,}989{,}888) \times 524{,}288 \approx 4.39 \times 10^{15} \text{ FLOPs/step} \tag{8}$$

This gives $T = \lfloor 2 \times 10^{19}/(4.39 \times 10^{15}) \rfloor = 4{,}552$ steps for a budget of 2×10^{19} FLOPs (matching Table 10).

Similarly, for the 1B hybrid AIRAhybrid-B Stretched model ($L_{\text{SSM}} = 16$, $L_{\text{MLP}} = 10$, $L_{\text{Attn}} = 6$):

$$C_{\text{step}} = (6 \cdot 263{,}192{,}576 + 10 \cdot 301{,}989{,}888 + 16 \cdot 161{,}333{,}696) \times 524{,}288 \approx 3.77 \times 10^{15} \text{ FLOPs/step} \tag{9}$$

This yields $T = \lfloor 2 \times 10^{19}/(3.77 \times 10^{15}) \rfloor = 5{,}308$ steps at a budget of 2×10^{19} FLOPs (matching Table 11).

The full set of training steps for all architectures and budgets is reported in Tables 10–11.

Table 10 Training steps required to reach target isoFLOP budgets. We include the count of Attention (A) and MLP (M) layers to contextualize the varying computational intensity per step. Calculations are based on a constant batch size of 524,288 tokens per step. Model base configurations: 350M ($d = 1536$, $h = 4096$), 1B ($d = 2048$, $h = 8192$), 3B ($d = 3072$, $h = 8192$).

				Target FLOP Budget				
Scale	**Architecture**	**A**	**M**	**2e19**	**4e19**	**8e19**	**2e20**	**4e20**
350M	Composite (Stacked)	8	16	11,484	22,967	45,934	114,835	229,670
	Composite (Stretched)	8	18	10,751	21,501	43,002	107,505	215,011
	AIRAformer-A (Stretched)	12	14	9,907	19,815	39,629	99,073	198,147
	AIRAformer-B (Stacked)	7	16	12,175	24,351	48,701	121,753	243,506
	AIRAformer-C (Stacked)	24	10	6,737	13,474	26,948	67,370	134,740
	AIRAformer-C (Stretched)	23	11	6,828	13,656	27,312	68,280	136,561
	AIRAformer-D (Stacked)	24	10	6,737	13,474	26,948	67,370	134,740
	AIRAformer-D (Stretched)	23	11	6,828	13,656	27,312	68,280	136,561
1B	Composite (Stacked)	10	19	4,552	9,104	18,208	45,520	91,041
	Composite (Stretched)	10	19	4,552	9,104	18,208	45,520	91,041
	AIRAformer-A (Stretched)	14	18	4,176	8,352	16,703	41,758	83,517
	AIRAformer-B (Stacked)	14	18	4,176	8,352	16,703	41,758	83,517
	AIRAformer-C (Stacked)	33	15	2,879	5,758	11,516	28,791	57,581
	AIRAformer-C (Stretched)	33	15	2,879	5,758	11,516	28,791	57,581
	AIRAformer-D (Stacked)	33	15	2,879	5,758	11,516	28,791	57,581
	AIRAformer-D (Stretched)	33	15	2,879	5,758	11,516	28,791	57,581
3B	Composite (Stacked)	17	34	1,651	3,302	6,605	16,512	33,024
	Composite (Stretched)	18	36	1,559	3,119	6,238	15,595	31,190
	AIRAformer-A (Stretched)	25	31	1,504	3,008	6,015	15,038	30,076
	AIRAformer-B (Stacked)	21	32	1,589	3,178	6,356	15,889	31,778
	AIRAformer-C (Stacked)	56	20	1,108	2,216	4,432	11,081	22,161
	AIRAformer-C (Stretched)	47	23	1,203	2,406	4,812	12,030	24,061
	AIRAformer-D (Stacked)	56	20	1,108	2,216	4,432	11,081	22,161
	AIRAformer-D (Stretched)	47	23	1,203	2,406	4,812	12,030	24,061

Table 11 Training steps required to reach target isoFLOP budgets across 11 architectures and 3 parameter scales. We report the count of Mamba (Mb), MLP (M), and Attention (A) layers to contextualize computational intensity. Calculations assume a constant batch size of 524,288 tokens per step.

					Target FLOP Budget				
Scale	**Architecture**	**Mb**	**M**	**A**	**2e19**	**4e19**	**8e19**	**2e20**	**4e20**
350M	Nemotron-H (Approx.)	8	8	2	15,402	30,804	61,608	154,022	308,044
	Nemotron-2 (Approx.)	8	8	1	16,672	33,345	66,691	166,727	333,455
	Composer (2Mb-M-3A)	10	4	12	9,857	19,715	39,430	98,577	197,154
	AIRAhybrid-A (Stretched)	16	4	0	17,660	35,320	70,641	176,603	353,207
	AIRAhybrid-B (Stretched)	10	6	4	14,130	28,261	56,523	141,309	282,618
	AIRAhybrid-B (Stacked)	10	6	4	14,130	28,261	56,523	141,309	282,618
	AIRAhybrid-C (Stretched)	9	4	16	8,416	16,833	33,667	84,168	168,337
	AIRAhybrid-D (Stretched)	7	8	3	14,826	29,652	59,305	148,262	296,525
	AIRAhybrid-D (Stacked)	7	8	3	14,826	29,652	59,305	148,262	296,525
	AIRAhybrid-E (Stretched)	6	8	6	12,521	25,042	50,084	125,210	250,421
	AIRAhybrid-E (Stacked)	6	8	6	12,521	25,042	50,084	125,210	250,421
1B	Nemotron-H (Approx.)	12	13	3	5,732	11,465	22,930	57,325	114,650
	Nemotron-2 (Approx.)	13	13	3	5,596	11,193	22,387	55,968	111,937
	Composer (M-2Mb-2A-M)	10	12	10	4,841	9,682	19,365	48,412	96,825
	AIRAhybrid-A (Stretched)	26	6	0	6,351	12,702	25,404	63,511	127,022
	AIRAhybrid-B (Stretched)	16	10	6	5,308	10,616	21,232	53,081	106,162
	AIRAhybrid-B (Stacked)	16	10	6	5,308	10,616	21,232	53,081	106,162
	AIRAhybrid-C (Stretched)	13	6	21	4,033	8,066	16,132	40,332	80,664
	AIRAhybrid-D (Stretched)	12	14	6	4,922	9,845	19,690	49,227	98,454
	AIRAhybrid-D (Stacked)	12	14	6	4,922	9,845	19,690	49,227	98,454
	AIRAhybrid-E (Stretched)	10	12	10	4,841	9,682	19,365	48,412	96,825
	AIRAhybrid-E (Stacked)	10	12	10	4,841	9,682	19,365	48,412	96,825
3B	Nemotron-H (Approx.)	16	17	4	1,978	3,956	7,913	19,782	39,565
	Nemotron-2 (Approx.)	17	17	3	1,986	3,973	7,947	19,868	39,737
	Composer (2Mb-M-3A)	20	10	27	1,443	2,887	5,774	14,435	28,871
	AIRAhybrid-A (Stretched)	36	8	0	2,033	4,067	8,135	20,339	40,679
	AIRAhybrid-B (Stretched)	22	13	8	1,852	3,704	7,408	18,520	37,041
	AIRAhybrid-B (Stacked)	22	13	8	1,852	3,704	7,408	18,520	37,041
	AIRAhybrid-C (Stretched)	20	9	32	1,361	2,723	5,447	13,618	27,237
	AIRAhybrid-D (Stretched)	15	17	7	1,881	3,763	7,527	18,817	37,635
	AIRAhybrid-D (Stacked)	15	17	7	1,881	3,763	7,527	18,817	37,635
	AIRAhybrid-E (Stretched)	14	16	14	1,703	3,407	6,814	17,035	34,070
	AIRAhybrid-E (Stacked)	14	16	14	1,703	3,407	6,814	17,035	34,070

E AIRA-Design: LRA - Task Details

Table 12 Tunable hyperparameters in the 3 *Configurable* tasks and their default values employed in their *Non-Configurable* counterparts.

Variable in `model.py`	Config Key	Text	ListOps	Retrieval	Description
SEQ_LEN / MAX_LENGTH	max_length	1000	2000	4000	Maximum seq. len.
LR / LEARNING_RATE	learning_rate	0.0001	0.0001	0.0001	Learning rate
BATCH_SIZE	batch_size	16	16	8	Batch size
NUM_TRAIN_STEPS / TRAIN_STEPS	num_train_steps	20000	5000	5000	Training steps
NUM_LAYERS	num_layers	6	4	4	Transformer layers
NUM_HEADS	num_heads	8	8	4	Attention heads
EMB_DIM / EMBED_DIM	emb_dim	512	512	128	Embedding dim.
QKV_DIM	qkv_dim	512	512	128	QKV dimension
MLP_DIM	mlp_dim	2048	1024	512	MLP hidden dim.
WEIGHT_DECAY	weight_decay	0.01	0.01	0.01	Weight decay
WARMUP_STEPS / WARMUP	warmup	500	1000	8000	LR warmup steps
EVAL_FREQUENCY / EVAL_FREQ	eval_frequency	50	50	50	Eval frequency
RANDOM_SEED / SEED	random_seed	0	0	0	Random seed

Agents conducted extensive hyperparameter tuning across the 3 *Configurable* version of the tasks, generating a total of 7,537 valid evaluation steps (3,189 for ListOps, 2,642 for Text, and 1,706 for Retrieval) across all greedy agents. We report a statistics of the attempted tuning in Table 13.

Table 13 Hyperparameter exploration across the three configurable LRA tasks. We report the range of values explored and the count of unique values evaluated by the agents for each configuration parameter.

	ListOps		Text		Retrieval	
Parameter	**Range Explored**	**Unique**	**Range Explored**	**Unique**	**Range Explored**	**Unique**
SEQ_LEN	200 – 4096	8	32 – 4096	18	64 – 8192	9
LR	5e-05 – 4e-03	18	1e-05 – 0.01	21	3e-05 – 5e-03	18
BATCH_SIZE	2 – 128	10	1 – 64	9	1 – 32	6
NUM_TRAIN_STEPS	10 – 20000	32	2 – 30000	34	5 – 15000	27
NUM_LAYERS	2 – 12	9	1 – 12	8	2 – 8	6
NUM_HEADS	1 – 16	7	1 – 8	5	1 – 16	6
EMB_DIM	32 – 512	12	8 – 512	10	64 – 512	7
QKV_DIM	32 – 512	11	8 – 512	11	32 – 512	8
MLP_DIM	128 – 2048	11	16 – 2048	13	64 – 2048	9
WEIGHT_DECAY	0.01 – 0.1	6	0.01 – 0.1	5	1e-04 – 0.05	5
WARMUP_STEPS	50 – 2000	18	5 – 4000	23	5 – 3000	22
EVAL_FREQUENCY	5 – 1000	9	50 – 4000	10	10 – 1000	12

Table 14 Breakdown of the best generated solutions for the 6 LRA AIRA-Design tasks, in their Configurable and Non-Configurable setups, detailing the generating agent, achieved score, and a brief summary of the architecture. For the Non-Configurable setup, we report the configs that deviate from default ones.

Task	Setup	Score	Agent	Architecture	Summary
ListOps (SOTA: 0.6379)	Non-Config.	0.4415	**Gemini 3.1 Pro**	Bi-SVR	*Bidirectional Selective Vector Recurrence:* Projects input to (r, k, v, a_f, a_b) with SiLU activations; computes element-wise $kv = k \odot v$ and data-dependent decays $\sigma(a + 2)$. Forward and backward states are accumulated via `lax.associative_scan` with the recurrence $(g_1 g_2,\ x_1 g_2 + x_2)$. The two directions are summed, GroupNorm-ed, and gated by receptance r. A depthwise 1D conv provides local mixing before projection.
	Config.	0.5050	**Opus 4.6**	DWConv + BiLinAttn `NUM_LAYERS:6, EMB_DIM:192, MLP_DIM:384, LR:1e-3, BATCH:64`	Three sub-layers per block: (1) a large-kernel depthwise conv gated by a learned sigmoid, (2) global non-causal linear attention with an $\text{ELU}(x) + 1$ feature map computing $\phi(Q)(\phi(K)^\top V)$, and (3) a SiLU-gated FFN $(\text{SiLU}(\text{Dense}(x)) \odot \text{Dense}(x))$. Learned positional embeddings; MEAN pooling.
Text (SOTA: 0.9053)	Non-Config.	0.8400	**Gemini 3 Pro**	XCTA + SwiGLU	*Cross-Covariance Text Attention:* Computes a $d \times d$ cross-covariance matrix $C = K^\top Q$ (after L2-normalizing Q, K over the sequence axis) instead of the $L \times L$ token attention matrix, achieving $O(L \cdot d^2)$ complexity. A learned per-head temperature τ scales C before softmax. Uses convolutional positional encoding (depthwise 1D conv, kernel 5) and a SwiGLU FFN: SiLU(gate) $\odot$ val from a single $2\times$ projection.
	Config.	0.8792	**Gemini 3 Pro**	Bi-SSM `NUM_LAYERS:4, EMB_DIM:128, MLP_DIM:256, LR:5e-4, BATCH:16`	*Bidirectional State Space Model:* Each direction runs a `ParallelLinearRNN` via associative scan: a depthwise 1D conv (kernel 3) followed by SiLU feeds into input-dependent forget $(\sigma(\cdot + 1))$ and input gates. Forward and backward hidden states are concatenated and modulated by a SiLU gate. No explicit positional embeddings—position is implicitly captured by the recurrent structure.
Retrieval (SOTA: 0.8176)	Non-Config.	0.7361	**Opus 4.5**	Linear Attn + Bilinear MLP	*Linear Attention Dual Encoder:* Replaces softmax attention with a $\phi(x) = \text{ELU}(x) + 1$ linear approximation: $\phi(Q)(\phi(K)^\top V)/(\phi(Q) \cdot \mathbf{1}^\top \phi(K))$. The MLP uses a bilinear GELU form: $\text{GELU}(W_g x) \odot \text{GELU}(W_v x)$. A shared-weight dual encoder processes both documents; pooled representations are combined via NLI interaction $[\mathbf{h}_1; \mathbf{h}_2; \mathbf{h}_1 \odot \mathbf{h}_2; \mathbf{h}_1 - \mathbf{h}_2]$.
	Config.	0.7943	**Opus 4.6**	Hybrid Local-Global Attn `SEQ_LEN:4000, NUM_LAYERS:4, EMB_DIM:192, MLP_DIM:512, LR:3e-4, BATCH:8`	*Hybrid Local-Global Attention:* Combines non-overlapping chunked softmax attention (W=256 tokens) with global linear attention (ELU+1). A per-position sigmoid gate $\sigma(Wx)$ blends: $\sigma \cdot \mathbf{y}_{\text{local}} + (1 - \sigma) \cdot \mathbf{y}_{\text{global}}$. Memory is $O(nW + nd^2)$. The encoder returns both hidden states and padding masks for masked mean pooling in the dual-encoder NLI head.

F AIRA-Design: Autoresearch - Task Details

Table 15 Python requirements comparison between the Autoresearch implementation in Karpathy (2026) and the two AIRA-DOJO tasks, both with and without literature.

Autoresearch (Karpathy, 2026)	AutoregressiveLanguageModellingAutoresearchBPB
`torch==2.9.1 (cu128)`	`torch==2.6.0 (cu126)`
`numpy>=2.2.6`	`numpy>=2.2.6`
`pyarrow>=21.0.0`	`pyarrow>=21.0.0`
`requests>=2.32.0`	`requests>=2.32.0`
`rustbpe>=0.1.0`	`rustbpe>=0.1.0`
`tiktoken>=0.11.0`	`tiktoken>=0.11.0`
`kernels>=0.11.7`	`triton==3.1.0`
`matplotlib>=3.10.8`	`flash-attn==2.8.3`
`pandas>=2.3.3`	`ninja`

We employ the pipeline of Seo et al. (2025) to provide agents with useful context for the Autoresearch task. The `AutoregressiveLanguageModellingWithLiteratureAutoresearchBPB` task differs from the base version by one additional folder, `pwc/`, containing a `task_info.jsonl` file with structured information extracted from 41 curated research papers. Each entry includes the paper title, task description, input/output specifications, a detailed pipeline description, datasets used, model architecture details, training hyperparameters, and experiment metadata, extracted by GPT-5 from the papers' `.tex` files. The papers are organized into three categories: Architecture Improvements (20 papers), Training Strategies (17 papers), and Optimizers (5 papers), as detailed in Tables 16–18. For 14 of the 41 papers, working GitHub repositories are additionally provided in `pwc/code/`. Agents are instructed to consult the structured summaries to identify promising techniques, browse the reference code for implementation details, and prioritize low-cost modifications with high impact on convergence speed within the fixed 5-minute training budget. Of the 41 articles provided, 2 were published in 2021, 7 in 2023, 21 in 2024, 8 in 2025 and 3 in 2026.

Table 16 Architecture Improvement papers included in the literature review of the Autoresearch task (20 papers).

Paper	Relevance	Code
Better Faster LLMs (Gloeckle et al., 2024)	Predicts multiple future tokens per position during training.	✗
Differential Transformer (Ye et al., 2025)	Replaces softmax attention with differential attention to cancel noise.	✓
FlexAttention (Dong et al., 2024a)	A PyTorch API for implementing custom attention patterns at FlashAttention speed.	✗
Forgetting Transformer (Lin et al., 2025)	Replaces standard softmax attention with a gated variant.	✗
Gated Delta Networks (Yang et al., 2024)	Combines a data-dependent decay gate with the delta update rule.	✗
KV Shifting Attention (Xu et al., 2024)	Modifies the attention module to shift key/value positions.	✗
Mixture of Hidden Dimensions Transformer (Chen et al., 2024)	Selectively activates hidden sub-dimensions across layers.	✗
MoBA (Lu et al., 2025)	A drop-in, dynamically sparse self-attention mechanism.	✓
MobileLLM (Liu et al., 2024)	Trains compact decoder-only Transformer models optimized for on-device usage.	✓
Native Sparse Attention (Yuan et al., 2025)	Hardware-aligned sparse attention patterns natively trainable end-to-end.	✓

gMLPs (Liu et al., 2021a)	Implements gated MLPs as a drop-in replacement for self-attention.	✗
Quantizable Transformers (Bondarenko et al., 2023)	Removes activation outliers to enable cleaner quantization.	✗
ReLU^2 Wins (Zhang et al., 2024)	Shows squared ReLU produces naturally sparse hidden representations.	✗
ResiDual (Xie et al., 2023)	Dual-residual architecture combining Post-LN and Pre-LN strengths.	✗
RWKV-7 "Goose" (Peng et al., 2023a)	Recurrent architecture for dynamic, expressive state evolution.	✗
Stick-breaking Attention (Tan et al., 2024)	Replaces softmax with a sequential stick-breaking process.	✓
Titans (Behrouz et al., 2024)	Augments attention with a neural long-term memory module.	✗
Ultra-Sparse Memory (Huang et al., 2024)	Replaces MLP layers with ultra-sparse memory layers.	✗
YaRN (Peng et al., 2023b)	Extends effective context window via modified RoPE interpolation.	✗
μnit Scaling (Narayan et al., 2025)	Unit-scaled initialization for stable FP8 training.	✗

Table 17 Training Strategy papers included in the literature review of the Autoresearch task (16 papers).

Paper	Relevance	Code
A Spectral Condition for Feature Learning (Yang et al., 2023)	Establishes theoretical conditions for feature learning in wide MLPs. Informs LR and width scaling choices.	✗
AutoResearch-RL: Perpetual Self-Evaluating RL Agents (Jain et al., 2026)	Implements an agent that edits training scripts to optimize rewards. Discovered configurations matching hand-tuned baselines.	✗
COAT: Compressing Optimizer States and Activations (Xi et al., 2024)	Quantizes both optimizer states and activations to FP8; achieves near-lossless performance with reduced memory.	✓
Cramming: Training a Language Model on a Single GPU (Geiping and Goldstein, 2023)	Trains BERT-style models in 24 hours on one GPU. Relevant to fixed-budget optimization.	✓
Depth Up-Scaling (SOLAR 10.7B) (Kim et al., 2024)	Scales Transformers by duplicating and fine-tuning middle layers to avoid training from scratch.	✗
Liger Kernel: Efficient Triton Kernels (Hsu et al., 2024)	Drop-in fused Triton kernels (CrossEntropy, RMSNorm) that are 3× faster with lower memory.	✓
LLM Speedrunner (Zhao et al., 2025)	An agentic approach that modifies scripts to achieve lower validation cross-entropy for NanoGPT.	✓
MiniCPM: Small Language Models (Hu et al., 2024)	Uses Warmup-Stable-Decay (WSD) schedule to make 2.4B models perform like 7B models.	✗
modded-nanogpt (Jordan et al., 2024)	The gold standard for speedruns; uses Muon, ReLU^2, and sliding window attention.	✓
Muon is Scalable (Moonlight) (Liu et al., 2025)	Validates the Muon optimizer on 16B-parameter MoE models up to 5.7T tokens.	✓
Batch Size in Stochastic Conditional Gradient (Islamov et al., 2026)	Shows validation loss is batch-independent once B and S are large enough.	✗
Parameters vs FLOPs (MoE) (Abnar et al., 2025)	Derives scaling laws for optimal sparsity in mixture-of-experts models.	✗

Optimal Hyperparameter Scaling Law (Li et al., 2025)	Introduces the "Step Law" to predict optimal LR and batch size via power-law functions.	✗
Scaling Law with LR Annealing (Tissue et al., 2024)	Proposes a specific loss curve fit ($R^2 > 0.999$) for cooldown schedules.	✗
Scaling Laws Beyond Fixed Durations (Hägele et al., 2024)	Uses constant LR plus short cooldown to allow flexibility in training duration.	✗
The AutoResearch Moment (He et al., 2026)	Discusses the transition to autonomous research loops via `program_md` specifications.	✗

Table 18 Optimizer papers included in the literature review of the Autoresearch task (5 papers).

Paper	Relevance	Code
AdEMAMix: Adaptive EMA Mixture Optimizer (Pagliardini et al., 2024)	Augments AdamW with a second, very slow momentum ($\beta_3 \approx 0.9999$) to leverage old gradients while maintaining reactivity via a fast momentum. A 1.3B model matches AdamW validation loss in nearly half the tokens.	✗
Grokfast (Lee et al., 2024)	An optimizer-agnostic, 5-line gradient filtering method. Inserts between `loss.backward()` and `optimizer.step()` with no changes to the optimizer itself.	✓
Maximal Update Parametrization (μP) (Yang et al., 2021)	Enables transfer of optimal hyperparameters from small proxy models to large targets by scaling per-layer according to width.	✓
Schedule-Free Optimization (Defazio et al., 2024)	A drop-in replacement that eliminates the need for a pre-specified LR schedule. Maintains three parameter sequences (base z, evaluation x, gradient location y).	✓
Sophia: A Scalable Second-Order Optimizer (Liu et al., 2023)	Uses diagonal Hessian estimates with per-coordinate update clipping. Achieves $\sim$2$\times$ speedup over AdamW across GPT-2 and GPT-NeoX models.	✗

G AIRA-Design: Autoresearch - Additional Results

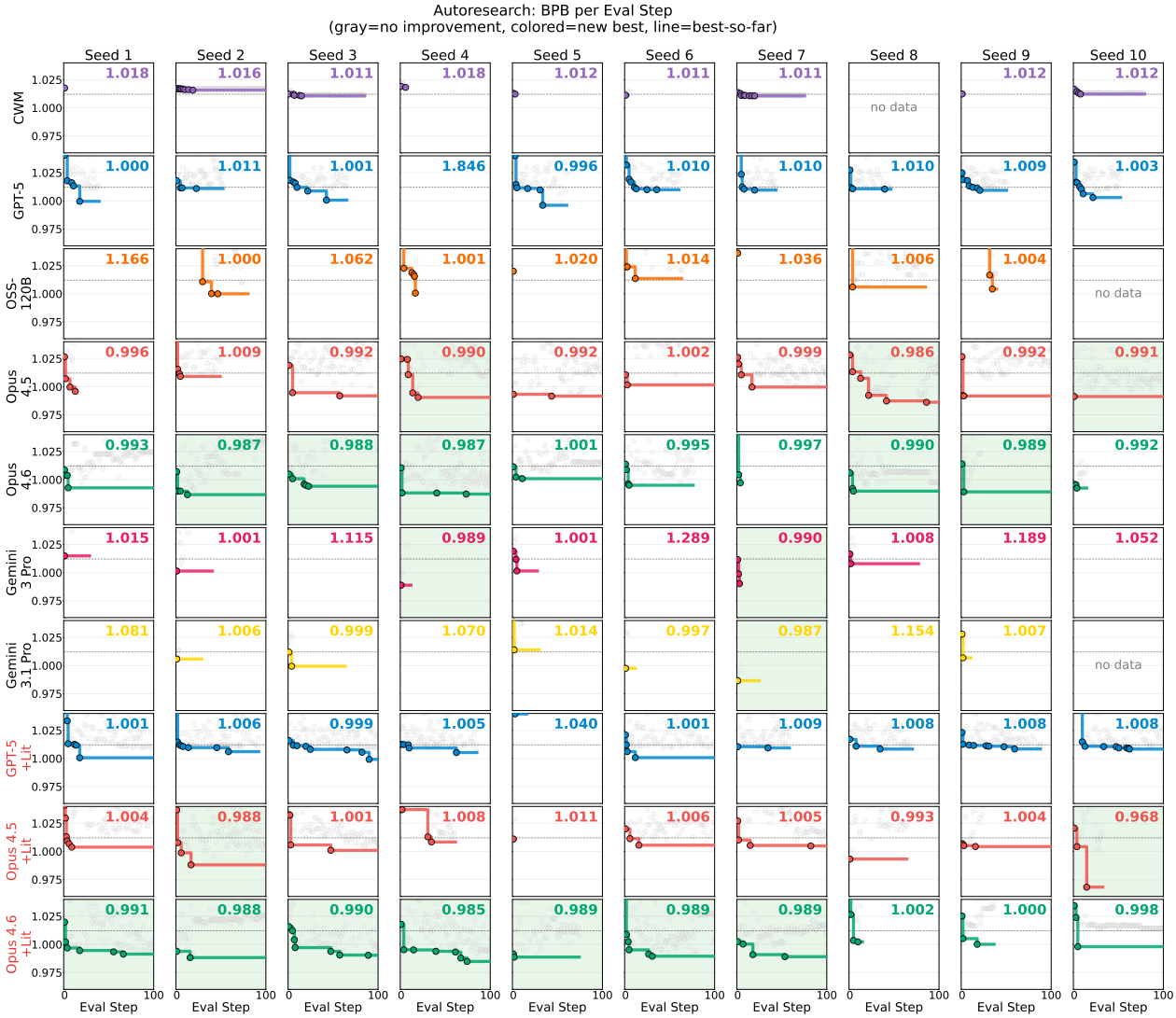


Figure 28 Autoresearch: BPB trajectories across 10 agents and 10 seeds. Each subplot shows the validation bits-per-byte at each evaluation step for a single agent–seed combination. Colored dots mark steps that set a new cumulative best (improvements), gray dots indicate non-improving steps, and the colored step line tracks the best-so-far BPB. The dashed line denotes the RADv1 baseline (BPB = 1.0121). Green-shaded cells indicate seeds where the agent improved over the baseline by at least 0.0205 BPB, which is the improvement obtained in Karpathy (2026). Each subplot is labelled with the final fitness value achieved by the agent.

We perform a structural feature analysis across all agents and 100 seeds. For each improvement step (i.e., each step that achieves a new cumulative-best BPB), we extract the generated `train.py` code, parse it into a structured feature vector covering 27 dimensions (MLP activation type, model depth, head dimension, window pattern, learning rates, regularization toggles, and others) and compare it against the previous best to identify which features changed. Because agents regenerate the entire file at each step, most improvement steps involve multiple simultaneous feature changes, making causal attribution challenging. We report the results at three levels of granularity: per-feature (Table 19), per-boolean-toggle (Table 20), and per-category (Table 21).

Across 156 total improvement steps, depth and width changes are the most frequent (46% of steps) with a median Δ of 0.007, while attention pattern changes yield the highest median Δ (0.009) but appear in fewer steps (37%). Learning rate modifications co-occur with nearly half of all improvements (49%), reflecting the tendency of agents to adjust hyperparameters alongside architectural changes. Among MLP type transitions, reverting to the baseline

Table 19 Per-feature BPB improvement aggregated across all agents. For each improvement step with positive ΔBPB over the previous best, we extract which features changed. "Count" is the number of such steps where the feature changed; "Solo" is the number where it was the *only* feature that changed.

Feature	Category	Count	Solo	Med. Δ	Mean Δ	Best Δ	Solo Med. Δ
parallel_residual	Embeddings / heads	2	0	0.0583	0.0583	0.0888	—
label_smoothing	Regularization	7	0	0.0291	0.0726	0.3240	—
n_kv_head	Depth / width	11	1	0.0278	0.0595	0.2614	0.0609
attn_dropout	Attention pattern	5	1	0.0113	0.0126	0.0291	0.0018
value_embeds	Embeddings / heads	17	0	0.0103	0.0421	0.3240	—
window_pattern	Attention pattern	49	0	0.0096	0.0302	0.3240	—
z_loss	Regularization	12	0	0.0094	0.0397	0.3240	—
partial_rope	Attention pattern	2	1	0.0083	0.0083	0.0137	0.0137
learned_prefix	Embeddings / heads	2	1	0.0075	0.0075	0.0137	0.0012
head_dim	Depth / width	22	1	0.0060	0.0202	0.1786	0.0024
depth	Depth / width	53	0	0.0058	0.0306	0.5226	—
mlp_type	MLP / activation	48	8	0.0050	0.0351	0.5226	0.0031
batch_size	Batch / curriculum	40	3	0.0043	0.0347	0.5226	0.0000
matrix_lr	Learning rates	58	1	0.0040	0.0168	0.3240	0.0000
grad_clipping	Regularization	16	1	0.0038	0.0114	0.0418	0.0007
embedding_lr	Learning rates	57	1	0.0038	0.0161	0.3240	0.0005
curriculum	Batch / curriculum	2	0	0.0035	0.0035	0.0061	—
weight_decay	Learning rates	46	0	0.0034	0.0170	0.3240	—
dual_rope_bases	Attention pattern	1	0	0.0034	0.0034	0.0034	—
ema	Regularization	10	0	0.0033	0.0076	0.0299	—
aux_heads	Embeddings / heads	4	0	0.0030	0.0061	0.0179	—
token_corruption	Regularization	2	0	0.0028	0.0028	0.0048	—
pos_loss_weight	Batch / curriculum	2	0	0.0024	0.0024	0.0037	—
weight_tying	Embeddings / heads	11	0	0.0018	0.0466	0.3240	—
attn_out_norm	Other	1	1	0.0012	0.0012	0.0012	0.0012
agc	Regularization	2	1	0.0010	0.0010	0.0013	0.0007
attn_temperature	Attention pattern	3	0	0.0009	0.0024	0.0062	—

Table 20 Impact of adding vs. removing boolean features. Only improvement steps with positive ΔBPB are included.

Feature	Add #	Add med. Δ	Rem #	Rem med. Δ	Verdict
agc	1	0.0013	1	0.0007	add helps more
attn_dropout	2	0.0154	3	0.0113	add helps more
attn_out_norm	1	0.0012	0	—	add only
attn_temperature	2	0.0036	1	0.0002	add helps more
aux_heads	2	0.0113	2	0.0009	add helps more
curriculum	1	0.0061	1	0.0008	add helps more
dual_rope_bases	1	0.0034	0	—	add only
ema	4	0.0148	6	0.0023	add helps more
grad_clipping	7	0.0015	9	0.0040	remove helps more
label_smoothing	2	0.1829	5	0.0228	add helps more
learned_prefix	1	0.0012	1	0.0137	remove helps more
parallel_residual	1	0.0888	1	0.0278	add helps more
partial_rope	1	0.0028	1	0.0137	remove helps more
pos_loss_weight	1	0.0011	1	0.0037	remove helps more
token_corruption	1	0.0008	1	0.0048	remove helps more
value_embeds	11	0.0136	6	0.0006	add helps more
weight_tying	5	0.0018	6	0.0025	remove helps more
z_loss	5	0.0048	7	0.0127	remove helps more

Table 21 Category-level summary of BPB improvements. "% of improvements" is the fraction of all positive-Δ steps involving at least one feature from that category. Categories are sorted by decreasing median Δ.

Category	Count	% of improvements	Med. Δ	Mean Δ	Best Δ
Attention pattern	59	37%	0.0087	0.0266	0.3240
Depth / width	74	46%	0.0072	0.0259	0.5226
MLP / activation	48	30%	0.0050	0.0351	0.5226
Embeddings / heads	34	21%	0.0044	0.0312	0.3240
Regularization	31	19%	0.0040	0.0214	0.3240
Batch / curriculum	42	26%	0.0039	0.0332	0.5226
Learning rates	78	49%	0.0036	0.0142	0.3240

ReLU^2 from alternative activations consistently yields larger gains than the reverse direction. The most frequent transitions are ReLU^2 $\leftrightarrow$ SwiGLU (14 and 13 occurrences respectively), but SwiGLU $\rightarrow$ ReLU^2 has a lower median Δ (0.003) than ReLU^2 $\rightarrow$ SwiGLU (0.007), suggesting that SwiGLU attempts are often exploratory and the revert merely restores a working configuration. Among boolean features (Table 20), adding value embeddings (median Δ=0.014, 11 occurrences) and EMA (median Δ=0.015, 4 occurrences) are the most consistently beneficial additions, while removing z-loss (median Δ=0.013) and gradient clipping (median Δ=0.004) helps more than adding them. The "Solo" column in Table 19 reveals that only 7 out of 27 features were ever the sole change in an improvement step, underscoring that agents predominantly make compound modifications. This is a direct consequence of the full-file regeneration paradigm, which prevents the surgical, single-variable edits that would enable cleaner ablation.

H Best Autoresearch Solution (Greedy Opus 4.5 + Literature access)

```python
import gc
import math
import time
from dataclasses import dataclass, asdict

import torch
import torch.nn as nn
import torch.nn.functional as F
from flash_attn import flash_attn_func

from prepare import MAX_SEQ_LEN, TIME_BUDGET, Tokenizer, make_dataloader, evaluate_bpb

# ---------------------------------------------------------------------------
# GPT Model
# ---------------------------------------------------------------------------

@dataclass
class GPTConfig:
    sequence_len: int = 2048
    vocab_size: int = 32768
    n_layer: int = 12
    n_head: int = 6
    n_kv_head: int = 6
    n_embd: int = 768
    window_pattern: str = "SSSL"


def norm(x):
    return F.rms_norm(x, (x.size(-1),))


def has_ve(layer_idx, n_layer):
    return layer_idx % 2 == (n_layer - 1) % 2


def apply_rotary_emb(x, cos, sin):
    assert x.ndim == 4
    d = x.shape[3] // 2
    x1, x2 = x[..., :d], x[..., d:]
    y1 = x1 * cos + x2 * sin
    y2 = x1 * (-sin) + x2 * cos
    return torch.cat([y1, y2], 3)


class CausalSelfAttention(nn.Module):
    def __init__(self, config, layer_idx):
        super().__init__()
        self.n_head = config.n_head
        self.n_kv_head = config.n_kv_head
        self.n_embd = config.n_embd
        self.head_dim = self.n_embd // self.n_head
        assert self.n_embd % self.n_head == 0
        assert self.n_kv_head <= self.n_head and self.n_head % self.n_kv_head == 0
        self.c_q = nn.Linear(self.n_embd, self.n_head * self.head_dim, bias=False)
        self.c_k = nn.Linear(self.n_embd, self.n_kv_head * self.head_dim, bias=False)
        self.c_v = nn.Linear(self.n_embd, self.n_kv_head * self.head_dim, bias=False)
        self.c_proj = nn.Linear(self.n_embd, self.n_embd, bias=False)
        self.ve_gate_channels = 32
        self.ve_gate = nn.Linear(self.ve_gate_channels, self.n_kv_head, bias=False) if has_ve(layer_idx, config.n_layer) else None

    def forward(self, x, ve, cos_sin, window_size):
        B, T, C = x.size()
        q = self.c_q(x).view(B, T, self.n_head, self.head_dim)
        k = self.c_k(x).view(B, T, self.n_kv_head, self.head_dim)
        v = self.c_v(x).view(B, T, self.n_kv_head, self.head_dim)

        if ve is not None:
            ve = ve.view(B, T, self.n_kv_head, self.head_dim)
            gate = 2 * torch.sigmoid(self.ve_gate(x[..., :self.ve_gate_channels]))
            v = v + gate.unsqueeze(-1) * ve

        cos, sin = cos_sin
        q, k = apply_rotary_emb(q, cos, sin), apply_rotary_emb(k, cos, sin)
        q, k = norm(q), norm(k)

        y = flash_attn_func(q, k, v, causal=True, window_size=window_size)
        y = y.contiguous().view(B, T, -1)
        y = self.c_proj(y)
        return y


class MLP(nn.Module):
    def __init__(self, config):
        super().__init__()
        self.c_fc = nn.Linear(config.n_embd, 4 * config.n_embd, bias=False)
        self.c_proj = nn.Linear(4 * config.n_embd, config.n_embd, bias=False)

    def forward(self, x):
        x = self.c_fc(x)
        x = F.relu(x).square()
        x = self.c_proj(x)
        return x


class Block(nn.Module):
    def __init__(self, config, layer_idx):
        super().__init__()
        self.attn = CausalSelfAttention(config, layer_idx)
        self.mlp = MLP(config)

    def forward(self, x, ve, cos_sin, window_size):
        x = x + self.attn(norm(x), ve, cos_sin, window_size)
        x = x + self.mlp(norm(x))
        return x
```

```python


class GPT(nn.Module):
    def __init__(self, config):
        super().__init__()
        self.config = config
        self.window_sizes = self._compute_window_sizes(config)
        self.transformer = nn.ModuleDict({
            "wte": nn.Embedding(config.vocab_size, config.n_embd),
            "h": nn.ModuleList([Block(config, i) for i in range(config.n_layer)]),
        })
        self.lm_head = nn.Linear(config.n_embd, config.vocab_size, bias=False)
        self.resid_lambdas = nn.Parameter(torch.ones(config.n_layer))
        self.x0_lambdas = nn.Parameter(torch.zeros(config.n_layer))
        head_dim = config.n_embd // config.n_head
        kv_dim = config.n_kv_head * head_dim
        self.value_embeds = nn.ModuleDict({
            str(i): nn.Embedding(config.vocab_size, kv_dim)
            for i in range(config.n_layer) if has_ve(i, config.n_layer)
        })
        self.rotary_seq_len = config.sequence_len * 10
        cos, sin = self._precompute_rotary_embeddings(self.rotary_seq_len, head_dim)
        self.register_buffer("cos", cos, persistent=False)
        self.register_buffer("sin", sin, persistent=False)

    @torch.no_grad()
    def init_weights(self):
        # Optimized embedding initialization with smaller std for faster convergence
        torch.nn.init.normal_(self.transformer.wte.weight, mean=0.0, std=0.5)
        torch.nn.init.normal_(self.lm_head.weight, mean=0.0, std=0.001)
        n_embd = self.config.n_embd
        s = 3**0.5 * n_embd**-0.5
        for block in self.transformer.h:
            torch.nn.init.uniform_(block.attn.c_q.weight, -s, s)
            torch.nn.init.uniform_(block.attn.c_k.weight, -s, s)
            torch.nn.init.uniform_(block.attn.c_v.weight, -s, s)
            torch.nn.init.zeros_(block.attn.c_proj.weight)
            torch.nn.init.uniform_(block.mlp.c_fc.weight, -s, s)
            torch.nn.init.zeros_(block.mlp.c_proj.weight)
        self.resid_lambdas.fill_(1.0)
        self.x0_lambdas.fill_(0.1)
        for ve in self.value_embeds.values():
            torch.nn.init.uniform_(ve.weight, -s, s)
        for block in self.transformer.h:
            if block.attn.ve_gate is not None:
                torch.nn.init.zeros_(block.attn.ve_gate.weight)
        head_dim = self.config.n_embd // self.config.n_head
        cos, sin = self._precompute_rotary_embeddings(self.rotary_seq_len, head_dim)
        self.cos, self.sin = cos, sin
        self.transformer.wte.to(dtype=torch.bfloat16)
        for ve in self.value_embeds.values():
            ve.to(dtype=torch.bfloat16)

    def _precompute_rotary_embeddings(self, seq_len, head_dim, base=10000, device=None):
        if device is None:
            device = self.transformer.wte.weight.device
        channel_range = torch.arange(0, head_dim, 2, dtype=torch.float32, device=device)
        inv_freq = 1.0 / (base ** (channel_range / head_dim))
        t = torch.arange(seq_len, dtype=torch.float32, device=device)
        freqs = torch.outer(t, inv_freq)
        cos, sin = freqs.cos(), freqs.sin()
        cos, sin = cos.bfloat16(), sin.bfloat16()
        cos, sin = cos[None, :, None, :], sin[None, :, None, :]
        return cos, sin

    def _compute_window_sizes(self, config):
        pattern = config.window_pattern.upper()
        assert all(c in "SL" for c in pattern)
        long_window = config.sequence_len
        short_window = long_window // 2
        char_to_window = {"L": (long_window, 0), "S": (short_window, 0)}
        window_sizes = []
        for layer_idx in range(config.n_layer):
            char = pattern[layer_idx % len(pattern)]
            window_sizes.append(char_to_window[char])
        window_sizes[-1] = (long_window, 0)
        return window_sizes

    def estimate_flops(self):
        nparams = sum(p.numel() for p in self.parameters())
        value_embeds_numel = sum(ve.weight.numel() for ve in self.value_embeds.values())
        nparams_exclude = (self.transformer.wte.weight.numel() + value_embeds_numel +
                           self.resid_lambdas.numel() + self.x0_lambdas.numel())
        h = self.config.n_head
        q = self.config.n_embd // self.config.n_head
        t = self.config.sequence_len
        attn_flops = 0
        for window_size in self.window_sizes:
            window = window_size[0]
            effective_seq = t if window < 0 else min(window, t)
            attn_flops += 12 * h * q * effective_seq
        return 6 * (nparams - nparams_exclude) + attn_flops

    def num_scaling_params(self):
        wte = sum(p.numel() for p in self.transformer.wte.parameters())
        value_embeds = sum(p.numel() for p in self.value_embeds.parameters())
        lm_head = sum(p.numel() for p in self.lm_head.parameters())
        transformer_matrices = sum(p.numel() for p in self.transformer.h.parameters())
        scalars = self.resid_lambdas.numel() + self.x0_lambdas.numel()
        total = wte + value_embeds + lm_head + transformer_matrices + scalars
        return {
            'wte': wte, 'value_embeds': value_embeds, 'lm_head': lm_head,
            'transformer_matrices': transformer_matrices, 'scalars': scalars, 'total': total,
        }

    def setup_optimizer(self, unembedding_lr=0.004, embedding_lr=0.2, matrix_lr=0.02,
                        weight_decay=0.0, adam_betas=(0.8, 0.95), scalar_lr=0.5):
        model_dim = self.config.n_embd
        matrix_params = list(self.transformer.h.parameters())
```

```
        value_embeds_params = list(self.value_embeds.parameters())
        embedding_params = list(self.transformer.wte.parameters())
        lm_head_params = list(self.lm_head.parameters())
        resid_params = [self.resid_lambdas]
        x0_params = [self.x0_lambdas]
        assert len(list(self.parameters())) == (len(matrix_params) + len(embedding_params) +
            len(lm_head_params) + len(value_embeds_params) + len(resid_params) + len(x0_params))
        dmodel_lr_scale = (model_dim / 768) ** -0.5
        print(f"Scaling AdamW LRs by 1/sqrt({model_dim}/768) = {dmodel_lr_scale:.6f}")
        param_groups = [
            dict(kind='adamw', params=lm_head_params, lr=unembedding_lr * dmodel_lr_scale, betas=adam_betas, eps=1e-10, weight_decay=0.0),
            dict(kind='adamw', params=embedding_params, lr=embedding_lr * dmodel_lr_scale, betas=adam_betas, eps=1e-10, weight_decay=0.0),
            dict(kind='adamw', params=value_embeds_params, lr=embedding_lr * dmodel_lr_scale, betas=adam_betas, eps=1e-10, weight_decay=0.0),
            dict(kind='adamw', params=resid_params, lr=scalar_lr * 0.01, betas=adam_betas, eps=1e-10, weight_decay=0.0),
            dict(kind='adamw', params=x0_params, lr=scalar_lr, betas=(0.96, 0.95), eps=1e-10, weight_decay=0.0),
        ]
        for shape in sorted({p.shape for p in matrix_params}):
            group_params = [p for p in matrix_params if p.shape == shape]
            param_groups.append(dict(
                kind='muon', params=group_params, lr=matrix_lr,
                momentum=0.95, ns_steps=4, beta2=0.95, weight_decay=weight_decay,
            ))
        optimizer = MuonAdamW(param_groups)
        for group in optimizer.param_groups:
            group["initial_lr"] = group["lr"]
        return optimizer

    def forward(self, idx, targets=None, reduction='mean'):
        B, T = idx.size()
        assert T <= self.cos.size(1)
        cos_sin = self.cos[:, :T], self.sin[:, :T]
        x = self.transformer.wte(idx)
        x = norm(x)
        x0 = x
        for i, block in enumerate(self.transformer.h):
            x = self.resid_lambdas[i] * x + self.x0_lambdas[i] * x0
            ve = self.value_embeds[str(i)](idx) if str(i) in self.value_embeds else None
            x = block(x, ve, cos_sin, self.window_sizes[i])
        x = norm(x)
        softcap = 18
        logits = self.lm_head(x)
        logits = logits.float()
        logits = softcap * torch.tanh(logits / softcap)
        if targets is not None:
            ce_loss = F.cross_entropy(logits.view(-1, logits.size(-1)), targets.view(-1),
                                      ignore_index=-1, reduction='none')
            probs = F.softmax(logits.view(-1, logits.size(-1)), dim=-1)
            targets_flat = targets.view(-1)
            valid_mask = targets_flat != -1
            pt = torch.zeros_like(ce_loss)
            pt[valid_mask] = probs[valid_mask].gather(1, targets_flat[valid_mask].unsqueeze(-1)).squeeze(-1)
            gamma = 1.0
            focal_weight = (1 - pt) ** gamma
            focal_loss = focal_weight * ce_loss

            if reduction == 'mean':
                return focal_loss[valid_mask].mean() if valid_mask.any() else focal_loss.mean()
            elif reduction == 'none':
                return focal_loss.view(B, T)
            else:
                return focal_loss.sum()
        return logits

# -----------------------------------------------------------------------------
# Optimizer (MuonAdamW)
# -----------------------------------------------------------------------------

polar_express_coeffs = [
    (8.156554524902461, -22.48329292557795, 15.878769915207462),
    (4.042929935166739, -2.808917465908714, 0.5000178451051316),
    (3.8916678022926607, -2.772484153217685, 0.5060648178503393),
    (3.285753657755655, -2.3681294933425376, 0.46449024233003106),
    (2.3465413258596377, -1.7097828382687081, 0.42323551169305323),
]

@torch.compile(dynamic=False, fullgraph=True)
def adamw_step_fused(p, grad, exp_avg, exp_avg_sq, step_t, lr_t, beta1_t, beta2_t, eps_t, wd_t):
    p.mul_(1 - lr_t * wd_t)
    exp_avg.lerp_(grad, (1 - beta1_t).to(exp_avg.dtype))
    exp_avg_sq.lerp_(grad.square(), (1 - beta2_t).to(exp_avg_sq.dtype))
    bias1 = 1 - beta1_t ** step_t
    bias2 = 1 - beta2_t ** step_t
    denom = (exp_avg_sq / bias2).sqrt() + eps_t
    step_size = lr_t / bias1
    p.add_(exp_avg / denom, alpha=-step_size)

@torch.compile(dynamic=False, fullgraph=True)
def muon_step_fused(stacked_grads, stacked_params, momentum_buffer, second_momentum_buffer,
                    momentum_t, lr_t, wd_t, beta2_t, ns_steps, red_dim):
    momentum = momentum_t.to(stacked_grads.dtype)
    momentum_buffer.lerp_(stacked_grads, (1 - momentum).to(momentum_buffer.dtype))
    g = stacked_grads.lerp_(momentum_buffer, momentum)
    X = g.bfloat16()
    X = X / (X.norm(dim=(-2, -1), keepdim=True) * 1.02 + 1e-6)
    if g.size(-2) > g.size(-1):
        for a, b, c in polar_express_coeffs[:ns_steps]:
            A = X.mT @ X
            B = b * A + c * (A @ A)
            X = a * X + X @ B
    else:
        for a, b, c in polar_express_coeffs[:ns_steps]:
            A = X @ X.mT
            B = b * A + c * (A @ A)
            X = a * X + B @ X
    g = X
    beta2 = beta2_t.to(g.dtype)
    v_mean = g.float().square().mean(dim=red_dim, keepdim=True)
    red_dim_size = g.size(red_dim)
    v_norm_sq = v_mean.sum(dim=(-2, -1), keepdim=True) * red_dim_size
```

```
    v_norm = v_norm_sq.sqrt()
    second_momentum_buffer.lerp_(v_mean.to(dtype=second_momentum_buffer.dtype), (1 - beta2).to(second_momentum_buffer.dtype))
    step_size = second_momentum_buffer.clamp_min(1e-10).rsqrt()
    scaled_sq_sum = (v_mean * red_dim_size) * step_size.float().square()
    v_norm_new = scaled_sq_sum.sum(dim=(-2, -1), keepdim=True).sqrt()
    final_scale = step_size * (v_norm / v_norm_new.clamp_min(1e-10))
    g = g * final_scale.to(g.dtype)
    lr = lr_t.to(g.dtype)
    wd = wd_t.to(g.dtype)
    mask = (g * stacked_params) >= 0
    stacked_params.sub_(lr * g + lr * wd * stacked_params * mask)


class MuonAdamW(torch.optim.Optimizer):
    def __init__(self, param_groups):
        super().__init__(param_groups, defaults={})
        self._adamw_step_t = torch.tensor(0.0, dtype=torch.float32, device="cpu")
        self._adamw_lr_t = torch.tensor(0.0, dtype=torch.float32, device="cpu")
        self._adamw_beta1_t = torch.tensor(0.0, dtype=torch.float32, device="cpu")
        self._adamw_beta2_t = torch.tensor(0.0, dtype=torch.float32, device="cpu")
        self._adamw_eps_t = torch.tensor(0.0, dtype=torch.float32, device="cpu")
        self._adamw_wd_t = torch.tensor(0.0, dtype=torch.float32, device="cpu")
        self._muon_momentum_t = torch.tensor(0.0, dtype=torch.float32, device="cpu")
        self._muon_lr_t = torch.tensor(0.0, dtype=torch.float32, device="cpu")
        self._muon_wd_t = torch.tensor(0.0, dtype=torch.float32, device="cpu")
        self._muon_beta2_t = torch.tensor(0.0, dtype=torch.float32, device="cpu")

    def _step_adamw(self, group):
        for p in group['params']:
            if p.grad is None: continue
            grad = p.grad
            state = self.state[p]
            if not state:
                state['step'] = 0
                state['exp_avg'] = torch.zeros_like(p)
                state['exp_avg_sq'] = torch.zeros_like(p)
            state['step'] += 1
            self._adamw_step_t.fill_(state['step'])
            self._adamw_lr_t.fill_(group['lr'])
            self._adamw_beta1_t.fill_(group['betas'][0])
            self._adamw_beta2_t.fill_(group['betas'][1])
            self._adamw_eps_t.fill_(group['eps'])
            self._adamw_wd_t.fill_(group['weight_decay'])
            adamw_step_fused(p, grad, state['exp_avg'], state['exp_avg_sq'],
                             self._adamw_step_t, self._adamw_lr_t, self._adamw_beta1_t,
                             self._adamw_beta2_t, self._adamw_eps_t, self._adamw_wd_t)

    def _step_muon(self, group):
        params = group['params']
        if not params: return
        p = params[0]
        state = self.state[p]
        num_params = len(params)
        shape, device, dtype = p.shape, p.device, p.dtype
        if "momentum_buffer" not in state:
            state["momentum_buffer"] = torch.zeros(num_params, *shape, dtype=dtype, device=device)
        if "second_momentum_buffer" not in state:
            state_shape = (num_params, shape[-2], 1) if shape[-2] >= shape[-1] else (num_params, 1, shape[-1])
            state["second_momentum_buffer"] = torch.zeros(state_shape, dtype=dtype, device=device)
        red_dim = -1 if shape[-2] >= shape[-1] else -2
        stacked_grads = torch.stack([p.grad for p in params])
        stacked_params = torch.stack(params)
        self._muon_momentum_t.fill_(group["momentum"])
        self._muon_beta2_t.fill_(group["beta2"] if group["beta2"] is not None else 0.0)
        self._muon_lr_t.fill_(group["lr"] * max(1.0, shape[-2] / shape[-1])**0.5)
        self._muon_wd_t.fill_(group["weight_decay"])
        muon_step_fused(stacked_grads, stacked_params,
                        state["momentum_buffer"], state["second_momentum_buffer"],
                        self._muon_momentum_t, self._muon_lr_t, self._muon_wd_t,
                        self._muon_beta2_t, group["ns_steps"], red_dim)
        torch._foreach_copy_(params, list(stacked_params.unbind(0)))

    @torch.no_grad()
    def step(self):
        for group in self.param_groups:
            if group['kind'] == 'adamw': self._step_adamw(group)
            elif group['kind'] == 'muon': self._step_muon(group)

# -----------------------------------------------------------------------------
# Training Logic
# -----------------------------------------------------------------------------

ASPECT_RATIO = 56
HEAD_DIM = 128
WINDOW_PATTERN = "SSSL"
TOTAL_BATCH_SIZE = 2**19
EMBEDDING_LR = 0.85
UNEMBEDDING_LR = 0.006
MATRIX_LR = 0.048
SCALAR_LR = 0.5
WEIGHT_DECAY = 0.18
ADAM_BETAS = (0.8, 0.95)
WARMUP_RATIO = 0.0
WARMDOWN_RATIO = 0.55
FINAL_LR_FRAC = 0.0
DEPTH = 8
DEVICE_BATCH_SIZE = 128

t_start = time.time()
torch.manual_seed(42)
torch.set_float32_matmul_precision("high")
device = torch.device("cuda")
autocast_ctx = torch.amp.autocast(device_type="cuda", dtype=torch.bfloat16)
H100_BF16_PEAK_FLOPS = 989.5e12

tokenizer = Tokenizer.from_directory()
vocab_size = tokenizer.get_vocab_size()

def build_model_config(depth):
```

```python
    base_dim = depth * ASPECT_RATIO
    model_dim = ((base_dim + HEAD_DIM - 1) // HEAD_DIM) * HEAD_DIM
    num_heads = model_dim // HEAD_DIM
    return GPTConfig(
        sequence_len=MAX_SEQ_LEN, vocab_size=vocab_size,
        n_layer=depth, n_head=num_heads, n_kv_head=num_heads, n_embd=model_dim,
        window_pattern=WINDOW_PATTERN,
    )

config = build_model_config(DEPTH)
with torch.device("meta"):
    model = GPT(config)
model.to_empty(device=device)
model.init_weights()
num_params = model.num_scaling_params()['total']
num_flops_per_token = model.estimate_flops()
grad_accum_steps = TOTAL_BATCH_SIZE // (DEVICE_BATCH_SIZE * MAX_SEQ_LEN)

optimizer = model.setup_optimizer(
    unembedding_lr=UNEMBEDDING_LR, embedding_lr=EMBEDDING_LR,
    scalar_lr=SCALAR_LR, adam_betas=ADAM_BETAS,
    matrix_lr=MATRIX_LR, weight_decay=WEIGHT_DECAY,
)
model = torch.compile(model, dynamic=False)
train_loader = make_dataloader(tokenizer, DEVICE_BATCH_SIZE, MAX_SEQ_LEN, "train")
x, y, epoch = next(train_loader)

def get_lr_multiplier(progress):
    if progress < 1.0 - WARMDOWN_RATIO: return 1.0
    cooldown = (1.0 - progress) / WARMDOWN_RATIO
    return cooldown * 1.0 + (1 - cooldown) * FINAL_LR_FRAC

t_start_training = time.time()
smooth_train_loss = 0
total_training_time = 0
step = 0

while True:
    torch.cuda.synchronize()
    t0 = time.time()
    for _ in range(grad_accum_steps):
        with autocast_ctx: loss = model(x, y)
        train_loss = loss.detach()
        (loss / grad_accum_steps).backward()
        x, y, epoch = next(train_loader)

    progress = min(total_training_time / TIME_BUDGET, 1.0)
    lrm = get_lr_multiplier(progress)
    for group in optimizer.param_groups:
        group["lr"] = group["initial_lr"] * lrm
        if group['kind'] == 'muon':
            group["momentum"] = (1 - min(step/300, 1)) * 0.85 + min(step/300, 1) * 0.95
            group["weight_decay"] = WEIGHT_DECAY * (1 - progress)
    optimizer.step()
    model.zero_grad(set_to_none=True)

    torch.cuda.synchronize()
    t1 = time.time()
    dt = t1 - t0
    if step > 10: total_training_time += dt
    smooth_train_loss = 0.9 * smooth_train_loss + 0.1 * train_loss.item()
    step += 1
    if step > 10 and total_training_time >= TIME_BUDGET: break

model.eval()
with autocast_ctx:
    val_bpb = evaluate_bpb(model, tokenizer, DEVICE_BATCH_SIZE)

print(f"val_bpb: {val_bpb:.6f}")
```